학습 교안

데이터 분석 용어

초보자를 위한 쉬운 풀이

기초 용어·통계·전처리·EDA·머신러닝·평가·KPI·분석 흐름·pandas
그리고 딥러닝 학습·프롬프트 엔지니어링 용어까지
12개 장 · 용어 161개 · 그림 40개 · 예제 123개 (실제 실행 결과 포함) · 퀴즈 74문제

강사 김대철

학습 앱: https://sinaboro.github.io/data-terms-lab/

이 교안의 사용법

이 교안은 데이터 분석을 처음 공부하는 사람이 **용어 때문에 막히지 않도록** 만들었어요. 용어마다 **한 줄 뜻 → 쉬운 설명과 비유 → 그림 → 실행 예제** 순서로 풀었습니다. 예제는 모두 실제로 실행해서 얻은 결과(pandas 3 기준)를 실었어요.

표시	뜻
용어 상자	절 제목 아래 초록 상자: 영어 이름과 한 줄 뜻. 먼저 이것만 외워도 좋아요
파이썬 코드	직접 실행해 볼 pandas·scikit-learn 코드 (초록 줄). 같은 장의 코드는 앞에서부터 이어서 실행돼요
실행 결과	그 코드를 실제로 실행한 결과 (주황 줄)
TIP	쉬운 비유, 알아 두면 좋은 내용
주의	자주 하는 오해·실수
총정리 퀴즈	장 끝의 확인 문제. 정답과 해설은 맨 뒤에 있어요
용어 찾아보기	맨 뒤 가나다순 용어 목록. 모르는 말이 나오면 여기서 절 번호를 찾으세요

학습 앱 교안과 똑같은 내용을 담은 학습 앱이 있어요. 스마트폰이나 PC 브라우저에서 https://sinaboro.github.io/data-terms-lab/ 을 열면 용어를 검색하고, 카드 퀴즈로 외우고, 예제 코드를 고쳐 가며 바로 실행할 수 있어요. 카카오톡으로 링크를 받아 홈 화면에 설치하면 앱처럼 쓸 수 있습니다.

공부 순서 추천

① 용어 상자의 한 줄 뜻을 소리 내어 읽고 → ② 설명과 그림으로 **왜 그런 이름인지** 이해하고 → ③ 예제를 실행해 숫자를 바꿔 보고 → ④ 장 끝 퀴즈와 앱의 카드 퀴즈로 확인하세요. 8장의 전체 흐름 그림을 먼저 보고 시작해도 좋아요.

목차

이 교안의 사용법 2

01장 데이터 분석 기초 용어 9

01.0 들어가며: 엑셀 표 하나로 시작하기 9

01.1 데이터셋 · Dataset 9

01.2 피처 · Feature 9

01.3 타깃 · Target 10

01.4 관측치 · Observation 10

01.5 변수 · Variable 10

01.6 범주형 데이터 · Categorical Data 10

01.7 수치형 데이터 · Numerical Data 11

01.8 결측치 · Missing Value 11

01.9 이상치 · Outlier 12

01.10 중복 데이터 · Duplicate Data 13

02장 기초 통계 용어 14

02.0 들어가며: 숫자 여러 개를 숫자 하나로 14

02.1 평균 · Mean 14

02.2 중앙값 · Median 14

02.3 최빈값 · Mode 15

02.4 분산 · Variance 15

02.5 표준편차 · Standard Deviation 16

02.6 최솟값 · 최댓값 · Minimum · Maximum 16

02.7 사분위수 · Quartile 17

02.8 백분위수 · Percentile 17

02.9 상관계수 · Correlation Coefficient 18

02.10 왜도 · Skewness 18

02.11 첨도 · Kurtosis 19

02.12 정규분포 · Normal Distribution 19

03장 데이터 전처리 용어 21

03.1 전처리 · Preprocessing 21

03.2 정규화 · Normalization 21

03.3 표준화 · Standardization 22

03.4 인코딩 · Encoding 22

03.5 원-핫 인코딩 · One-Hot Encoding . . . 22
03.6 레이블 인코딩 · Label Encoding . . . 23
03.7 데이터 병합 · Merge / Join . . . 23
03.8 그룹화 · Grouping . . . 23
03.9 집계 · Aggregation . . . 24
03.10 피처 엔지니어링 · Feature Engineering . . . 24
03.11 데이터 누수 · Data Leakage . . . 24
04장 탐색적 데이터 분석(EDA) 용어 . . . 26
04.0 들어가며: 탐색적 데이터 분석 · EDA . . . 26
04.1 분포 · Distribution . . . 27
04.2 히스토그램 · Histogram . . . 27
04.3 산점도 · Scatter Plot . . . 28
04.4 박스플롯 · Box Plot . . . 28
04.5 히트맵 · Heatmap . . . 29
04.6 상관분석 · Correlation Analysis . . . 30
04.7 교차표 · Crosstab . . . 30
05장 머신러닝 기초 용어 . . . 32
05.0 들어가며: 머신러닝이란? . . . 32
05.1 지도학습 · Supervised Learning . . . 32
05.1.1 선형 회귀 · Linear Regression . . . 33
05.1.2 로지스틱 회귀 · Logistic Regression . . . 33
05.1.3 K-최근접 이웃 · KNN . . . 34
05.1.4 결정 트리 · Decision Tree . . . 34
05.1.5 랜덤 포레스트 · Random Forest . . . 35
05.1.6 그레이디언트 부스팅 · Gradient Boosting . . . 35
05.1.7 서포트 벡터 머신 · SVM . . . 36
05.1.8 나이브 베이즈 · Naive Bayes . . . 36
05.1.9 신경망 · Neural Network . . . 36
05.2 비지도학습 · Unsupervised Learning . . . 37
05.2.1 K-평균 군집 · K-Means . . . 38
05.2.2 계층적 군집 · Hierarchical Clustering . . . 38
05.2.3 DBSCAN . . . 39
05.2.4 주성분 분석 · PCA . . . 39
05.2.5 t-SNE . . . 39
05.2.6 연관 규칙 · Association Rules . . . 40

05.2.7 이상 탐지 · Anomaly Detection . . . 40
05.3 분류 · Classification . . . 41
05.4 회귀 · Regression . . . 41
05.5 군집화 · Clustering . . . 42
05.6 차원 축소 · Dimensionality Reduction . . . 42
05.7 훈련 / 검증 / 테스트 데이터 · Train / Validation / Test . . . 43
05.8 과적합 · Overfitting . . . 44
05.9 과소적합 · Underfitting . . . 44
05.10 하이퍼파라미터 · Hyperparameter . . . 44
06장 모델 평가 용어 . . . 47
06.0 들어가며: 모델 성적표 . . . 47
06.1 혼동행렬 · Confusion Matrix . . . 47
06.2 정확도 · Accuracy . . . 48
06.3 정밀도 · Precision . . . 48
06.4 재현율 · Recall . . . 48
06.5 F1-score . . . 49
06.6 ROC-AUC . . . 49
06.7 MAE · 평균 절대 오차 . . . 50
06.8 MSE · 평균 제곱 오차 . . . 51
06.9 RMSE · 평균 제곱근 오차 . . . 51
06.10 R^2 · 결정계수 . . . 51
07장 비즈니스·KPI 용어 . . . 53
07.0 들어가며: 숫자로 장사 실력 재기 . . . 53
07.1 핵심성과지표 · KPI . . . 53
07.2 전환율 · Conversion Rate . . . 53
07.3 유지율 · Retention Rate . . . 54
07.4 이탈률 · Churn Rate . . . 54
07.5 사용자당 평균 매출 · ARPU . . . 55
07.6 고객 생애 가치 · LTV / CLV . . . 55
07.7 고객 획득 비용 · CAC . . . 55
07.8 평균 주문 금액 · AOV . . . 56
07.9 코호트 분석 · Cohort Analysis . . . 56
07.10 A/B 테스트 · A/B Test . . . 57
08장 데이터 분석 전체 흐름 . . . 60

08.0 한눈에 보는 6단계 . . . 60
08.1 수집 · Data Collection . . . 60
08.2 전처리 단계 . . . 61
08.3 EDA 단계 . . . 61
08.4 피처 엔지니어링 단계 · Feature Selection / Creation . . . 62
08.5 모델 학습 단계 . . . 62
08.6 평가·해석 단계 . . . 63
09장 파이썬·pandas 용어 . . . 64
09.0 들어가며: 영어 단어를 알면 코드가 읽힌다 . . . 64
09.1 dtype · 자료형 . . . 64
09.2 Series · 시리즈 . . . 65
09.3 DataFrame · 데이터프레임 . . . 65
09.4 axis · 축 . . . 65
09.5 inplace · 제자리에서 . . . 66
09.6 delimiter · 구분자 . . . 67
09.7 describe · 요약 통계 . . . 67
09.8 numeric · numeric_only . . . 68
09.9 query · 질의 . . . 68
09.10 aggregation · agg . . . 69
09.11 statistic · 통계치 . . . 69
09.12 matplotlib · 맷플롯립 . . . 69
09.13 scatter · 산점도 . . . 70
09.14 prompt · 프롬프트 . . . 70
10장 통계 모델·분석 분야 용어 . . . 72
10.1 Regression · 회귀 . . . 72
10.2 LinearRegression · 선형 회귀 . . . 72
10.3 Intercept · 절편 . . . 73
10.4 Coefficient · 계수 . . . 73
10.5 statsmodels · 통계 모델 라이브러리 . . . 74
10.6 OLS · 최소제곱법 . . . 74
10.7 Shapiro-Wilk · 정규성 검정 . . . 74
10.8 KMeans · K-평균 군집화 . . . 75
10.9 Elbow · 엘보 방법 . . . 76
10.10 패널 데이터 · Panel Data . . . 77

10.11 금융 데이터 · Financial Data . . . 77
10.12 계량 분석 · Econometrics . . . 78
10.13 정리: 표준화 vs 정규화 . . . 78
11장 인공지능(딥러닝) 학습 용어 . . . 80
11.0 들어가며: AI 는 어떻게 공부할까? . . . 80
11.1 사전학습 · Pre-training . . . 80
11.2 미세조정 · Fine-tuning . . . 81
11.3 토큰 · 토큰화 · Token / Tokenization . . . 81
11.4 sentencepiece · 센텐스피스 . . . 81
11.5 임베딩 · Embedding . . . 82
11.6 순전파 · 역전파 · Propagation . . . 83
11.7 엔트로피 · Entropy . . . 83
11.8 최적화 · Optimization . . . 84
11.8.1 손실 함수 · Loss Function . . . 85
11.8.2 기울기 · Gradient . . . 85
11.8.3 학습률 · Learning Rate . . . 86
11.9 옵티마이저 · Optimizer . . . 86
11.9.1 SGD · 확률적 경사하강법 . . . 87
11.9.2 Momentum · 모멘텀 . . . 87
11.9.3 AdaGrad · 아다그라드 . . . 88
11.9.4 RMSProp · 알엠에스프롭 . . . 88
11.9.5 Adam · 아담 . . . 88
11.10 에포크 · Epoch . . . 89
11.11 배치 크기 · Batch Size . . . 90
11.12 이터레이션 · Iteration . . . 90
11.13 분류기 · Classifier . . . 91
11.14 감성 분석 · Sentiment Analysis . . . 91
11.15 Pydantic · 파이댄틱 . . . 92
11.16 SpooledTemporaryFile · 스풀드 템퍼러리 파일 . . . 92
12장 프롬프트 엔지니어링·생성형 AI 용어 . . . 94
12.0 들어가며: AI 에게 일 잘 시키기 . . . 94
12.1 프롬프트 · Prompt . . . 94
12.2 시스템 프롬프트 · System Prompt . . . 94
12.3 사용자 프롬프트 · User Prompt . . . 95

12.4 제로샷 · Zero-shot . . . 95
12.5 원샷 · One-shot . . . 95
12.6 퓨샷 · Few-shot . . . 96
12.7 생각의 사슬 · Chain of Thought . . . 96
12.8 컨텍스트 윈도우 · Context Window . . . 96
12.9 토큰 수 계산 · Token . . . 97
12.10 온도 · Temperature . . . 97
12.11 프롬프트 체이닝 · Prompt Chaining . . . 98
12.12 RAG · 검색 증강 생성 . . . 98
12.13 환각 · Hallucination . . . 99
12.14 역할 지정 · Role / Persona . . . 99
12.15 출력 형식 제한 · Format Constraint . . . 99
12.16 프롬프트 라이브러리 · Prompt Library . . . 100
12.17 기법 선택 가이드 . . . 100
정답과 해설 . . . 102
용어 찾아보기 . . . 105

CHAPTER 01

데이터 분석 기초 용어

학습 목표

- 데이터셋·피처·타깃·관측치·변수가 표의 어디를 가리키는지 말할 수 있다.
- 범주형과 수치형 데이터를 구분한다.
- 결측치·이상치·중복 데이터가 왜 문제인지 안다.

01.0 들어가며: 엑셀 표 하나로 시작하기

데이터 분석의 첫 용어들은 모두 **표(테이블)의 어느 부분**을 부르는 이름이에요. 아래 "카페 손님" 표를 기준으로 하나씩 짚어 볼게요. 파이썬에서는 이런 표를 **pandas 의 DataFrame** 으로 다뤄요.

파이썬 코드

```
import pandas as pd

# 카페 손님 데이터: 한 줄 = 손님 한 명
df = pd.DataFrame({
    "나이":   [23, 35, 41, 29, 35],
    "성별":   ["여", "남", "여", "여", "남"],
    "방문횟수": [12, 3, 8, 15, 3],
    "재방문":  ["예", "아니오", "예", "예", "아니오"],
})
print(df)
```

실행 결과

```
   나이 성별  방문횟수  재방문
0  23  여    12    예
1  35  남     3  아니오
2  41  여     8    예
3  29  여    15    예
4  35  남     3  아니오
```

01.1 데이터셋 · Dataset

Dataset
분석할 데이터 전체

분석하려고 모아 둔 **데이터 묶음 전체**예요. 위의 카페 손님 표 전체가 하나의 데이터셋이에요. CSV 파일 하나, 엑셀 시트 하나, DB 테이블 하나가 보통 데이터셋 하나가 됩니다.

TIP 비유: 데이터셋은 **반 전체의 학생 명부**예요. 명부 한 권이 데이터셋입니다.

01.2 피처 · Feature

Feature
분석·예측에 사용하는 입력 변수

예측할 때 **힌트로 쓰는 열(칼럼)**이에요. "재방문할까?"를 맞히려고 나이, 성별, 방문횟수를 본다면 이 세 열이 피처예요. **독립변수, 설명변수, 입력 변수(X)** 라고도 불러요.

TIP 비유: 날씨를 예측할 때 쓰는 **구름 양, 습도, 바람** 같은 단서가 피처예요.

01.3 타깃 · Target

Target
예측하려는 정답 변수

모델이 **맞혀야 하는 정답 열**이에요. 위 표에서는 "재방문" 이 타깃이에요. **종속변수, 레이블(label), y** 라고도 불러요.

파이썬 코드
```
X = df[["나이", "성별", "방문횟수"]]  # 피처 (문제)
y = df["재방문"]                    # 타깃 (정답)
print(X.shape, y.shape)             # (행 수, 열 수)
```

실행 결과
```
(5, 3) (5,)
```

TIP 피처 = **시험 문제**, 타깃 = **정답지**. 머신러닝은 문제와 정답을 보며 규칙을 배워요.

01.4 관측치 · Observation

Observation
데이터 한 행에 해당하는 관측 대상

표의 **한 행(줄)** 이에요. 손님 한 명, 주문 한 건, 하루치 날씨처럼 "관측한 대상 하나"를 뜻해요. **행(row), 레코드, 샘플, 인스턴스** 라고도 불러요. 위 표는 관측치가 5개예요.

01.5 변수 · Variable

Variable
데이터의 속성 또는 특성

표의 **한 열(칼럼)** 이에요. 나이, 성별처럼 관측치마다 **값이 달라질 수 있는 항목**이라서 "변하는 수", 변수라고 불러요. 피처와 타깃도 모두 변수예요. (피처 = 입력으로 쓰는 변수, 타깃 = 정답으로 쓰는 변수)

파이썬 코드
```
print(df.columns.tolist())  # 변수(열) 이름
print(len(df))              # 관측치(행) 개수
```

실행 결과
```
['나이', '성별', '방문횟수', '재방문']
5
```

01.6 범주형 데이터 · Categorical Data

Categorical Data
성별, 지역, 상품 종류처럼 종류를 나타내는 데이터

종류나 그룹을 나타내는 데이터예요. 더하거나 평균을 내는 것이 의미 없어요. ("여" + "남" = ?)

- **명목형**: 순서가 없음 — 성별, 지역, 혈액형
- **순서형**: 순서가 있음 — 학년(1·2·3), 만족도(낮음·보통·높음)

> **TIP** 숫자로 적혀 있어도 범주형일 수 있어요. **우편번호, 학번, 1=남/2=여** 는 숫자지만 크기 비교가 의미 없으니 범주형이에요.

01.7 수치형 데이터 · Numerical Data

> **Numerical Data**
> **나이, 매출, 가격처럼 크기를 비교·계산할 수 있는 숫자 데이터**

크기를 비교하고 계산할 수 있는 숫자 데이터예요. 평균, 합계를 낼 수 있어요.

- **이산형**: 셀 수 있는 값 — 방문횟수, 자녀 수 (정수)
- **연속형**: 어떤 값이든 가능 — 키, 몸무게, 온도

파이썬 코드
```
print(df.dtypes)                          # 열마다 자료형 확인
print(df.select_dtypes("number").columns.tolist())   # 수치형 열만
```

실행 결과
```
나이      int64
성별        str
방문횟수    int64
재방문       str
dtype: object
['나이', '방문횟수']
```

> **TIP** pandas 버전에 따라 글자 열의 자료형이 `object` 또는 `str` 로 보여요. 둘 다 "문자(범주형 후보)"라는 뜻이에요.

01.8 결측치 · Missing Value

> **Missing Value**
> **값이 비어 있는(누락된) 데이터**

값이 비어 있는 칸이에요. 설문에서 나이를 안 적었거나, 센서가 잠깐 꺼졌을 때 생겨요. pandas 에서는 `NaN`(Not a Number) 또는 `None` 으로 보여요.

파이썬 코드
```
import numpy as np
d = pd.DataFrame({"나이": [23, np.nan, 41], "점수": [80, 75, np.nan]})
print(d.isnull().sum())          # 열마다 결측치 개수
print(d.fillna(d.mean()))        # 평균으로 채우기
print(d.dropna())                # 결측치가 있는 행 지우기
```

실행 결과

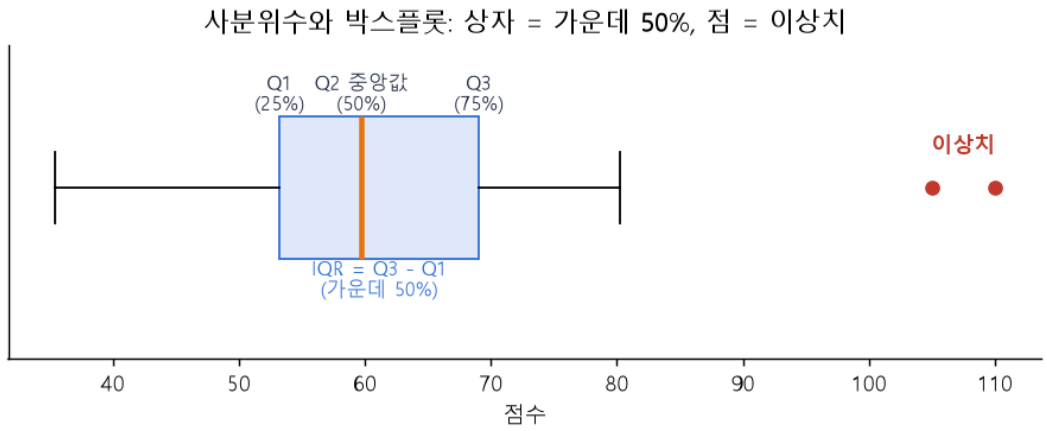
```
나이    1
점수    1
dtype: int64
     나이    점수
0  23.0  80.0
1  32.0  75.0
2  41.0  77.5
     나이    점수
0  23.0  80.0
```

TIP 결측치를 그대로 두면 평균이 틀어지거나 모델이 오류를 내요. **지우기(dropna)** 또는 **채우기(fillna: 평균·중앙값·최빈값)** 중 상황에 맞게 골라요.

01.9 이상치 · Outlier

Outlier
다른 값들과 크게 차이 나는 데이터

다른 값들과 **동떨어지게 크거나 작은 값**이에요. 반 평균 키가 160cm 인데 250cm 가 있다면 이상치예요. 입력 실수일 수도 있고, 진짜 특별한 경우일 수도 있어요.



▲ 박스플롯의 점이 이상치예요

파이썬 코드

```
s = pd.Series([62, 58, 65, 60, 59, 61, 150])
q1, q3 = s.quantile(0.25), s.quantile(0.75)
iqr = q3 - q1
print(s[(s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)])   # IQR 규칙으로 찾은 이상치
```

실행 결과

```
6    150
dtype: int64
```

TIP 이상치는 무조건 지우지 말고 **왜 생겼는지** 먼저 봐요. 카드 사기 탐지에서는 이상치가 바로 찾고 싶은 대상이에요.

01.10 중복 데이터 · Duplicate Data

Duplicate Data
같은 정보가 반복해서 들어간 데이터

똑같은 행이 두 번 이상 들어간 거예요. 주문이 실수로 두 번 저장되면 매출이 부풀려져요.

파이썬 코드

```
orders = pd.DataFrame({"주문번호": [1, 2, 2, 3], "금액": [5000, 7000, 7000, 3000]})
print(orders.duplicated().sum())          # 중복 행 개수
print(orders.drop_duplicates())           # 중복 제거
```

실행 결과

```
1
   주문번호    금액
0     1  5000
1     2  7000
3     3  3000
```

총정리 퀴즈

Q1. "재방문 여부를 예측한다"에서 재방문 열은?

① 피처
② 타깃
③ 관측치
④ 결측치

Q2. 표의 한 행(줄)을 부르는 이름이 아닌 것은?

① 관측치
② 레코드
③ 샘플
④ 피처

Q3. 다음 중 범주형 데이터는?

① 키
② 매출액
③ 우편번호
④ 온도

Q4. 값이 비어 있는 칸을 무엇이라 하나요?

① 결측치
② 이상치
③ 중복 데이터
④ 타깃

Q5. 반 평균 키 160cm 인 데이터에 250cm 가 있다면?

① 결측치
② 이상치
③ 범주형
④ 중복

CHAPTER 02

기초 통계 용어

학습 목표

- 평균·중앙값·최빈값을 구하고, 언제 무엇을 쓰는지 안다.
- 분산·표준편차로 데이터가 퍼진 정도를 읽는다.
- 사분위수·백분위수·상관계수·왜도·첨도의 뜻을 그림으로 이해한다.

02.0 들어가며: 숫자 여러 개를 숫자 하나로

데이터가 100개, 1만 개면 하나하나 볼 수 없어요. 그래서 **"대표값"**(가운데가 어디?)과 **"퍼진 정도"**(얼마나 흩어졌나?)를 숫자 하나로 요약해요. 이 장의 예제는 아래 시험 점수를 같이 써요.

파이썬 코드

```
import pandas as pd
scores = pd.Series([55, 60, 65, 70, 70, 75, 80, 85, 95, 100])
print(scores.describe())        # 개수, 평균, 표준편차, 최솟값, 사분위수, 최댓값을 한 번에
```

실행 결과

```
count     10.000000
mean      75.500000
std       14.615441
min       55.000000
25%       66.250000
50%       72.500000
75%       83.750000
max      100.000000
dtype: float64
```

02.1 평균 · Mean

Mean
데이터 합계를 개수로 나눈 값

모두 더해서 **개수로 나눈 값**이에요. 가장 많이 쓰는 대표값이지만, **이상치에 크게 흔들려요.**

파이썬 코드

```
print(scores.mean())                    # (55 + 60 + ... + 100) / 10
print(pd.Series([3, 3, 4, 4, 100]).mean())   # 100 하나 때문에 평균이 22.8 로 튐
```

실행 결과

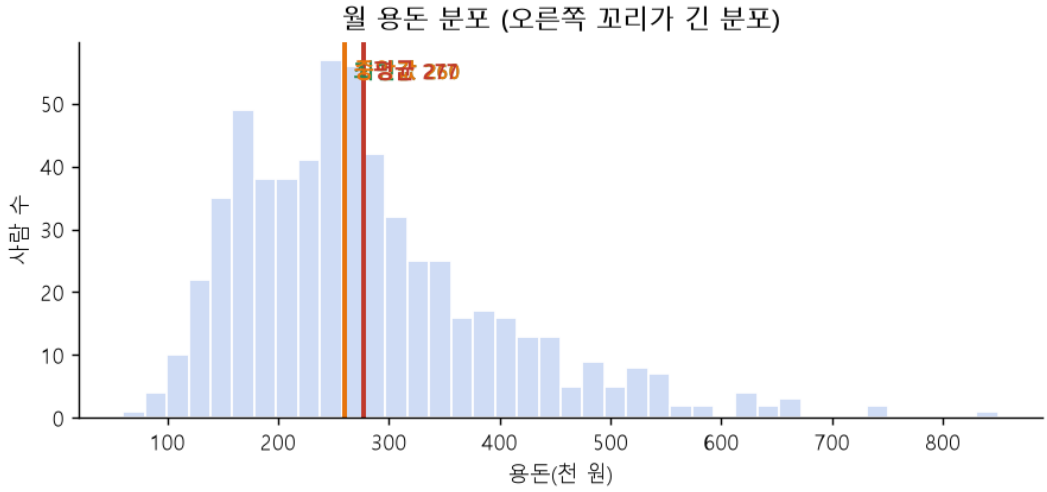
```
75.5
22.8
```

TIP 비유: 친구 4명 용돈이 3만 원인데 1명이 100만 원이면 "평균 용돈 22만 원" 이 돼요. 이럴 땐 평균이 대표값으로 어울리지 않아요.

02.2 중앙값 · Median

Median
크기순으로 정렬했을 때 가운데 값

값을 **크기순으로 줄 세웠을 때 한가운데** 값이에요. 개수가 짝수면 가운데 두 값의 평균이에요. **이상치에 잘 흔들리지 않아서** 소득·집값처럼 한쪽으로 치우친 데이터에 써요.



▲ 오른쪽 꼬리가 긴 분포에서는 평균 > 중앙값 > 최빈값

파이썬 코드
```
print(scores.median())                      # 정렬 후 5·6번째 (70, 75)의 평균
print(pd.Series([3, 3, 4, 4, 100]).median()) # 4 — 100 이 있어도 그대로
```

실행 결과
```
72.5
4.0
```

02.3 최빈값 · Mode

Mode
가장 자주 나타나는 값

가장 많이 나온 값이에요. 숫자뿐 아니라 범주형(가장 많이 팔린 메뉴)에도 쓸 수 있어요. 가장 많은 값이 여러 개면 최빈값도 여러 개예요.

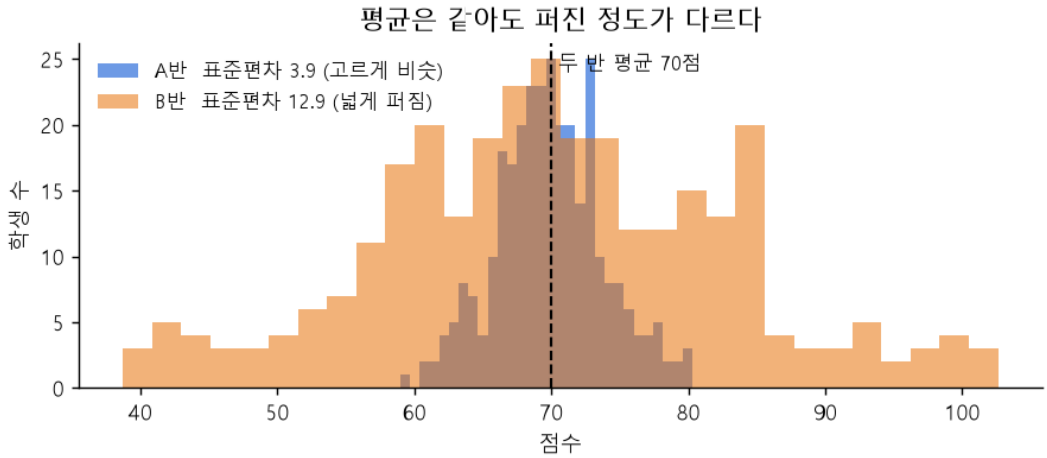
파이썬 코드
```
print(scores.mode().tolist())                              # [70]
print(pd.Series(["라떼", "아메리카노", "라떼"]).mode()[0])    # 라떼
```

실행 결과
```
[70]
라떼
```

02.4 분산 · Variance

Variance
데이터가 평균에서 얼마나 퍼져 있는지 나타내는 값

각 값이 **평균에서 떨어진 거리를 제곱해서 평균**낸 값이에요. 클수록 데이터가 넓게 퍼져 있어요. 제곱을 해서 단위도 제곱(점², 원²)이 돼서 직접 읽기는 어려워요.



▲ 평균은 같아도 퍼진 정도는 다르다

파이썬 코드

```
import numpy as np
x = np.array([60, 70, 80])
print(((x - x.mean()) ** 2).mean())   # 직접 계산: (100 + 0 + 100) / 3
print(x.var(), pd.Series(x).var())    # numpy 는 n 으로, pandas 는 n-1 로 나눔(표본분산)
```

실행 결과

```
66.66666666666667
66.66666666666667 100.0
```

TIP numpy 와 pandas 의 분산 값이 다른 이유: pandas 는 **표본분산**(n-1 로 나눔)이 기본이에요. 표본으로 모집단을 추정할 때 더 정확하거든요.

02.5 표준편차 · Standard Deviation

Standard Deviation
분산의 제곱근 (평균에서 보통 얼마나 떨어져 있나)

분산에 **루트(제곱근)** 를 씌운 값이에요. 단위가 원래대로 돌아와서 "평균에서 **보통 이 정도** 떨어져 있다"로 읽을 수 있어요.

파이썬 코드

```
print(round(scores.std(), 2))   # 점수가 평균에서 보통 약 15점 정도 떨어져 있다
```

실행 결과

```
14.62
```

TIP 평균 70점, 표준편차 5점 반은 다들 비슷하고, 표준편차 20점 반은 실력 차이가 커요.

02.6 최솟값 · 최댓값 · Minimum · Maximum

Min / Max
가장 작은 값 / 가장 큰 값

가장 작은 값과 가장 큰 값이에요. 둘의 차이를 **범위(range)** 라고 해요. 데이터를 처음 받으면 최솟값·최댓값부터 보면 **입력 실수**(나이 -5, 키 999)를 빨리 찾을 수 있어요.

파이썬 코드

```
print(scores.min(), scores.max(), scores.max() - scores.min())
```

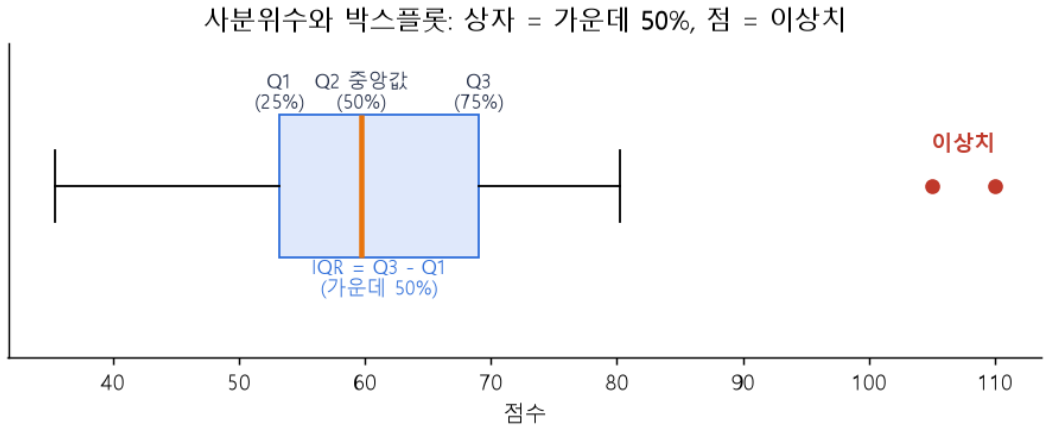
실행 결과

```
55 100 45
```

02.7 사분위수 · Quartile

Quartile
데이터를 4등분하는 경계값 (Q1·Q2·Q3)

정렬한 데이터를 **4등분하는 세 지점**이에요. **Q1**(25%), **Q2**(50% = 중앙값), **Q3**(75%). Q3 - Q1 을 **IQR**(사분위 범위)이라 하고, 가운데 50% 가 퍼진 폭이에요. 박스플롯이 바로 사분위수 그림이에요.



▲ 사분위수와 박스플롯

파이썬 코드

```
print(scores.quantile([0.25, 0.5, 0.75]))
```

실행 결과

```
0.25    66.25
0.50    72.50
0.75    83.75
dtype: float64
```

02.8 백분위수 · Percentile

Percentile
데이터를 100 등분했을 때의 위치값

아래에서 몇 % 지점에 있는 값이에요. "상위 10%" 는 90 백분위수 이상이에요. 사분위수는 25·50·75 백분위수와 같아요.

파이썬 코드

```
print(scores.quantile(0.9))     # 90 백분위수: 이 점수 이상이면 상위 10%
```

실행 결과

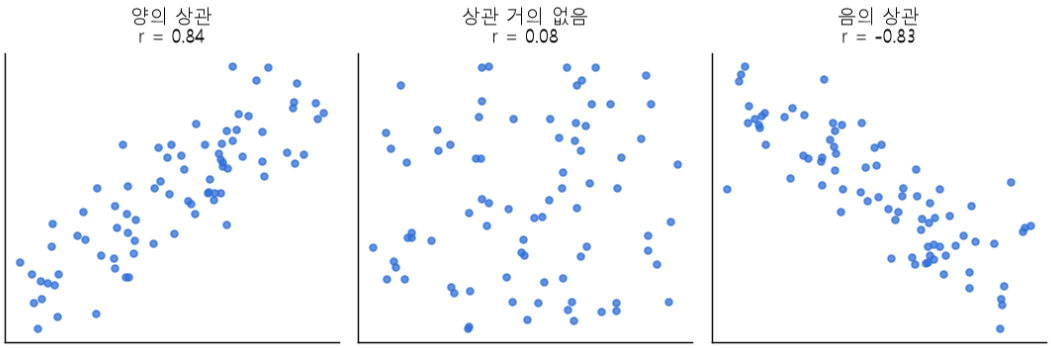
```
95.5
```

TIP 수능 백분위 95 = 나보다 점수가 낮은 사람이 95% 라는 뜻이에요.

02.9 상관계수 · Correlation Coefficient

Correlation Coefficient
두 변수가 함께 움직이는 정도 (-1 ~ 1)

두 변수가 **같이 커지는지(+), 반대로 움직이는지(-)** 를 -1 ~ 1 로 나타내요. 0 에 가까우면 **직선 관계가 거의 없어요.**



▲ 양의 상관, 상관 없음, 음의 상관

파이썬 코드

```
study = pd.Series([1, 2, 3, 4, 5, 6])
score = pd.Series([52, 60, 63, 71, 78, 83])
print(round(study.corr(score), 3))      # 1 에 가까움 = 강한 양의 상관
```

실행 결과

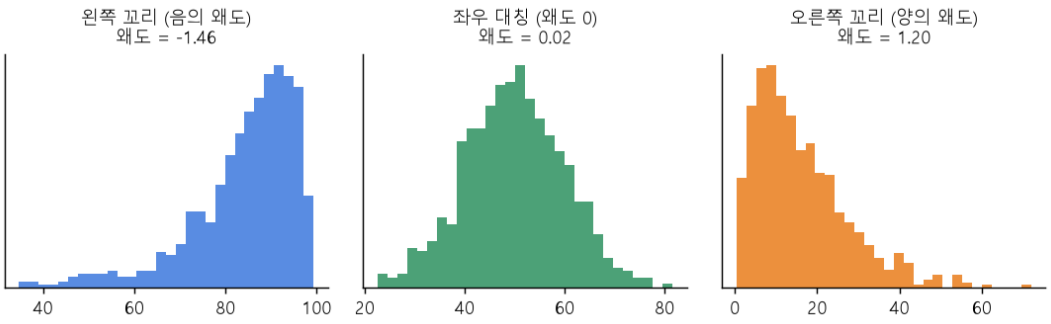
```
0.995
```

주의 **상관관계는 인과관계가 아니에요.** 아이스크림 판매량과 물놀이 사고는 상관이 높지만, 아이스크림이 사고를 일으키는 게 아니라 둘 다 "더운 날씨" 때문이에요.

02.10 왜도 · Skewness

Skewness
분포가 좌우로 얼마나 비대칭인지

분포가 **한쪽으로 치우친 정도**예요. 0 이면 좌우 대칭, **양수**면 오른쪽 꼬리가 길고(소득·집값), **음수**면 왼쪽 꼬리가 길어요(쉬운 시험 점수).



▲ 왜도: 음수, 0, 양수

파이썬 코드

```
print(round(pd.Series([1, 2, 2, 3, 3, 3, 4, 20]).skew(), 2))   # 20 때문에 오른쪽 꼬리 → 양수
```

실행 결과

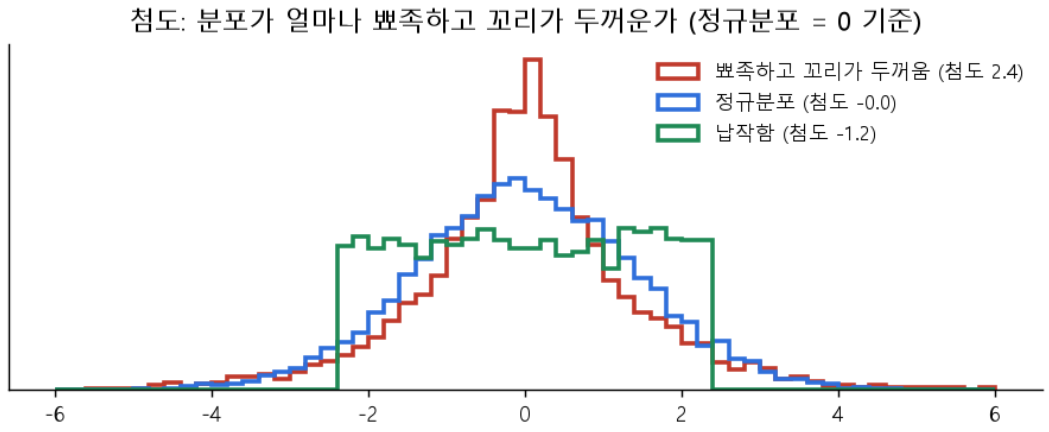
```
2.71
```

TIP 오른쪽 꼬리가 긴(양의 왜도) 데이터는 보통 **평균 > 중앙값** 이에요. 그래서 왜도가 크면 평균 대신 중앙값을 대표값으로 써요.

02.11 첨도 · Kurtosis

Kurtosis
분포가 얼마나 뾰족하고 꼬리가 두꺼운지

분포의 **뾰족함과 꼬리 두께**를 나타내요. pandas 는 정규분포를 **0** 으로 놓고 계산해요. 양수면 가운데가 뾰족하고 꼬리가 두꺼워 **극단값이 자주** 나오고, 음수면 납작해요.



▲ 첨도 비교

파이썬 코드

```
import numpy as np
rng = np.random.default_rng(0)
print(round(pd.Series(rng.normal(size=5000)).kurt(), 2))    # 정규분포 → 0 근처
print(round(pd.Series(rng.laplace(size=5000)).kurt(), 2))   # 뾰족한 분포 → 양수
```

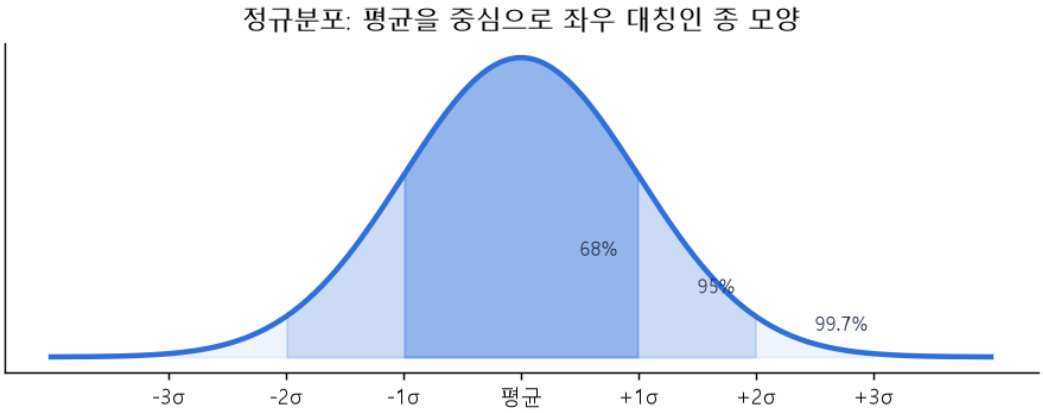
실행 결과

```
-0.1
3.9
```

02.12 정규분포 · Normal Distribution

Normal Distribution
평균을 중심으로 좌우 대칭인 종 모양 분포

키, 시험 점수처럼 많은 데이터가 따르는 **종 모양 분포**예요. 평균 ± 1 표준편차 안에 약 **68%**, ± 2 표준편차 안에 약 **95%** 가 들어가요. 많은 통계 방법이 "데이터가 정규분포를 따른다"고 가정해서, 정규성 검정(9장 Shapiro-Wilk)으로 확인해요.



▲ 정규분포와 68-95-99.7 규칙

총정리 퀴즈

Q1. 이상치가 있을 때 대표값으로 더 알맞은 것은?

① 평균
② 중앙값
③ 최댓값
④ 분산

Q2. 데이터 [2, 4, 4, 6, 9] 의 중앙값과 최빈값은?

① 5, 4
② 4, 4
③ 4, 9
④ 5, 6

Q3. 분산과 표준편차의 관계는?

① 표준편차 = 분산 × 2
② 표준편차 = 분산의 제곱근
③ 분산 = 표준편차 ÷ 2
④ 관계 없음

Q4. 상관계수가 -0.9 라면?

① 관계가 거의 없다
② 한쪽이 커지면 다른 쪽도 커진다
③ 한쪽이 커지면 다른 쪽은 작아진다
④ 인과관계가 있다

Q5. 소득처럼 오른쪽 꼬리가 긴 분포의 왜도는?

① 양수
② 음수
③ 0
④ 항상 1

Q6. IQR 은 무엇인가요?

① 최댓값 - 최솟값
② Q3 - Q1
③ 평균 - 중앙값
④ 표준편차의 제곱

CHAPTER 03

데이터 전처리 용어

학습 목표

- 전처리가 왜 필요한지 설명한다.
- 정규화와 표준화의 차이를 그림으로 안다.
- 원-핫 인코딩과 레이블 인코딩을 구분해 쓴다.
- 병합·그룹화·집계·피처 엔지니어링을 pandas 로 해 본다.
- 데이터 누수를 피하는 법을 안다.

03.1 전처리 · Preprocessing

Preprocessing
데이터를 분석하기 알맞게 다듬는 일

받은 데이터를 **분석·학습하기 좋게 다듬는** 모든 작업이에요. 결측치 처리, 이상치 처리, 중복 제거, 자료형 바꾸기, 스케일 맞추기, 인코딩이 다 포함돼요. 실무에서는 분석 시간의 **절반 이상**이 전처리라고 할 만큼 중요해요.

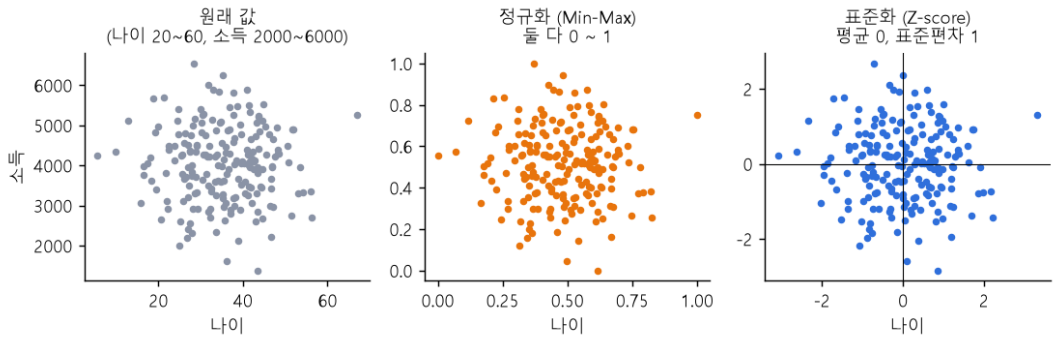
TIP 비유: 요리 전에 **재료 손질**(씻고, 다듬고, 썰기)을 하는 것과 같아요. 손질이 엉망이면 요리도 엉망이에요. (Garbage in, garbage out)

03.2 정규화 · Normalization

Normalization
값을 일정한 범위(보통 0~1)로 바꾸는 일

값을 **0 ~ 1 사이로 줄이고 늘리는** 거예요(Min-Max 스케일링). 가장 작은 값은 0, 가장 큰 값은 1 이 돼요.

공식: (x - 최솟값) / (최댓값 - 최솟값)



▲ 원래 값 · 정규화 · 표준화

파이썬 코드

```
import pandas as pd
age = pd.Series([20, 30, 40, 60])
print(((age - age.min()) / (age.max() - age.min())).tolist())
```

실행 결과

```
[0.0, 0.25, 0.5, 1.0]
```

TIP 왜 필요할까? 나이(20~60)와 소득(2000~6000만)을 그대로 쓰면 숫자가 큰 소득이 결과를 좌지우지해요. 범위를 맞춰 **공평하게** 비교하게 해요. (거리를 쓰는 KNN·K-Means 에서 특히 중요)

03.3 표준화 · Standardization

Standardization
평균 0, 표준편차 1 이 되도록 바꾸는 일

각 값에서 평균을 빼고 표준편차로 나눠 **평균 0, 표준편차 1** 로 맞추는 거예요(Z-score). 결과값은 "평균에서 표준편차 몇 개만큼 떨어졌나"를 뜻해요.

공식: (x - 평균) / 표준편차

파이썬 코드
```
z = (age - age.mean()) / age.std(ddof=0)
print(z.round(2).tolist(), round(z.mean(), 2), round(z.std(ddof=0), 2))
```

실행 결과
```
[-1.18, -0.51, 0.17, 1.52] 0.0 1.0
```

구분	정규화 (Min-Max)	표준화 (Z-score)
기준	**범위** 기준 (크기 맞추기)	**평균** 기준 (위치·흩어짐 맞추기)
결과 범위	0 ~ 1	대략 -3 ~ 3 (제한 없음)
이상치	크게 영향받음	덜 영향받음
sklearn	MinMaxScaler	StandardScaler

03.4 인코딩 · Encoding

Encoding
범주형 데이터를 숫자로 바꾸는 일

컴퓨터 모델은 **숫자만** 계산할 수 있어서, "서울/부산", "남/여" 같은 글자를 숫자로 바꿔야 해요. 대표적인 방법이 원-핫 인코딩과 레이블 인코딩이에요.

03.5 원-핫 인코딩 · One-Hot Encoding

One-Hot Encoding
범주마다 0과 1로 된 열을 만드는 방법

범주 하나하나를 **새 열**로 만들고, 해당하면 1(또는 True), 아니면 0 을 넣어요. 순서가 없는 범주(지역, 색상)에 알맞아요.

파이썬 코드
```
city = pd.DataFrame({"도시": ["서울", "부산", "서울", "대구"]})
print(pd.get_dummies(city, columns=["도시"], dtype=int))
```

실행 결과

```
   도시_대구  도시_부산  도시_서울
0      0      0      1
1      0      1      0
2      0      0      1
3      1      0      0
```

TIP 범주가 아주 많으면(예: 상품 1만 개) 열이 너무 많아져요. 그럴 땐 자주 나오는 것만 남기거나 다른 방법을 써요.

03.6 레이블 인코딩 · Label Encoding

Label Encoding
범주를 0, 1, 2 같은 정수로 바꾸는 방법

범주마다 **정수 번호**를 붙여요. 열이 늘지 않아 간단하지만, 모델이 숫자 크기(2 > 1)를 **순서로 오해**할 수 있어요. 그래서 **순서가 있는 범주**(낮음·보통·높음)나 트리 모델에 주로 써요.

파이썬 코드

```
level = pd.Series(["낮음", "높음", "보통", "낮음"])
print(level.map({"낮음": 0, "보통": 1, "높음": 2}).tolist())   # 순서를 직접 정해 주기
```

실행 결과

```
[0, 2, 1, 0]
```

03.7 데이터 병합 · Merge / Join

Merge / Join
여러 데이터를 공통 키로 이어 붙이는 일

두 표를 **공통 열(키)** 로 이어 붙여요. 주문 표에는 고객번호만 있고 고객 이름은 고객 표에 있을 때, 고객번호로 합치면 한 표에서 볼 수 있어요. SQL 의 JOIN 과 같아요.

파이썬 코드

```
orders = pd.DataFrame({"고객번호": [1, 2, 1], "금액": [5000, 3000, 7000]})
customers = pd.DataFrame({"고객번호": [1, 2], "이름": ["김파이", "이코딩"]})
print(pd.merge(orders, customers, on="고객번호", how="left"))
```

실행 결과

```
   고객번호    금액   이름
0     1  5000  김파이
1     2  3000  이코딩
2     1  7000  김파이
```

TIP how 로 합치는 방식을 정해요: inner(양쪽에 다 있는 것만), left(왼쪽 표 전부), outer(양쪽 전부).

03.8 그룹화 · Grouping

Grouping
특정 기준으로 데이터를 묶는 일

같은 값끼리 **묶는** 거예요. "도시별", "월별", "성별" 처럼 기준을 정해 나눠요. 그룹화만 하면 결과가 안 보이고, 보통 바로 **집계**와 함께 써요.

03.9 집계 · Aggregation

> **Aggregation**
> **여러 값을 모아 합계·평균·개수 등 하나로 요약**

여러 값을 **하나의 숫자로 요약**해요: 합계(sum), 평균(mean), 개수(count), 최댓값(max), 최솟값(min).

파이썬 코드
```
sales = pd.DataFrame({"도시": ["서울", "부산", "서울", "부산", "서울"], "매출": [100, 80, 120, 60, 90]})
print(sales.groupby("도시")["매출"].agg(["sum", "mean", "count"]))   # 그룹화 + 집계
```

실행 결과
```
    sum        mean  count
도시
부산  140   70.000000      2
서울  310  103.333333      3
```

TIP "도시별 평균 매출" = 도시로 **그룹화** 하고, 매출을 평균으로 **집계**. SQL 의 `GROUP BY` 와 같아요.

03.10 피처 엔지니어링 · Feature Engineering

> **Feature Engineering**
> **예측에 도움이 되는 새 변수를 만드는 일**

원래 데이터를 가공해 **모델에 더 도움이 되는 새 피처**를 만드는 거예요. 좋은 피처 하나가 복잡한 모델보다 효과가 클 때가 많아요.

파이썬 코드
```
d = pd.DataFrame({"주문일": pd.to_datetime(["2026-11-01", "2026-11-07"]), "총액": [30000, 12000], "수량": [3, 4]})
d["요일"] = d["주문일"].dt.day_name()        # 날짜 → 요일
d["주말"] = d["주문일"].dt.dayofweek >= 5    # 주말 여부
d["개당가격"] = d["총액"] / d["수량"]          # 두 열로 새 열
print(d)
```

실행 결과
```
        주문일     총액  수량        요일    주말     개당가격
0 2026-11-01  30000   3    Sunday  True  10000.0
1 2026-11-07  12000   4  Saturday  True   3000.0
```

03.11 데이터 누수 · Data Leakage

> **Data Leakage**
> **학습에 쓰면 안 되는 정보가 모델에 섞여 들어가는 문제**

모델이 **실제로 예측할 때는 알 수 없는 정보**를 학습에 써 버리는 실수예요. 학습 점수는 엄청 좋게 나오는데, 실제로 쓰면 형편없어요.

- 예 ① "환불 여부"를 예측하는데 **환불 처리일** 칼럼을 피처로 넣음 (환불돼야 생기는 값)
- 예 ② 전체 데이터로 평균·표준편차를 구해 표준화한 뒤 훈련/테스트를 나눔 (테스트 정보가 섞임)

주의 막는 법: **먼저 훈련/테스트를 나누고**, 표준화·결측치 채우기 기준은 **훈련 데이터로만** 정해요. 예측 시점에 알 수 없는 칼럼은 빼요.

TIP 비유: 시험 전에 **정답지를 본** 학생이에요. 모의고사는 100점이지만 진짜 시험에서는 실력이 드러나요.

총정리 퀴즈

Q1. 값을 0 ~ 1 범위로 맞추는 방법은?

① 정규화 (Min-Max)
② 표준화 (Z-score)
③ 원-핫 인코딩
④ 집계

Q2. 표준화 후 데이터의 평균과 표준편차는?

① 0, 0
② 0, 1
③ 1, 0
④ 0.5, 1

Q3. 순서가 없는 '도시' 열을 모델에 넣을 때 알맞은 방법은?

① 레이블 인코딩
② 원-핫 인코딩
③ 표준화
④ 삭제

Q4. '도시별 평균 매출'을 구하는 pandas 코드는?

① df.merge("도시")
② df.groupby("도시")["매출"].mean()
③ df.sort_values("매출")
④ df.dropna()

Q5. 데이터 누수를 막는 올바른 순서는?

① 전체를 표준화 → 훈련/테스트 분리
② 훈련/테스트 분리 → 훈련 데이터 기준으로 표준화
③ 테스트 데이터로 표준화 기준 정하기
④ 상관없다

CHAPTER 04

탐색적 데이터 분석(EDA) 용어

학습 목표

- EDA 가 무엇이고 왜 먼저 하는지 안다.
- 분포·히스토그램·산점도·박스플롯·히트맵을 보고 읽을 수 있다.
- 상관분석과 교차표로 변수 사이의 관계를 확인한다.

04.0 들어가며: 탐색적 데이터 분석 · EDA

EDA
Exploratory Data Analysis, 데이터를 여러 각도로 살펴보는 첫 분석

EDA(Exploratory Data Analysis)는 모델을 만들기 전에 **데이터를 요리조리 살펴보는** 단계예요. 요약 통계를 보고, 그래프를 그리고, "이상한 값은 없나? 어떤 변수끼리 관계가 있나?"를 찾아요.

TIP 비유: 여행 가기 전에 **지도를 펼쳐 보는 것**이에요. 길이 어떻게 생겼는지 알아야 어디로 갈지 정할 수 있어요.

파이썬 코드

```
import pandas as pd
import numpy as np

rng = np.random.default_rng(1)
# 학생 100명 가상 데이터
df = pd.DataFrame({
    "공부시간": rng.integers(0, 10, 100),
    "반": rng.choice(["A", "B"], 100),
})
df["점수"] = (50 + df["공부시간"] * 4 + rng.normal(0, 6, 100)).round()
print(df.head())         # 앞 5줄 보기
print(df.describe())     # 수치형 요약
```

실행 결과

```
   공부시간  반    점수
0     4  A  68.0
1     5  B  66.0
2     7  B  83.0
3     9  B  85.0
4     0  B  54.0
            공부시간         점수
count  100.000000  100.00000
mean     4.620000   68.13000
std      2.834777   12.36781
min      0.000000   40.00000
25%      2.000000   58.75000
50%      4.500000   67.50000
75%      7.000000   78.00000
max      9.000000   90.00000
```

04.1 분포 · Distribution

Distribution
데이터 값들이 어떻게 퍼져 있는지 나타낸 모양

값들이 **어디에 많이 모여 있고 어디에 적은지**를 보여 주는 모양이에요. 가운데 몰려 있는지(정규분포), 한쪽으로 치우쳤는지(왜도), 봉우리가 두 개인지 등을 봐요. 분포는 주로 히스토그램으로 확인해요.

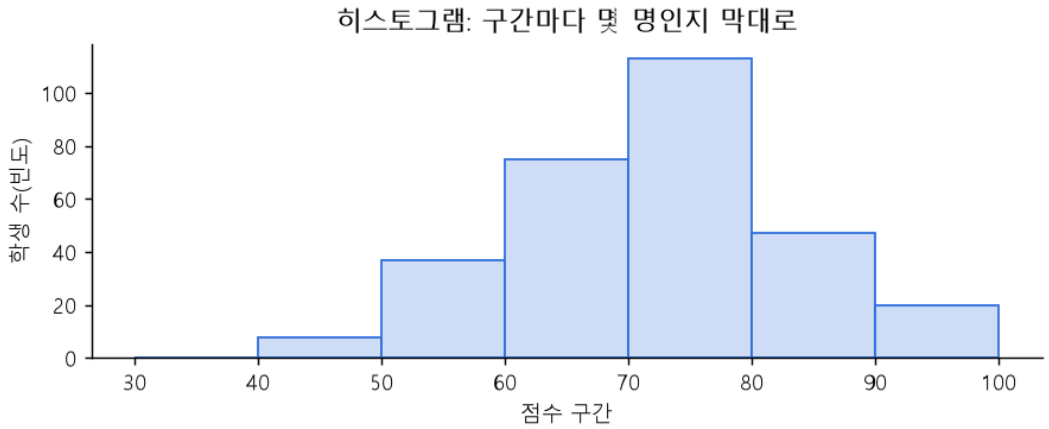
파이썬 코드
```
print(df["점수"].value_counts(bins=5).sort_index())   # 구간별 개수 = 분포를 숫자로
```

실행 결과
```
(39.949000000000005, 50.0]     8
(50.0, 60.0]                  21
(60.0, 70.0]                  27
(70.0, 80.0]                  22
(80.0, 90.0]                  22
Name: count, dtype: int64
```

04.2 히스토그램 · Histogram

Histogram
구간별 데이터 개수를 막대로 나타낸 그래프

값의 범위를 **구간(bin)** 으로 나누고, 구간마다 **몇 개 있는지** 막대로 그려요. 분포 모양을 한눈에 볼 수 있어요. 막대 사이가 붙어 있는 게 막대그래프와 다른 점이에요(연속된 숫자 구간이라서).



▲ 히스토그램: 구간별 개수

파이썬 코드
```
import matplotlib
matplotlib.use("Agg")                # 화면 없이 그림만 만들기
import matplotlib.pyplot as plt
plt.hist(df["점수"], bins=10)        # 10개 구간으로 나눠 개수 세기
plt.show()                           # 앱에서는 결과 창에 그림이 나와요
print("히스토그램 완성")
```

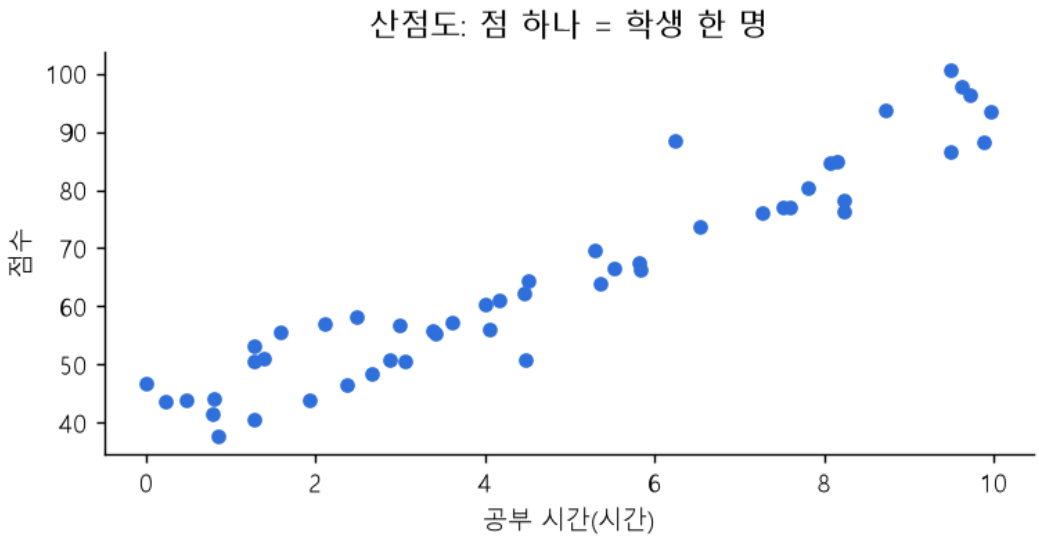
실행 결과
```
히스토그램 완성
```

TIP 구간 수(bins)가 너무 적으면 뭉개지고, 너무 많으면 들쭉날쭉해요. 몇 가지로 바꿔 보며 봐요.

04.3 산점도 · Scatter Plot

Scatter Plot
두 수치형 변수의 관계를 점으로 찍은 그래프

가로축에 한 변수, 세로축에 다른 변수를 놓고 **관측치마다 점 하나**를 찍어요. 점들이 오른쪽 위로 올라가면 양의 관계, 내려가면 음의 관계, 흩어져 있으면 관계가 약해요. 이상치도 잘 보여요.



▲ 공부시간과 점수의 산점도

파이썬 코드

```
plt.figure()
plt.scatter(df["공부시간"], df["점수"])
plt.xlabel("study"); plt.ylabel("score")
plt.show()
print("산점도 완성")
```

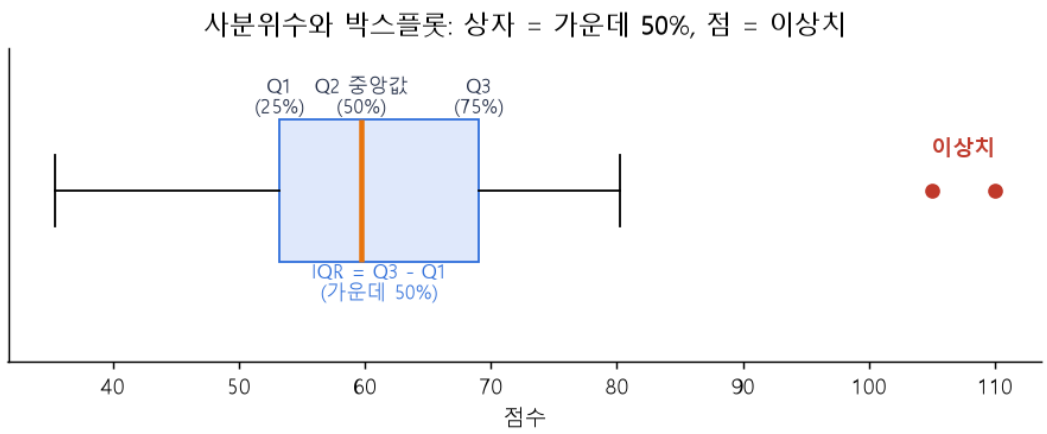
실행 결과

```
산점도 완성
```

04.4 박스플롯 · Box Plot

Box Plot
사분위수와 이상치를 상자 모양으로 나타낸 그래프

상자는 Q1~Q3(가운데 50%), 상자 안의 **선**은 중앙값, 위아래 **수염**은 일반적인 범위, 수염 밖의 **점**은 이상치예요. 여러 그룹의 분포를 나란히 비교할 때 좋아요.



▲ 박스플롯 읽는 법

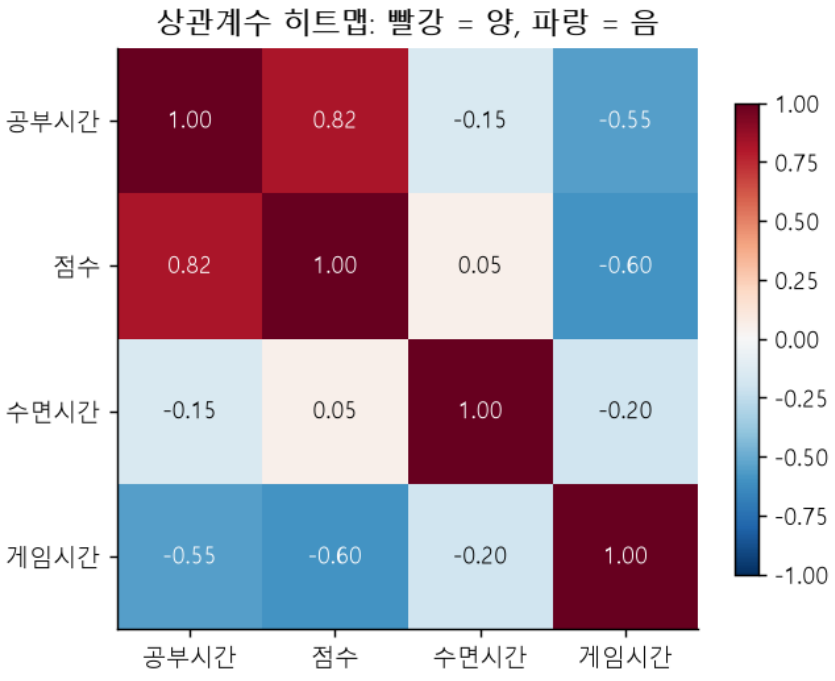
```
파이썬 코드
print(df.groupby("반")["점수"].describe()[["25%", "50%", "75%"]])   # 박스플롯이 보여 주는 숫자
```

```
실행 결과
     25%   50%    75%
반
A  58.25  66.0  76.00
B  63.00  70.0  78.75
```

04.5 히트맵 · Heatmap

Heatmap
숫자 크기를 색의 진하기로 나타낸 표 그래프

표의 숫자를 **색깔**로 바꿔 보여 줘요. 진할수록 값이 커요. 변수가 많을 때 **상관계수 표**를 히트맵으로 그리면 어떤 변수끼리 관계가 큰지 금방 찾을 수 있어요.



▲ 상관계수 히트맵

04.6 상관분석 · Correlation Analysis

Correlation Analysis
변수들 사이의 관계 정도를 분석하는 것

여러 변수 사이의 **상관계수를 한꺼번에 구해** 관계를 살펴보는 분석이에요. `df.corr()` 한 줄로 상관계수 표(상관행렬)가 나와요.

파이썬 코드
```
num = df[["공부시간", "점수"]]
print(num.corr().round(2))          # 대각선은 자기 자신과의 상관이라 항상 1
```

실행 결과
```
      공부시간    점수
공부시간  1.00  0.88
점수    0.88  1.00
```

주의 상관계수는 **직선 관계**만 잡아요. U자 모양처럼 휘어진 관계는 상관계수가 0 에 가까워도 관계가 있을 수 있으니 꼭 산점도와 함께 봐요.

04.7 교차표 · Crosstab

Crosstab
두 범주형 변수의 조합별 개수를 센 표

두 **범주형** 변수를 가로·세로로 놓고, 칸마다 **몇 개인지** 센 표예요. "반별로 합격자가 몇 명?" 같은 질문에 답해요. 엑셀의 피벗 테이블과 비슷해요.

파이썬 코드
```
df["합격"] = np.where(df["점수"] >= 70, "합격", "불합격")
print(pd.crosstab(df["반"], df["합격"]))                          # 개수
print(pd.crosstab(df["반"], df["합격"], normalize="index").round(2))   # 반별 비율
```

실행 결과
```
합격  불합격  합격
반
A    26  20
B    26  28
합격   불합격   합격
반
A   0.57  0.43
B   0.48  0.52
```

총정리 퀴즈

Q1. 모델을 만들기 전에 데이터를 그래프와 통계로 살펴보는 단계는?

① EDA
② 하이퍼파라미터
③ 과적합
④ 인코딩

Q2. 한 변수의 분포 모양을 보기에 가장 알맞은 그래프는?

① 히스토그램
② 산점도
③ 교차표
④ 히트맵

Q3. 두 수치형 변수의 관계를 점으로 찍은 그래프는?

① 박스플롯
② 산점도
③ 히스토그램
④ 파이차트

Q4. 박스플롯에서 상자 안의 선은?

① 평균
② 중앙값
③ 최댓값
④ 최빈값

Q5. '반'과 '합격 여부' 두 범주형 변수의 조합별 개수를 세는 표는?

① 상관행렬
② 교차표
③ 히스토그램
④ 산점도

CHAPTER 05

머신러닝 기초 용어

학습 목표

- 지도학습과 비지도학습을 구분하고, 각각의 대표 모델을 말할 수 있다.
- 분류·회귀·군집화·차원 축소가 각각 어떤 문제를 푸는지 안다.
- 훈련·검증·테스트 데이터를 나누는 이유를 안다.
- 과적합·과소적합·하이퍼파라미터의 뜻을 그림으로 이해한다.

05.0 들어가며: 머신러닝이란?

Machine Learning
데이터에서 규칙을 스스로 찾아내는 방법

사람이 규칙을 일일이 코딩하는 대신, **데이터를 보여 주면 컴퓨터가 규칙을 찾아내는** 방법이에요. 스팸 메일 규칙을 사람이 다 적는 대신, 스팸/정상 메일 수천 개를 보여 주고 스스로 구별법을 배우게 해요.

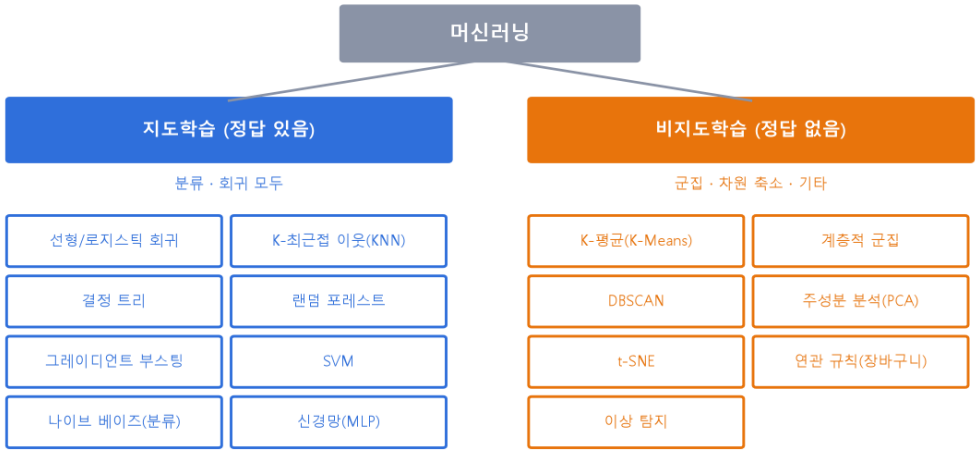
TIP 학습한 결과물(찾아낸 규칙)을 **모델(model)** 이라고 해요. 모델에 새 데이터를 넣어 답을 얻는 것을 **예측(predict)** 이라 해요.

05.1 지도학습 · Supervised Learning

Supervised Learning
정답(레이블)이 있는 데이터로 학습하는 방법

문제와 정답을 함께 주고 배우게 해요. 피처(X)와 타깃(y)이 둘 다 있어요. 대표적으로 **분류**(정답이 범주: 어느 칸?)와 **회귀**(정답이 숫자: 얼마?)가 있어요. (자세한 뜻은 05.3·05.4)

TIP 비유: 선생님이 **답지가 있는 문제집**으로 가르치는 것. 틀리면 정답을 보고 고쳐요.



▲ 머신러닝 모델 지도: 지도학습과 비지도학습의 대표 모델

지도학습의 대표 모델 9가지를 아래 **05.1.1 ~ 05.1.9** 에서 하나씩 소개해요. 예제는 모두 붓꽃(iris) 데이터로 같은 문제를 풀어서 정확도를 비교해 볼 수 있게 했어요.

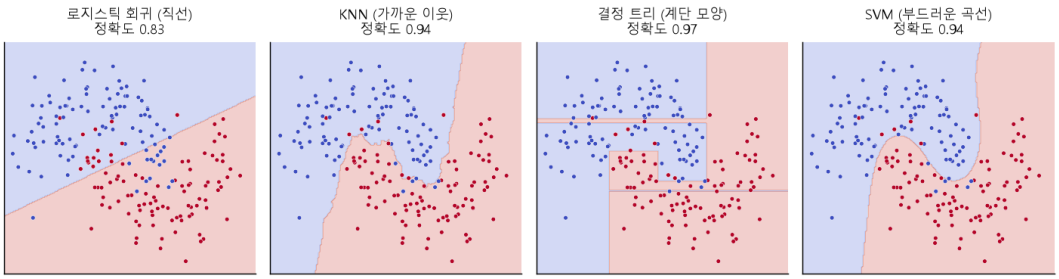
파이썬 코드

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# 붓꽃 150송이: 꽃잎·꽃받침 길이와 너비(피처 4개) → 품종 3가지(타깃)
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)
print(X_tr.shape, X_te.shape)       # 훈련 105송이, 테스트 45송이
```

실행 결과

```
(105, 4) (45, 4)
```



▲ 같은 데이터를 모델마다 다르게 나누는 모습(결정 경계)

05.1.1 선형 회귀 · Linear Regression

Linear Regression
직선(y = ax + b)으로 숫자를 예측하는 가장 기본 회귀 모델

피처와 타깃 사이에 **곧은 직선** 관계가 있다고 보고, 가장 잘 맞는 직선을 찾아 **숫자를 예측**해요. 집값, 매출 예측의 출발점이에요. 계수(기울기)를 보면 "어떤 피처가 얼마나 영향을 주는지" 설명할 수 있어서 해석하기 쉬워요. (10장에서 더 자세히)

- 쓰는 곳: 회귀 · 장점: 빠르고 설명하기 쉬움 · 단점: 휘어진 관계는 못 잡음

파이썬 코드

```
from sklearn.linear_model import LinearRegression
# 꽃잎 길이(2번 열)로 꽃잎 너비(3번 열) 예측하기
reg = LinearRegression().fit(X_tr[:, [2]], X_tr[:, 3])
print("기울기", reg.coef_.round(3), "절편", round(reg.intercept_, 3))
print("R² =", round(reg.score(X_te[:, [2]], X_te[:, 3]), 3))
```

실행 결과

```
기울기 [0.419] 절편 -0.375
R² = 0.896
```

05.1.2 로지스틱 회귀 · Logistic Regression

Logistic Regression
확률(0~1)을 계산해 범주를 고르는 분류 모델

이름에 '회귀'가 있지만 **분류 모델**이에요. 직선 계산 결과를 **S자 곡선(시그모이드)** 에 넣어 0~1 사이의 **확률**로 바꾸고, 확률이 높은 쪽을 정답으로 골라요. "합격 확률 83%" 처럼 확률을 함께 알려 줘요.

- 쓰는 곳: 분류 (스팸, 이탈 예측) · 장점: 빠르고 확률을 줌 · 단점: 경계가 직선이라 복잡한 모양은 약함

파이썬 코드

```
from sklearn.linear_model import LogisticRegression
m = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
print("정확도", round(m.score(X_te, y_te), 3))
print("첫 꽃의 품종별 확률", m.predict_proba(X_te[:1]).round(2))
```

실행 결과

```
정확도 0.978
첫 꽃의 품종별 확률 [[0.   0.06 0.94]]
```

05.1.3 K-최근접 이웃 · KNN

KNN

K-Nearest Neighbors, 가장 가까운 이웃 K개의 다수결로 예측

새 데이터와 **가장 가까운 이웃 K개**를 찾아서, 이웃들이 가장 많이 속한 쪽으로 정해요(분류). 회귀에서는 이웃 값의 평균을 써요. 따로 "공부"하지 않고 데이터를 외워 두었다가 물어볼 때 비교해요.

TIP 비유: 이사 온 동네에서 **가까운 이웃 5집**이 다 짜장면을 시키면 나도 짜장면 집을 고르는 것.

- 장점: 이해하기 쉬움 · 단점: 데이터가 많으면 느림, 거리를 쓰니 **표준화**가 꼭 필요

파이썬 코드

```
from sklearn.neighbors import KNeighborsClassifier
for k in [1, 5, 15]:                    # K(이웃 수)는 하이퍼파라미터
    m = KNeighborsClassifier(n_neighbors=k).fit(X_tr, y_tr)
    print("K =", k, "정확도", round(m.score(X_te, y_te), 3))
```

실행 결과

```
K = 1 정확도 0.978
K = 5 정확도 0.978
K = 15 정확도 0.978
```

05.1.4 결정 트리 · Decision Tree

Decision Tree

예/아니오 질문을 이어 가며 답을 찾는 나무 모양 모델

"꽃잎 길이가 2.5cm 보다 작은가?" 같은 **예/아니오 질문**을 차례로 던져 가지를 나누고, 마지막 잎(leaf)에서 답을 정해요. 스무고개와 같아요. 규칙을 사람이 그대로 읽을 수 있어서 **설명하기 좋아요.**

- 장점: 해석 쉬움, 표준화 필요 없음 · 단점: 깊게 만들면 **과적합**되기 쉬움 (max_depth 로 조절)

파이썬 코드

```
from sklearn.tree import DecisionTreeClassifier, export_text
tree = DecisionTreeClassifier(max_depth=2, random_state=0).fit(X_tr, y_tr)
print("정확도", round(tree.score(X_te, y_te), 3))
print(export_text(tree, feature_names=["꽃받침길이", "꽃받침너비", "꽃잎길이", "꽃잎너비"]))   # 질문 규칙 보기
```

실행 결과
```
정확도 0.911
|--- 꽃잎너비 <= 0.75
|   |--- class: 0
|--- 꽃잎너비 >  0.75
|   |--- 꽃잎길이 <= 4.95
|   |   |--- class: 1
|   |--- 꽃잎길이 >  4.95
|   |   |--- class: 2
```

05.1.5 랜덤 포레스트 · Random Forest

Random Forest
결정 트리 여러 그루의 투표로 예측하는 앙상블 모델

결정 트리를 **수백 그루** 만들어 **투표(다수결)** 로 정해요. 나무마다 데이터와 피처를 **무작위(random)** 로 조금씩 다르게 줘서, 한 나무의 실수를 다른 나무들이 바로잡아요. 이렇게 여러 모델을 합치는 방법을 **앙상블(ensemble)** 이라 해요.

TIP 비유: 한 명의 의견보다 **여러 사람의 투표**가 더 믿을 만한 것과 같아요.

- 장점: 성능이 좋고 과적합에 강함, 피처 중요도를 알려 줌 · 단점: 나무가 많아 규칙을 한눈에 읽기 어려움

파이썬 코드
```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=0).fit(X_tr, y_tr)
print("정확도", round(rf.score(X_te, y_te), 3))
print("피처 중요도", rf.feature_importances_.round(2))   # 꽃잎 길이·너비가 중요
```

실행 결과
```
정확도 0.978
피처 중요도 [0.1  0.02 0.48 0.39]
```

05.1.6 그레이디언트 부스팅 · Gradient Boosting

Gradient Boosting
앞 나무가 틀린 부분을 다음 나무가 고쳐 가며 쌓는 앙상블 모델

랜덤 포레스트가 나무들을 **동시에 따로** 만든다면, 부스팅은 나무를 **차례로** 만들어요. 다음 나무는 **앞 나무들이 틀린 부분**을 집중해서 고쳐요. 표 형태 데이터에서 가장 성능이 좋은 편이라 대회·실무에서 많이 써요. **XGBoost, LightGBM, CatBoost** 가 유명한 부스팅 라이브러리예요.

TIP 비유: 오답 노트를 보며 **틀린 문제만 다시 공부**하기를 반복하는 것.

파이썬 코드
```
from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier(random_state=0).fit(X_tr, y_tr)
print("정확도", round(gb.score(X_te, y_te), 3))
```

실행 결과
```
정확도 0.978
```

05.1.7 서포트 벡터 머신 · SVM

SVM
Support Vector Machine, 두 그룹 사이의 간격(여백)이 가장 넓은 경계를 찾는 모델

두 그룹을 나누는 선 중에서 **양쪽과의 간격(margin)이 가장 넓은 선**을 골라요. 경계에 가장 가까운 점들을 **서포트 벡터**라 불러요. **커널(kernel)** 이라는 기법으로 휘어진 경계도 만들 수 있어요.

TIP 비유: 두 마을 사이에 길을 낼 때 **양쪽 집에서 최대한 멀리** 넓은 도로를 내는 것.

- 장점: 데이터가 많지 않을 때 성능 좋음 · 단점: 데이터가 아주 많으면 느림, 표준화 필요

파이썬 코드
```
from sklearn.svm import SVC
svm = SVC(kernel="rbf").fit(X_tr, y_tr)
print("정확도", round(svm.score(X_te, y_te), 3))
```

실행 결과
```
정확도 0.978
```

05.1.8 나이브 베이즈 · Naive Bayes

Naive Bayes
확률(베이즈 정리)로 가장 그럴듯한 범주를 고르는 분류 모델

"이 단어들이 나왔을 때 스팸일 확률은?"처럼 **확률을 계산**해 가장 가능성이 높은 범주를 골라요. 피처들이 서로 **관계없다고 단순하게(naive, 순진하게) 가정**해서 계산이 아주 빨라요. 스팸 메일·문서 분류에 많이 써요.

파이썬 코드
```
from sklearn.naive_bayes import GaussianNB
nb = GaussianNB().fit(X_tr, y_tr)
print("정확도", round(nb.score(X_te, y_te), 3))
```

실행 결과
```
정확도 1.0
```

05.1.9 신경망 · Neural Network

Neural Network
뇌의 신경세포를 본뜬 층(layer)을 쌓아 복잡한 패턴을 배우는 모델

입력 → **여러 층의 노드(뉴런)** → 출력으로 이어진 모델이에요. 층마다 숫자를 곱하고 더하며 복잡한 관계를 배워요. 층을 아주 깊게 쌓은 것이 **딥러닝**이고, 사진·음성·글(ChatGPT 등)에 강해요. sklearn 의 `MLPClassifier`(다층 퍼셉트론)로 간단히 써 볼 수 있어요.

- 장점: 아주 복잡한 패턴도 학습 · 단점: 데이터와 계산이 많이 필요, 왜 그렇게 답했는지 설명하기 어려움

파이썬 코드
```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
sc = StandardScaler().fit(X_tr)                    # 신경망은 표준화가 중요
mlp = MLPClassifier(hidden_layer_sizes=(10,), max_iter=2000, random_state=0).fit(sc.transform(X_tr), y_tr)
print("정확도", round(mlp.score(sc.transform(X_te), y_te), 3))
```

실행 결과
```
정확도 0.978
```

모델	분류	회귀	한 줄 특징
선형 회귀	-	○	직선, 해석 쉬움
로지스틱 회귀	○	-	확률을 주는 분류 기본 모델
KNN	○	○	가까운 이웃 다수결
결정 트리	○	○	예/아니오 질문, 설명 쉬움
랜덤 포레스트	○	○	나무 여러 그루 투표
그레이디언트 부스팅	○	○	틀린 것 고쳐 가며 쌓기, 성능 최상위
SVM	○	○	간격이 가장 넓은 경계
나이브 베이즈	○	-	확률 계산, 아주 빠름
신경망	○	○	층을 쌓아 복잡한 패턴

TIP 처음에는 **로지스틱 회귀(또는 선형 회귀)로 기준 점수**를 내고, **랜덤 포레스트·부스팅**으로 올려 보는 순서를 많이 써요. 회귀용은 이름 끝이 Regressor 예요 (예: `RandomForestRegressor`).

05.2 비지도학습 · Unsupervised Learning

Unsupervised Learning
정답 없이 데이터 속 구조를 찾는 방법

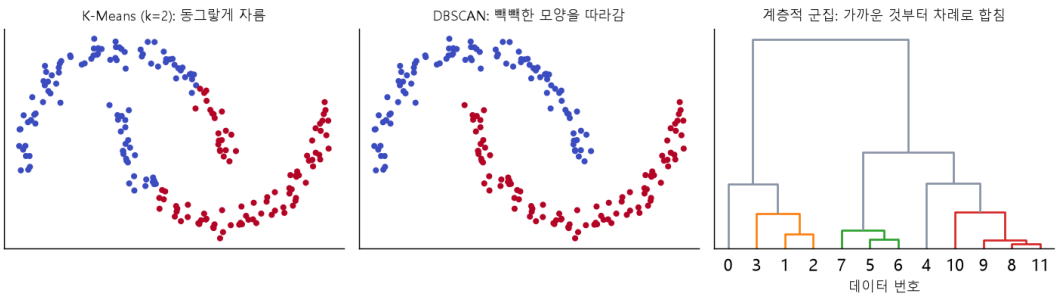
정답 없이 데이터만 주고, 비슷한 것끼리 묶거나 숨은 구조를 찾게 해요. 대표적으로 **군집화**와 **차원 축소**가 있어요.

TIP 비유: 정답 없이 사진 1000장을 주고 "비슷한 것끼리 나눠 봐" 하는 것. 컴퓨터가 알아서 고양이끼리, 강아지끼리 묶어요(이름은 몰라도).

비지도학습의 대표 모델 7가지를 아래 **05.2.1 ~ 05.2.7** 에서 소개해요. 붓꽃 데이터에서 **정답(품종)은 빼고** 피처만 써요.

파이썬 코드
```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
X, y = load_iris(return_X_y=True)          # y(품종)는 나중에 '맞게 묶였나' 확인할 때만 사용
Xs = StandardScaler().fit_transform(X)      # 거리를 쓰는 모델이 많아 표준화
print(Xs.shape)
```

실행 결과
```
(150, 4)
```



▲ 군집 모델 비교: K-Means, DBSCAN, 계층적 군집

05.2.1 K-평균 군집 · K-Means

K-Means
중심점 K개를 기준으로 가까운 데이터끼리 묶는 군집 모델

그룹 수 **K** 를 정해 주면, 중심점을 놓고 → 가까운 점을 묶고 → 중심을 평균 위치로 옮기기를 반복해요. 가장 많이 쓰는 군집 모델이에요. (10장 KMeans·Elbow 에서 자세히)

- 장점: 빠르고 간단 · 단점: K 를 미리 정해야 함, 동그란 모양의 무리만 잘 찾음

파이썬 코드
```
from sklearn.cluster import KMeans
import pandas as pd
km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(Xs)
print(pd.crosstab(km.labels_, y, rownames=["군집"], colnames=["실제 품종"]))   # 정답과 비교
```

실행 결과
```
실제 품종   0   1   2
군집
0       0  39  14
1      50   0   0
2       0  11  36
```

05.2.2 계층적 군집 · Hierarchical Clustering

Hierarchical Clustering
가장 가까운 것부터 차례로 합쳐 나무 모양(덴드로그램)을 만드는 군집

처음엔 데이터 하나하나가 각자 그룹이에요. **가장 가까운 두 그룹을 합치기**를 계속해서 결국 하나가 될 때까지 쌓아요. 그 과정을 그린 나무 그림이 **덴드로그램**이고, 원하는 높이에서 잘라 그룹 수를 정해요.

TIP 비유: 족보(가계도)처럼 가까운 친척부터 묶어 올라가는 것.

파이썬 코드
```
from sklearn.cluster import AgglomerativeClustering
hc = AgglomerativeClustering(n_clusters=3).fit(Xs)
print(pd.Series(hc.labels_).value_counts().sort_index().tolist())   # 군집별 개수
```

실행 결과
```
[71, 49, 30]
```

05.2.3 DBSCAN

> **DBSCAN**
> **점이 빽빽하게 모인 곳을 따라 묶고, 외딴 점은 이상치로 남기는 군집**

점이 빽빽한(밀도가 높은) 곳을 이어 가며 하나의 군집으로 묶어요. K 를 정하지 않아도 되고, **초승달처럼 휜 모양**도 잘 찾아요. 어디에도 속하지 않는 외딴 점은 **-1(잡음, 이상치)** 로 표시해요.

- 하이퍼파라미터: `eps`(이웃으로 볼 거리), `min_samples`(빽빽하다고 볼 최소 점 수)

파이썬 코드
```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.6, min_samples=5).fit(Xs)
print(sorted(set(db.labels_.tolist())))          # -1 은 잡음(이상치)
print("잡음으로 본 점:", (db.labels_ == -1).sum(), "개")
```

실행 결과
```
[-1, 0, 1]
잡음으로 본 점: 26 개
```

05.2.4 주성분 분석 · PCA

> **PCA**
> **Principal Component Analysis, 정보를 가장 많이 담은 새 축으로 차원을 줄이는 방법**

여러 피처를 섞어서 **정보(분산)를 가장 많이 담는 새 축(주성분)** 을 만들고, 중요한 축 몇 개만 남겨 차원을 줄여요. 피처 4개를 2개로 줄여 **그래프로 그려 보거나**, 모델을 가볍게 할 때 써요.

파이썬 코드
```
from sklearn.decomposition import PCA
pca = PCA(n_components=2).fit(Xs)
print("주성분 2개가 담은 정보 비율", pca.explained_variance_ratio_.round(3), "합계",
round(pca.explained_variance_ratio_.sum(), 3))
```

실행 결과
```
주성분 2개가 담은 정보 비율 [0.73  0.229] 합계 0.958
```

05.2.5 t-SNE

> **t-SNE**
> **고차원 데이터를 2~3차원 그림으로 보여 주는 시각화용 차원 축소**

고차원에서 **가까웠던 점은 그림에서도 가깝게** 놓이도록 2차원(또는 3차원)으로 펼쳐요. 비슷한 데이터가 덩어리로 보여서 **눈으로 군집을 확인**할 때 좋아요. 다만 계산이 느리고 축 자체에는 뜻이 없어서, 모델 입력보다는 **그림 그리기용**으로 써요.

파이썬 코드
```
from sklearn.manifold import TSNE
emb = TSNE(n_components=2, perplexity=30, random_state=0).fit_transform(Xs)
print(emb.shape)            # 150개 점 → 2차원 좌표
```

실행 결과
```
(150, 2)
```

05.2.6 연관 규칙 · Association Rules

Association Rules
"A를 사면 B도 산다" 같은 함께 일어나는 규칙을 찾는 방법

장바구니 데이터에서 **함께 자주 사는 상품 규칙**을 찾아요(장바구니 분석). 세 가지 숫자로 규칙을 평가해요.

- **지지도(support)**: A와 B를 함께 산 비율 (얼마나 흔한가)
- **신뢰도(confidence)**: A를 산 사람 중 B도 산 비율
- **향상도(lift)**: 신뢰도 ÷ B 구매율. **1보다 크면** A가 B 구매를 끌어올린다는 뜻

파이썬 코드
```
baskets = pd.DataFrame({           # 1 = 구매, 0 = 구매 안 함 (장바구니 6개)
    "빵":  [1, 1, 1, 0, 1, 1],
    "우유": [1, 1, 0, 1, 1, 0],
    "잼":  [1, 0, 1, 0, 1, 1],
})
both = (baskets["빵"] & baskets["잼"]).mean()          # 빵과 잼을 함께 산 비율
conf = both / baskets["빵"].mean()                     # 빵을 산 사람 중 잼도 산 비율
lift = conf / baskets["잼"].mean()
print(f"빵→잼  지지도 {both:.2f}, 신뢰도 {conf:.2f}, 향상도 {lift:.2f}")
```

실행 결과
```
빵→잼  지지도 0.67, 신뢰도 0.80, 향상도 1.20
```

TIP 대표 알고리즘은 **Apriori** 예요. "기저귀를 사는 사람이 맥주도 산다" 같은 진열·추천 전략에 써요.

05.2.7 이상 탐지 · Anomaly Detection

Anomaly Detection
대부분과 다른 드문 데이터(이상치)를 찾아내는 방법

정상 데이터의 패턴을 배우고, 거기서 **크게 벗어난 데이터**를 찾아요. 카드 사기 거래, 공장 설비 고장, 해킹 탐지에 써요. 대표 모델 **Isolation Forest** 는 "몇 번 만에 따로 떼어 낼 수 있나"를 보고, **빨리 고립되는 점일수록 이상치**로 판단해요.

파이썬 코드
```
from sklearn.ensemble import IsolationForest
import numpy as np
pay = np.array([[12], [15], [11], [14], [13], [16], [12], [250]])   # 결제 금액(천 원)
iso = IsolationForest(contamination=0.13, random_state=0).fit(pay)
print(iso.predict(pay))      # 1 = 정상, -1 = 이상
```

실행 결과
```
[ 1  1  1  1  1  1  1 -1]
```

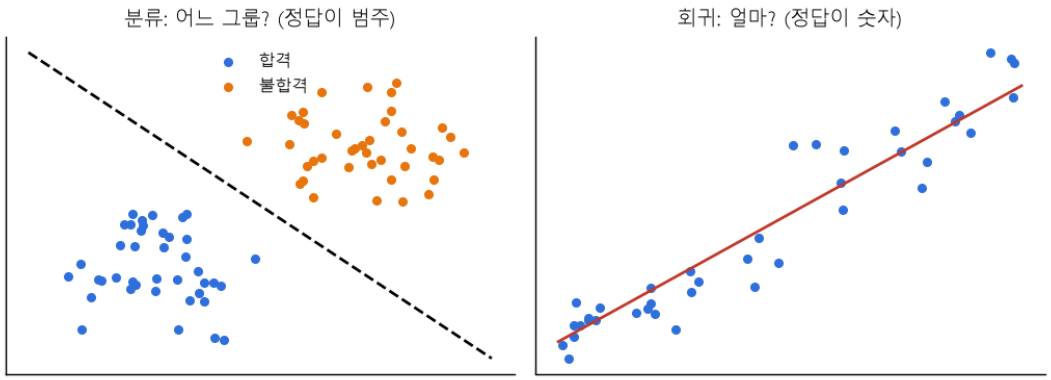
모델	하는 일	한 줄 특징
K-Means	군집	K 개 중심점, 빠르고 간단
계층적 군집	군집	덴드로그램으로 그룹 수 고르기
DBSCAN	군집	모양 자유, 이상치 표시, K 불필요
PCA	차원 축소	정보를 많이 담은 축만 남기기

모델	하는 일	한 줄 특징
t-SNE	차원 축소	그림으로 보기 위한 2차원 펼치기
연관 규칙	규칙 찾기	함께 사는 상품 (지지도·신뢰도·향상도)
이상 탐지	이상치 찾기	사기·고장 탐지, Isolation Forest

05.3 분류 · Classification

Classification
정해진 범주 중 하나를 예측하는 문제

정답이 **범주(종류)** 인 문제예요. 스팸/정상, 합격/불합격, 고양이/강아지/새. 답이 "어느 칸에 들어가나?"예요.



▲ 분류는 칸 나누기, 회귀는 숫자 맞히기

파이썬 코드
```
from sklearn.tree import DecisionTreeClassifier

# 공부시간, 출석일 → 합격(1)/불합격(0)
X = [[1, 5], [2, 8], [6, 15], [8, 18], [3, 9], [7, 17]]
y = [0, 0, 1, 1, 0, 1]
model = DecisionTreeClassifier(random_state=0).fit(X, y)   # 학습
print(model.predict([[5, 14], [1, 4]]))                    # 새 학생 2명 예측
```

실행 결과
```
[1 0]
```

05.4 회귀 · Regression

Regression
연속된 숫자 값을 예측하는 문제

정답이 **숫자**인 문제예요. 집값, 내일 기온, 매출액처럼 "얼마?"를 맞혀요. (분류: 어느 칸? / 회귀: 얼마?)

파이썬 코드
```
from sklearn.linear_model import LinearRegression

X = [[1], [2], [3], [4], [5]]          # 공부시간
y = [55, 62, 68, 74, 81]               # 점수
reg = LinearRegression().fit(X, y)
print(reg.predict([[6]]).round(1))     # 6시간 공부하면 몇 점?
```

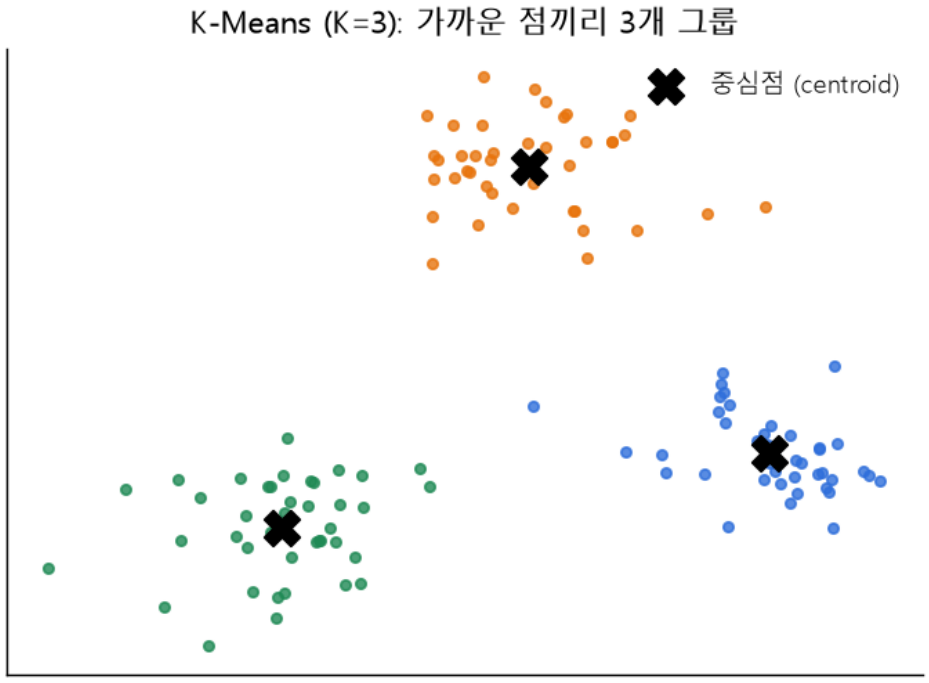
실행 결과
```
[87.2]
```

TIP "회귀"라는 이름의 유래는 9장 Regression 에서 자세히 볼게요.

05.5 군집화 · Clustering

Clustering
비슷한 데이터끼리 그룹으로 묶는 것

정답 없이 **비슷한 것끼리 그룹(군집)** 으로 묶어요. 고객을 구매 패턴으로 나눠 "알뜰형·충성형·신규형"처럼 마케팅에 써요. 대표 알고리즘이 K-Means(9장)예요.



▲ K-Means 군집화

05.6 차원 축소 · Dimensionality Reduction

Dimensionality Reduction
변수(차원) 개수를 줄이면서 정보는 최대한 남기는 것

피처(열)가 너무 많을 때 **핵심만 남겨 개수를 줄이는** 거예요. 계산이 빨라지고, 2~3차원으로 줄이면 그래프로 볼 수도 있어요. 대표 방법이 **PCA**(주성분 분석)예요.

파이썬 코드

```
from sklearn.decomposition import PCA
import numpy as np

rng = np.random.default_rng(0)
base = rng.normal(size=(50, 1))
X4 = np.hstack([base, base * 2, base + rng.normal(0, 0.1, (50, 1)), rng.normal(size=(50, 1))])  # 4개 열(3개는 비슷)
pca = PCA(n_components=2).fit(X4)
print(pca.explained_variance_ratio_.round(2))   # 2개로 줄여도 정보의 몇 % 가 남나
```

실행 결과

```
[0.84 0.15]
```

TIP 비유: 사람을 키·몸무게·발 크기·팔 길이로 설명하는 대신 "체격" 하나로 요약하는 것과 비슷해요.

05.7 훈련 / 검증 / 테스트 데이터 · Train / Validation / Test

Train / Validation / Test
학습용 / 조정용 / 최종 평가용으로 나눈 데이터

데이터를 세 덩어리로 나눠요.

- **훈련(Train)**: 모델이 **공부하는** 데이터 (교과서)
- **검증(Validation)**: 설정을 **고르고 조정하는** 데이터 (모의고사)
- **테스트(Test)**: 마지막에 **딱 한 번 실력을 재는** 데이터 (수능)



▲ 데이터 나누기 60 / 20 / 20

파이썬 코드

```
from sklearn.model_selection import train_test_split
data = list(range(100))
train, test = train_test_split(data, test_size=0.2, random_state=42)   # 80 : 20
train, val = train_test_split(train, test_size=0.25, random_state=42)  # 80의 25% = 20
print(len(train), len(val), len(test))
```

실행 결과

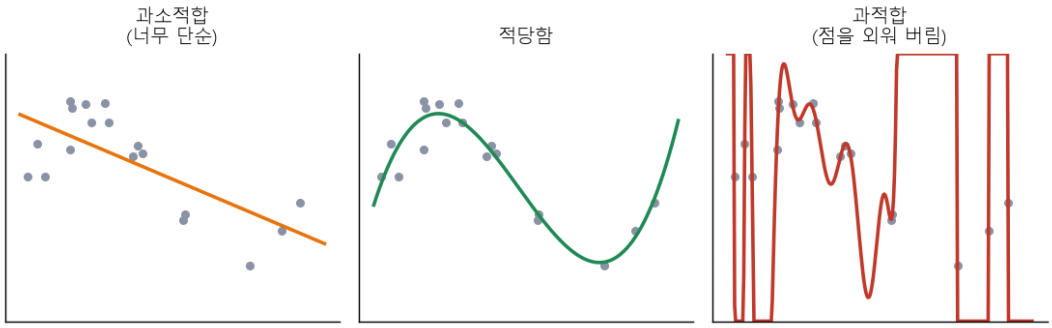
```
60 20 20
```

주의 테스트 데이터를 보며 모델을 고치면 안 돼요. 그러면 테스트가 "모의고사"가 되어 진짜 실력을 모르게 돼요(데이터 누수의 한 종류).

05.8 과적합 · Overfitting

Overfitting
훈련 데이터에만 너무 맞춰져 새 데이터에서 성능이 떨어지는 현상

모델이 훈련 데이터를 **통째로 외워 버려서**, 처음 보는 데이터에는 약한 상태예요. 훈련 점수는 높은데 테스트 점수가 낮아요.



▲ 과소적합 · 적당함 · 과적합

TIP 비유: 기출문제 **답만 외운** 학생. 똑같은 문제는 100점, 숫자만 바꿔도 못 풀어요.

해결: 데이터 더 모으기, 모델을 단순하게, 규제(regularization), 교차 검증.

05.9 과소적합 · Underfitting

Underfitting
모델이 너무 단순해 데이터의 패턴을 제대로 못 배운 상태

모델이 **너무 단순해서** 훈련 데이터조차 잘 못 맞히는 상태예요. 훈련 점수도, 테스트 점수도 둘 다 낮아요.

TIP 비유: 공부를 **너무 대충** 한 학생. 쉬운 문제도 틀려요.

해결: 더 복잡한 모델, 더 좋은 피처(피처 엔지니어링), 학습 더 오래.

상태	훈련 점수	테스트 점수
과소적합	낮음	낮음
적당함	높음	높음 (비슷)
과적합	매우 높음	낮음 (차이 큼)

05.10 하이퍼파라미터 · Hyperparameter

Hyperparameter
학습 전에 사람이 정해 주는 모델 설정값

모델이 **스스로 배우는 값이 아니라, 사람이 미리 정하는 설정**이에요. 결정 트리의 최대 깊이(max_depth), K-Means 의 그룹 수(k), 학습률 등이 있어요. 검증 데이터로 여러 값을 시험해 가장 좋은 걸 골라요.

파이썬 코드

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)
for depth in [1, 2, 3, 10]:                        # 하이퍼파라미터 바꿔 보기
    m = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(X_tr, y_tr)
    print(depth, round(m.score(X_tr, y_tr), 2), round(m.score(X_te, y_te), 2))   # 훈련, 테스트 정확도
```

실행 결과

```
1 0.7 0.6
2 0.96 0.91
3 0.98 0.98
10 1.0 0.98
```

TIP **파라미터**(모델이 배우는 값, 예: 회귀의 기울기) vs **하이퍼파라미터**(사람이 정하는 값). 오븐으로 비유하면 온도·시간은 하이퍼파라미터, 빵이 부푸는 건 모델이 알아서 해요.

총정리 퀴즈

Q1. 정답(레이블)이 있는 데이터로 학습하는 방법은?

① 지도학습
② 비지도학습
③ 군집화
④ 차원 축소

Q2. '내일 기온 예측'은 어떤 문제인가요?

① 분류
② 회귀
③ 군집화
④ 차원 축소

Q3. 고객을 정답 없이 비슷한 그룹으로 나누는 것은?

① 분류
② 회귀
③ 군집화
④ 테스트

Q4. 훈련 점수 99%, 테스트 점수 60% 라면?

① 과적합
② 과소적합
③ 적당함
④ 데이터 누수 없음

Q5. 다음 중 하이퍼파라미터는?

① 회귀식의 기울기
② 결정 트리의 max_depth
③ 예측 결과
④ 타깃 값

Q6. 마지막에 딱 한 번 실력을 재는 데이터는?

① 훈련 데이터
② 검증 데이터
③ 테스트 데이터
④ 원본 데이터

Q7. 결정 트리 여러 그루의 투표로 예측하는 모델은?

① KNN
② 랜덤 포레스트
③ 로지스틱 회귀
④ PCA

Q8. 다음 중 비지도학습 모델은?

① 로지스틱 회귀
② 서포트 벡터 머신
③ DBSCAN
④ 그레이디언트 부스팅

Q9. 이름에 '회귀'가 있지만 실제로는 분류에 쓰는 모델은?

① 로지스틱 회귀
② 선형 회귀
③ K-Means
④ t-SNE

CHAPTER 06

모델 평가 용어

학습 목표

- 혼동행렬의 TP·FP·FN·TN 을 읽는다.
- 정확도·정밀도·재현율·F1·ROC-AUC 를 언제 쓰는지 안다.
- 회귀 평가 지표 MAE·MSE·RMSE·R^2 를 계산하고 해석한다.

06.0 들어가며: 모델 성적표

모델을 만들었으면 **얼마나 잘 맞히는지** 숫자로 재야 해요. 분류와 회귀는 평가 방법이 달라요.

- **분류**: 맞혔다/틀렸다를 센다 → 정확도, 정밀도, 재현율, F1, ROC-AUC
- **회귀**: 얼마나 차이 나는지 잰다 → MAE, MSE, RMSE, R^2

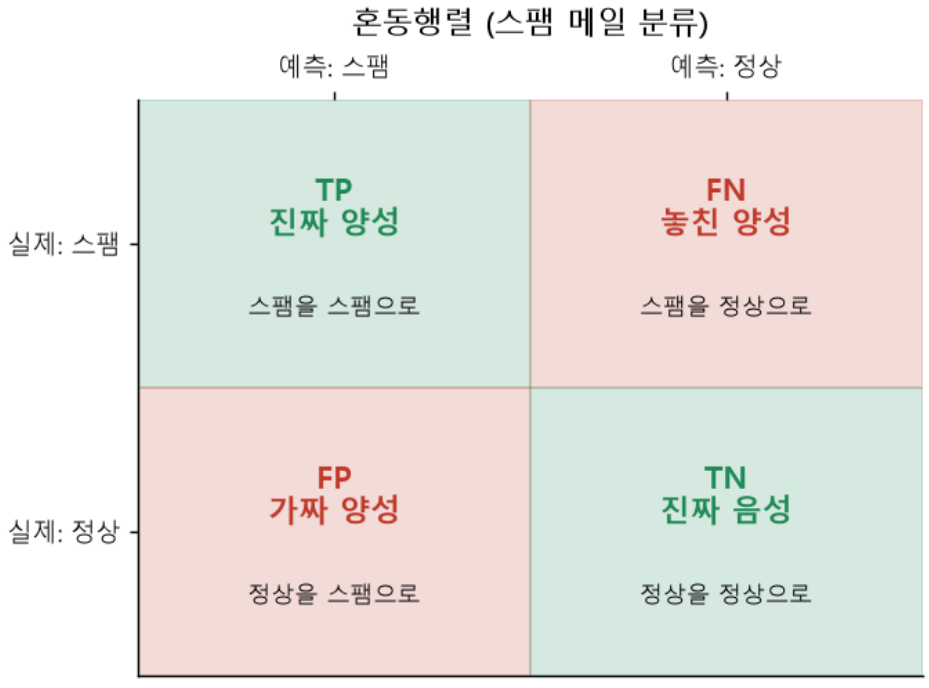
이 장의 분류 예시는 **스팸 메일 필터**예요. 스팸 = 양성(Positive, 1), 정상 = 음성(Negative, 0).

06.1 혼동행렬 · Confusion Matrix

> **Confusion Matrix**
> **실제 정답과 예측 결과를 2×2 표로 정리한 것**

실제와 예측을 비교해 **네 칸**으로 나눈 표예요.

- **TP** (True Positive): 스팸을 스팸이라고 맞힘
- **TN** (True Negative): 정상을 정상이라고 맞힘
- **FP** (False Positive): 정상을 스팸이라고 잘못 판단 (중요한 메일이 스팸함으로!)
- **FN** (False Negative): 스팸을 정상이라고 놓침 (스팸이 받은편지함으로)



▲ 혼동행렬: 스팸 필터 예

파이썬 코드

```
from sklearn.metrics import confusion_matrix
y_true = [1, 1, 1, 1, 0, 0, 0, 0, 0, 0]   # 실제 (1 = 스팸)
y_pred = [1, 1, 1, 0, 1, 0, 0, 0, 0, 0]   # 모델 예측
print(confusion_matrix(y_true, y_pred))    # [[TN FP] [FN TP]] 순서
```

실행 결과

```
[[5 1]
 [1 3]]
```

TIP 이름 읽는 법: 뒤 글자(P/N)는 **모델이 뭐라고 했나**, 앞 글자(T/F)는 **그게 맞았나**. FP = "양성이라 했는데(P) 틀렸다(F)".

06.2 정확도 · Accuracy

Accuracy
전체 중 맞힌 비율

전체 예측 중 **맞힌 비율**이에요. (TP + TN) / 전체.

파이썬 코드

```
from sklearn.metrics import accuracy_score
print(accuracy_score(y_true, y_pred))     # 10개 중 8개 맞힘
```

실행 결과

```
0.8
```

주의 **데이터가 한쪽으로 쏠리면 정확도가 속여요.** 환자가 1%인 데이터에서 "모두 정상"이라고만 해도 정확도 99%예요. 그래서 정밀도·재현율을 같이 봐요.

06.3 정밀도 · Precision

Precision
양성이라고 예측한 것 중 진짜 양성의 비율

모델이 **"스팸이다!" 라고 한 것 중 진짜 스팸**의 비율이에요. TP / (TP + FP).

FP(괜히 양성이라 한 것)가 치명적일 때 중요해요: 스팸 필터(중요 메일이 스팸함으로 가면 큰일), 유죄 판결.

파이썬 코드

```
from sklearn.metrics import precision_score
print(precision_score(y_true, y_pred))    # 스팸이라 한 4개 중 3개가 진짜
```

실행 결과

```
0.75
```

06.4 재현율 · Recall

Recall
실제 양성 중 모델이 찾아낸 비율

실제 스팸 중 모델이 찾아낸 비율이에요. TP / (TP + FN). 민감도(Sensitivity)라고도 해요.

FN(놓친 것)이 치명적일 때 중요해요: 암 진단(환자를 놓치면 큰일), 사기 거래 탐지.

파이썬 코드
```
from sklearn.metrics import recall_score
print(recall_score(y_true, y_pred))       # 실제 스팸 4개 중 3개 찾음
```

실행 결과
```
0.75
```

TIP 기억법 — 정밀도: "**내가 찍은 것**이 얼마나 정확해?" / 재현율: "**찾아야 할 것**을 얼마나 찾았어?" 둘은 보통 한쪽을 올리면 다른 쪽이 내려가요.

06.5 F1-score

F1-score
정밀도와 재현율의 조화평균

정밀도와 재현율을 **하나로 합친 점수**예요. 조화평균을 써서 **둘 중 하나라도 낮으면 점수가 낮게** 나와요. 데이터가 쏠려 있을 때 정확도 대신 많이 써요.

공식: 2 × 정밀도 × 재현율 / (정밀도 + 재현율)

파이썬 코드
```
from sklearn.metrics import f1_score, classification_report
print(round(f1_score(y_true, y_pred), 3))
print(classification_report(y_true, y_pred, target_names=["정상", "스팸"]))   # 한 번에 보기
```

실행 결과
```
0.75
              precision    recall  f1-score   support

          정상       0.83      0.83      0.83         6
          스팸       0.75      0.75      0.75         4

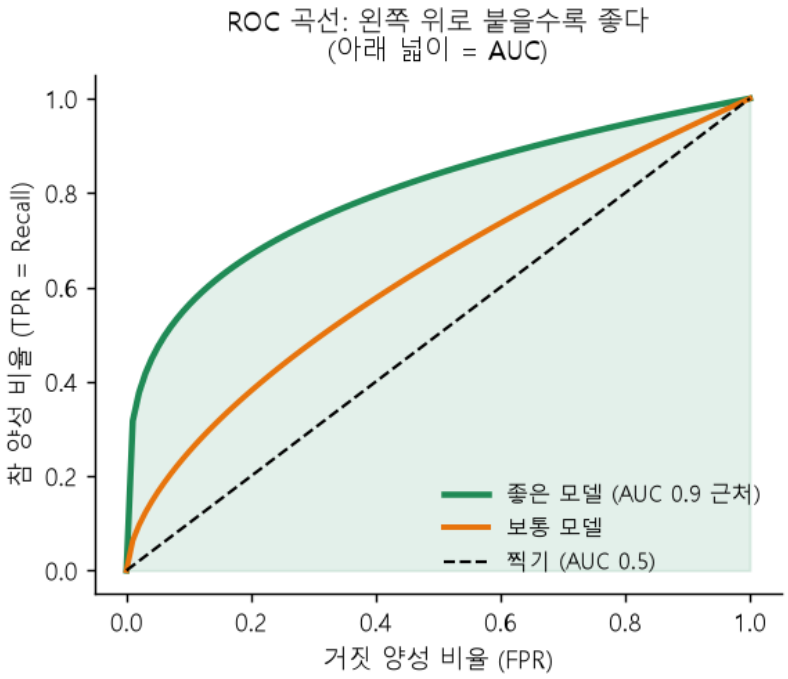
    accuracy                           0.80        10
   macro avg       0.79      0.79      0.79        10
weighted avg       0.80      0.80      0.80        10
```

06.6 ROC-AUC

ROC-AUC
분류 모델이 양성과 음성을 얼마나 잘 구별하는지 (0.5 ~ 1)

모델은 보통 "스팸일 확률 0.7" 처럼 **확률**을 내고, 기준(예: 0.5)을 넘으면 스팸이라 해요. 이 기준을 0~1 로 바꿔 가며 그린 곡선이 **ROC 곡선**, 그 아래 넓이가 **AUC** 예요.

- AUC = **1.0**: 완벽하게 구별
- AUC = **0.5**: 동전 던지기 수준 (대각선)
- 보통 0.8 이상이면 좋은 편



▲ ROC 곡선과 AUC

파이썬 코드

```
from sklearn.metrics import roc_auc_score
prob = [0.9, 0.8, 0.7, 0.4, 0.6, 0.3, 0.2, 0.2, 0.1, 0.1]   # 스팸일 확률
print(round(roc_auc_score(y_true, prob), 3))
```

실행 결과

```
0.958
```

TIP AUC 는 기준값을 정하지 않아도 모델 자체의 **구별 능력**을 비교할 수 있어서 편해요.

06.7 MAE · 평균 절대 오차

MAE

Mean Absolute Error, 오차 절댓값의 평균

회귀에서 **예측과 실제 차이(오차)** 의 절댓값을 평균 낸 거예요. "평균적으로 **몇 만큼** 틀리나"로 바로 읽을 수 있어요. 단위가 원래 데이터와 같아요.

파이썬 코드

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
actual = [100, 150, 200, 250]     # 실제 집값(백만 원)
pred   = [110, 140, 210, 230]     # 예측
print(mean_absolute_error(actual, pred))   # (10 + 10 + 10 + 20) / 4
```

실행 결과

```
12.5
```

06.8 MSE · 평균 제곱 오차

MSE
Mean Squared Error, 오차 제곱의 평균

오차를 **제곱해서** 평균 낸 거예요. 제곱하니까 **큰 오차를 더 크게 벌줘요.** (오차 20 → 400). 단위가 제곱이라 직접 읽기는 어렵고, 모델 학습(손실 함수)에 많이 써요.

파이썬 코드
```
print(mean_squared_error(actual, pred))    # (100 + 100 + 100 + 400) / 4
```

실행 결과
```
175.0
```

06.9 RMSE · 평균 제곱근 오차

RMSE
Root Mean Squared Error, MSE 의 제곱근

MSE 에 **루트**를 씌워 단위를 원래대로 돌린 거예요. MAE 처럼 "얼마나 틀리나"로 읽되, 큰 오차에 더 민감해요. 회귀에서 가장 많이 보는 지표예요.

파이썬 코드
```
import numpy as np
print(round(np.sqrt(mean_squared_error(actual, pred)), 2))   # 약 13.2
```

실행 결과
```
13.23
```

TIP MAE 10 인데 RMSE 가 13 이면 "가끔 크게 틀리는 경우가 있구나" 하고 읽어요. 둘의 차이가 클수록 큰 오차가 섞여 있어요.

06.10 R^2 · 결정계수

R^2
모델이 데이터의 변화를 얼마나 설명하는지 (보통 0 ~ 1)

"그냥 **평균으로만 찍었을 때**보다 얼마나 나은가"를 나타내요.

- **1**: 완벽하게 설명
- **0**: 평균으로 찍은 것과 같음
- **음수**: 평균으로 찍는 것보다도 못함

파이썬 코드
```
print(round(r2_score(actual, pred), 3))   # 1 에 가까울수록 좋음
```

실행 결과
```
0.944
```

TIP R^2 = 0.8 은 "점수 변화의 80% 를 모델(공부시간 등)로 설명할 수 있다"는 뜻이에요.

지표	쓰는 문제	좋은 값	한 줄 요약
Accuracy	분류	높을수록	전체 중 맞힌 비율
Precision	분류	높을수록	양성이라 한 것 중 진짜
Recall	분류	높을수록	진짜 양성 중 찾은 것
F1	분류	높을수록	정밀도·재현율의 균형
ROC-AUC	분류	1 에 가까울수록	구별 능력
MAE / RMSE	회귀	낮을수록	평균적으로 틀린 크기
R^2	회귀	1 에 가까울수록	설명력

총정리 퀴즈

Q1. 정상 메일을 스팸이라고 잘못 판단한 경우는?

① TP
② FP
③ FN
④ TN

Q2. 암 진단처럼 환자를 놓치면 안 될 때 가장 중요한 지표는?

① 정밀도
② 재현율
③ 정확도
④ MSE

Q3. 환자 1% 데이터에서 '모두 정상'이라 예측하면 정확도는?

① 1%
② 50%
③ 99%
④ 0%

Q4. ROC-AUC 가 0.5 라면?

① 완벽한 모델
② 동전 던지기 수준
③ 아주 좋은 모델
④ 계산 오류

Q5. 오차가 [10, -10, 20] 일 때 MAE 는?

① 10
② 약 13.3
③ 200
④ 0

Q6. R^2 가 0 이라면?

① 완벽하다
② 평균으로 찍은 것과 같다
③ 오차가 0 이다
④ 과적합이다

CHAPTER 07

비즈니스·KPI 용어

학습 목표

- KPI 가 무엇이고 어떻게 정하는지 안다.
- 전환율·유지율·이탈률·ARPU·LTV·CAC·AOV 를 계산한다.
- 코호트 분석과 A/B 테스트로 무엇을 알 수 있는지 안다.

07.0 들어가며: 숫자로 장사 실력 재기

회사에서 데이터 분석을 하는 이유는 결국 **"사업이 잘되고 있나? 무엇을 바꾸면 더 좋아질까?"** 에 답하기 위해서예요. 이 장의 용어는 쇼핑몰·앱 서비스에서 매일 쓰는 말이에요. 계산은 대부분 **나눗셈 하나**로 끝나니 겁먹지 마세요.

07.1 핵심성과지표 · KPI

KPI
Key Performance Indicator, 목표 달성 정도를 재는 핵심 지표

목표를 잘 이루고 있는지 확인하는 가장 중요한 숫자예요. 회사·팀마다 달라요. 쇼핑몰은 "월 매출, 구매 전환율", 앱은 "일 활성 사용자 수(DAU), 유지율" 같은 것이 KPI 가 돼요.

TIP 비유: 다이어트의 KPI 는 **몸무게**, 마라톤 연습의 KPI 는 **5km 기록**. 많은 숫자 중 "이것만은 꼭 본다"는 숫자예요.

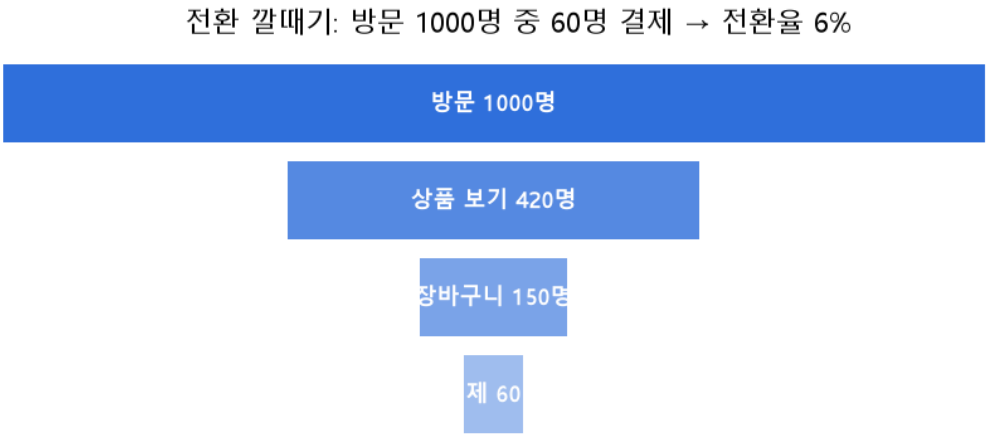
주의 좋은 KPI 는 **측정할 수 있고, 행동으로 바꿀 수 있어야** 해요. "고객 행복도 올리기"보다 "재구매율 30% → 35%"가 좋은 KPI 예요.

07.2 전환율 · Conversion Rate

Conversion Rate
방문자 중 목표 행동(구매·가입 등)을 한 사람의 비율

방문한 사람 중 **우리가 원하는 행동**(구매, 회원가입, 앱 설치)을 한 사람의 비율이에요.

공식: 목표 행동 수 / 방문자 수 × 100



▲ 방문 → 장바구니 → 구매 퍼널

파이썬 코드
```
visitors, buyers = 10000, 250
print(f"전환율 {buyers / visitors * 100:.1f}%")
```

실행 결과
```
전환율 2.5%
```

TIP 단계별로 사람이 줄어드는 모양을 **퍼널(깔때기)** 이라고 해요. 어느 단계에서 많이 빠지는지 보면 고칠 곳이 보여요.

07.3 유지율 · Retention Rate

Retention Rate
일정 기간 뒤에도 계속 이용하는 고객의 비율

처음 온 고객 중 **일정 기간 뒤에도 다시 쓰는** 고객의 비율이에요. "1주 차 유지율 40%" = 가입한 사람 중 40% 가 1주 뒤에도 앱을 썼다는 뜻.

파이썬 코드
```
start_users, still_active = 1000, 420
print(f"유지율 {still_active / start_users * 100:.0f}%")
```

실행 결과
```
유지율 42%
```

07.4 이탈률 · Churn Rate

Churn Rate
일정 기간 동안 서비스를 떠난 고객의 비율

일정 기간 동안 **떠난(해지한)** 고객의 비율이에요. 유지율의 반대예요: **이탈률 = 100% - 유지율** (같은 기간·같은 고객 기준).

파이썬 코드
```
print(f"이탈률 {(start_users - still_active) / start_users * 100:.0f}%")
```

실행 결과
```
이탈률 58%
```

TIP 구독 서비스(넷플릭스, 멜론)는 이탈률이 생명이에요. 새 고객 데려오기는 기존 고객 지키기보다 보통 훨씬 비싸거든요.

주의 웹 분석 도구의 **"이탈률(Bounce Rate)"** 은 다른 말이에요. 한 페이지만 보고 바로 나간 방문 비율이에요. 이름이 같으니 문맥을 확인하세요.

07.5 사용자당 평균 매출 · ARPU

ARPU
Average Revenue Per User, 사용자 한 명당 평균 매출

전체 매출 ÷ 사용자 수예요. 한 명이 평균 얼마를 벌어다 주는지 알려 줘요. 통신사·게임 회사가 많이 써요.

파이썬 코드
```
revenue, users = 50_000_000, 20_000
print(f"ARPU {revenue / users:,.0f}원")
```

실행 결과
```
ARPU 2,500원
```

TIP 비슷한 말: **ARPPU** = 돈을 낸 사용자(Paying User)만 기준으로 나눈 값이라 ARPU 보다 커요.

07.6 고객 생애 가치 · LTV / CLV

LTV / CLV
Life Time Value, 한 고객이 거래하는 동안 가져다주는 총 가치

한 고객이 **처음부터 떠날 때까지** 우리에게 가져다주는 돈의 합이에요. 간단한 계산법:

LTV ≒ 평균 구매 금액 × 구매 빈도 × 거래 기간

파이썬 코드
```
aov, per_year, years = 30000, 4, 3     # 한 번에 3만 원, 1년에 4번, 3년 이용
print(f"LTV 약 {aov * per_year * years:,}원")
```

실행 결과
```
LTV 약 360,000원
```

07.7 고객 획득 비용 · CAC

CAC
Customer Acquisition Cost, 새 고객 한 명을 데려오는 데 드는 비용

마케팅·광고비 ÷ 새로 얻은 고객 수예요.

파이썬 코드

```
ad_cost, new_customers = 6_000_000, 300
cac = ad_cost / new_customers
print(f"CAC {cac:,.0f}원, LTV/CAC = {360000 / cac:.1f}")
```

실행 결과

```
CAC 20,000원, LTV/CAC = 18.0
```

TIP **LTV > CAC** 여야 돈을 벌어요. 보통 LTV 가 CAC 의 **3배 이상**이면 건강하다고 봐요. 고객 한 명 데려오는 데 2만 원 썼는데 평생 1만 원만 쓰고 떠나면 손해예요.

07.8 평균 주문 금액 · AOV

AOV

Average Order Value, 주문 한 건당 평균 금액

총 매출 ÷ 주문 건수예요. ARPU 는 "사람 기준", AOV 는 "주문 기준"이에요. "2만 원 이상 무료배송", "함께 사면 할인"은 AOV 를 올리려는 전략이에요.

파이썬 코드

```
import pandas as pd
orders = pd.DataFrame({"고객": ["A", "A", "B", "C"], "금액": [20000, 30000, 15000, 35000]})
print("AOV", orders["금액"].mean())                          # 주문 기준
print("고객당 매출", orders.groupby("고객")["금액"].sum().mean())   # 사람 기준
```

실행 결과

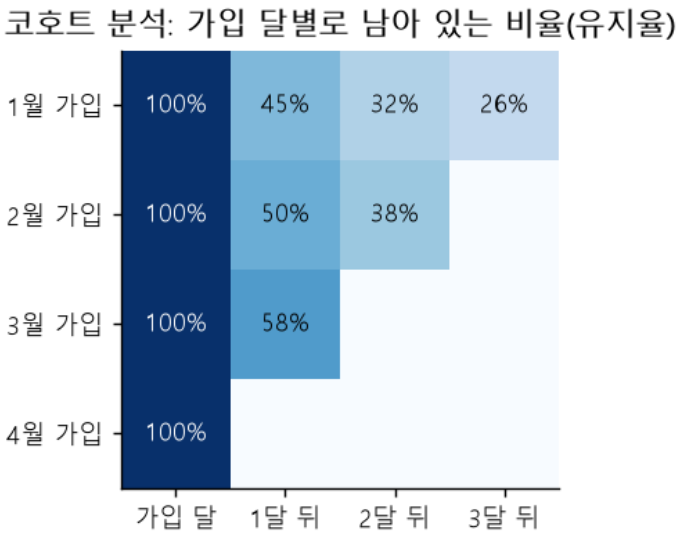
```
AOV 25000.0
고객당 매출 33333.333333333336
```

07.9 코호트 분석 · Cohort Analysis

Cohort Analysis

같은 시기·특징을 가진 고객 그룹을 묶어 시간에 따라 비교하는 분석

코호트는 같은 시기에 가입한 사람처럼 **공통점이 있는 그룹**이에요. "1월 가입자, 2월 가입자, 3월 가입자"를 따로 묶어 시간이 지나며 유지율이 어떻게 변하는지 비교해요.



▲ 월별 가입 코호트의 유지율 표

TIP 표를 읽는 법: **가로 한 줄** = 한 코호트의 시간 흐름, **세로 한 줄** = 같은 경과 기간에서 코호트끼리 비교. 3월 가입자의 1개월 차 유지율이 확 올랐다면 "3월에 바꾼 기능이 효과가 있었나?" 하고 살펴봐요.

파이썬 코드

```
log = pd.DataFrame({
    "고객": ["a", "a", "b", "b", "c", "d", "d"],
    "가입월": ["1월", "1월", "1월", "1월", "2월", "2월", "2월"],
    "경과월": [0, 1, 0, 2, 0, 0, 1],
})
cohort = log.pivot_table(index="가입월", columns="경과월", values="고객", aggfunc="nunique")
print(cohort.div(cohort[0], axis=0).round(2))   # 0개월 차 대비 비율 = 유지율
```

실행 결과

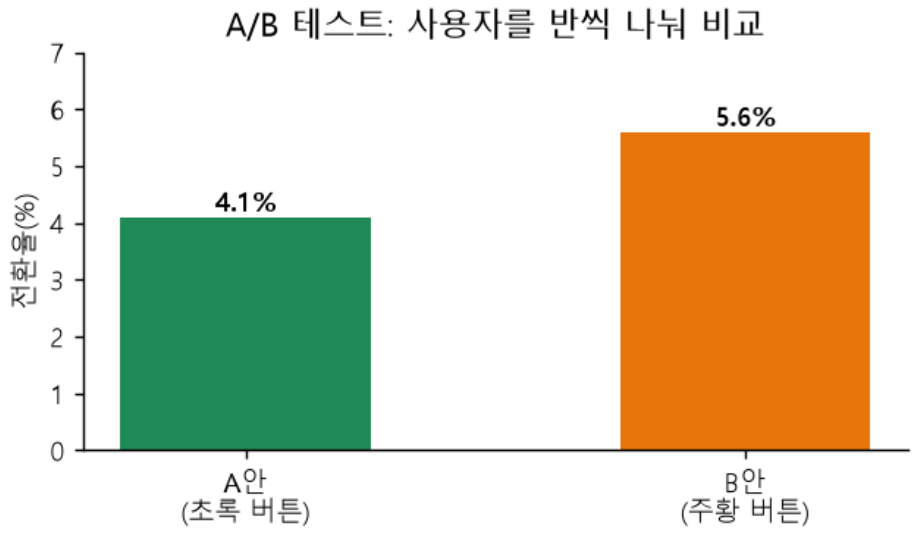
```
경과월    0    1    2
가입월
1월   1.0  0.5  0.5
2월   1.0  0.5  NaN
```

07.10 A/B 테스트 · A/B Test

A/B Test

두 가지 버전을 무작위로 나눠 보여 주고 어느 쪽이 나은지 비교하는 실험

사용자를 **무작위로 두 그룹**으로 나눠 A 그룹엔 기존 버튼(파란색), B 그룹엔 새 버튼(빨간색)을 보여 주고, 전환율을 비교해요. 그룹만 다르고 나머지는 같으니 차이는 **버튼 때문**이라고 말할 수 있어요.



▲ A/B 테스트 결과 비교

파이썬 코드

```
from scipy.stats import chi2_contingency
#            구매  안 함
table = [[120, 2380],     # A 안: 2500명 중 120명 구매 (4.8%)
         [160, 2340]]     # B 안: 2500명 중 160명 구매 (6.4%)
chi2, p, dof, _ = chi2_contingency(table)
print(f"p-value = {p:.4f}")   # 0.05 보다 작으면 '우연이라 보기 어렵다'
```

실행 결과

```
p-value = 0.0164
```

TIP **p-value** 는 "실제로는 차이가 없는데 우연히 이만큼 차이 날 확률"이에요. 보통 0.05(5%) 보다 작으면 "의미 있는 차이"라고 판단해요.

주의 사람 수가 너무 적거나 기간이 너무 짧으면 결과를 믿기 어려워요. 그리고 한 번에 **한 가지만** 바꿔야 무엇 때문인지 알 수 있어요.

총정리 퀴즈

Q1. 방문자 2,000명 중 50명이 구매했다면 전환율은?

① 0.5%
② 2.5%
③ 25%
④ 40%

Q2. 유지율이 70% 라면 같은 기준의 이탈률은?

① 30%
② 70%
③ 100%
④ 알 수 없다

Q3. 전체 매출 ÷ 주문 건수는?

① ARPU
② CAC
③ AOV
④ LTV

Q4. 건강한 사업이 되려면 LTV 와 CAC 의 관계는?

① LTV 가 CAC 보다 충분히 커야 한다
② CAC 가 LTV 보다 커야 한다
③ 둘이 같아야 한다
④ 관계 없다

Q5. 사용자를 무작위로 나눠 두 버전을 비교하는 실험은?

① 코호트 분석
② A/B 테스트
③ 교차표
④ 군집화

Q6. 1월 가입자, 2월 가입자를 따로 묶어 유지율 변화를 비교하는 분석은?

① 코호트 분석
② 상관분석
③ 회귀분석
④ 차원 축소

CHAPTER 08

데이터 분석 전체 흐름

학습 목표

- 데이터 분석 6단계(수집 → 전처리 → EDA → 피처 엔지니어링 → 모델 학습 → 평가·해석)를 말할 수 있다.
- 앞에서 배운 용어가 어느 단계에서 쓰이는지 연결한다.
- 작은 예제로 6단계를 처음부터 끝까지 따라 해 본다.

08.0 한눈에 보는 6단계

지금까지 배운 용어들은 따로 노는 게 아니라 **하나의 흐름** 안에서 차례로 등장해요.



▲ 데이터 분석 6단계 흐름

단계	하는 일	이 단계의 핵심 용어
01 수집	데이터 모으기	Dataset, CSV, SQL
02 전처리	데이터 다듬기	Missing Value, Scaling, Encoding
03 EDA	데이터 살펴보기	Distribution, Correlation
04 피처 엔지니어링	좋은 피처 고르기·만들기	Feature Selection, Feature Creation
05 모델 학습	규칙 배우기	Classification, Regression, Clustering
06 평가·해석	성적 매기고 의미 찾기	Accuracy, F1-score, RMSE, R^2

TIP 실제로는 한 번에 끝나지 않고 **왔다 갔다** 해요. 평가해 보니 점수가 낮으면 피처 엔지니어링이나 전처리로 돌아가요.

08.1 수집 · Data Collection

Data Collection
분석에 필요한 데이터를 모으는 단계

파일(CSV·엑셀), 데이터베이스(SQL), 웹(API·크롤링), 설문, 센서 등에서 데이터를 가져와요.

- **CSV**(Comma-Separated Values): 쉼표로 칸을 나눈 글자 파일. 가장 흔한 데이터 형식이에요.
- **SQL**: 데이터베이스에서 원하는 데이터를 꺼내는 언어예요. (`SELECT * FROM 주문`)

파이썬 코드

```
import pandas as pd
import io
# 실제로는 pd.read_csv("파일.csv") – 여기서는 글자를 파일처럼 읽기
csv_text = """이름,나이,구매액,재구매
김,25,30000,1
이,,45000,1
박,41,,0
최,33,52000,1
정,29,8000,0
한,52,61000,1
"""
df = pd.read_csv(io.StringIO(csv_text))
print(df)
```

실행 결과

```
  이름    나이      구매액  재구매
0  김  25.0  30000.0    1
1  이   NaN  45000.0    1
2  박  41.0      NaN    0
3  최  33.0  52000.0    1
4  정  29.0   8000.0    0
5  한  52.0  61000.0    1
```

08.2 전처리 단계

Preprocessing Step
결측치·스케일·인코딩 등으로 데이터를 다듬는 단계

3장에서 배운 결측치 처리, 스케일링(정규화·표준화), 인코딩을 하는 단계예요.

파이썬 코드

```
df["나이"] = df["나이"].fillna(df["나이"].median())       # 결측치 → 중앙값
df["구매액"] = df["구매액"].fillna(df["구매액"].median())
print(df.isnull().sum().sum(), "개 남은 결측치")
```

실행 결과

```
0 개 남은 결측치
```

TIP **Scaling(스케일링)** 은 정규화·표준화를 합쳐 부르는 말이에요. 값의 크기(scale)를 맞춘다는 뜻이에요.

08.3 EDA 단계

EDA Step
분포와 상관관계를 보며 데이터를 이해하는 단계

4장의 분포·상관을 확인해요. "재구매한 사람은 구매액이 더 큰가?" 같은 질문을 던져요.

파이썬 코드

```
print(df.groupby("재구매")["구매액"].mean())
print(df[["나이", "구매액", "재구매"]].corr().round(2)["재구매"])
```

실행 결과

```
재구매
0    26500.0
1    47000.0
Name: 구매액, dtype: float64
나이     0.04
구매액    0.56
재구매    1.00
Name: 재구매, dtype: float64
```

08.4 피처 엔지니어링 단계 · Feature Selection / Creation

Feature Selection / Creation
쓸모 있는 피처를 고르고(선택) 새로 만드는(생성) 단계

- **피처 선택(Selection)**: 예측에 도움 안 되는 열은 빼요. (이름은 재구매와 상관없으니 제외)
- **피처 생성(Creation)**: 새 열을 만들어요. (나이 → 연령대)

파이썬 코드

```
df["연령대"] = (df["나이"] // 10 * 10).astype(int)     # 생성: 25 → 20대
X = df[["나이", "구매액"]]                              # 선택: 이름은 뺌
y = df["재구매"]
print(X.join(df["연령대"]))
```

실행 결과

```
     나이      구매액  연령대
0  25.0  30000.0   20
1  33.0  45000.0   30
2  41.0  45000.0   40
3  33.0  52000.0   30
4  29.0   8000.0   20
5  52.0  61000.0   50
```

08.5 모델 학습 단계

Model Training
데이터로 모델이 규칙을 배우게 하는 단계

5장의 분류·회귀·군집화 중 문제에 맞는 것을 골라 `fit()` 으로 학습해요. 재구매 여부(0/1)는 **분류** 문제예요.

파이썬 코드

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression().fit(X, y)      # 학습
print(model.predict([[30, 50000], [45, 9000]]))   # 새 고객 2명 예측
```

실행 결과

```
[1 0]
```

TIP 이름에 Regression 이 들어 있지만 **LogisticRegression 은 분류 모델**이에요. "확률"을 회귀처럼 계산한 뒤 0/1 로 나누거든요. 헷갈리기 쉬운 이름이에요.

08.6 평가·해석 단계

Evaluation & Interpretation
성능을 재고 결과의 의미를 설명하는 단계

6장의 지표로 성능을 재고, **"그래서 무엇을 하면 되나?"** 를 말로 설명해요. 숫자를 보고하는 것에서 끝나지 않고, 행동으로 이어지는 결론을 내는 게 분석가의 일이에요.

파이썬 코드
```
from sklearn.metrics import accuracy_score
print("정확도", accuracy_score(y, model.predict(X)))   # 연습이라 훈련 데이터로 평가(실제로는 테스트 데이터로!)
```

실행 결과
```
정확도 1.0
```

TIP 해석 예: "구매액이 클수록 재구매 확률이 높다 → 첫 구매 금액을 올리는 쿠폰(예: 5만 원 이상 10% 할인)을 시험해 보자 → A/B 테스트로 확인하자."

총정리 퀴즈

Q1. 데이터 분석 흐름의 올바른 순서는?

① 수집 → 전처리 → EDA → 피처 엔지니어링 → 모델 학습 → 평가
② 수집 → 모델 학습 → 전처리 → 평가 → EDA
③ EDA → 수집 → 평가 → 전처리
④ 모델 학습 → 수집 → 평가

Q2. 결측치 처리와 인코딩은 어느 단계인가요?

① 수집
② 전처리
③ 평가
④ 모델 학습

Q3. '나이'로 '연령대' 열을 새로 만드는 것은?

① 피처 선택
② 피처 생성
③ 정규화
④ 평가

Q4. F1-score, RMSE 를 보는 단계는?

① EDA
② 수집
③ 평가·해석
④ 전처리

CHAPTER 09

파이썬·pandas 용어

학습 목표

- 코드에 자주 나오는 영어 단어(dtype, axis, inplace, delimiter …)의 뜻을 단어 풀이로 이해한다.
- Series 와 DataFrame 의 차이를 안다.
- axis=0 과 axis=1 의 방향을 헷갈리지 않는다.
- describe, query, agg 로 데이터를 빠르게 요약한다.

09.0 들어가며: 영어 단어를 알면 코드가 읽힌다

pandas 함수와 옵션 이름은 대부분 **평범한 영어 단어**예요. 단어 뜻을 알면 외우지 않아도 코드가 "읽혀요". 이 장은 단어를 **쪼개서 풀이**하는 방식으로 정리했어요.

파이썬 코드

```
import pandas as pd
import numpy as np
# 이 장에서 같이 쓰는 성적표
df = pd.DataFrame({
    "이름": ["민수", "지아", "도윤"],
    "국어": [80, 90, 70],
    "수학": [75, 85, 95],
    "영어": [85, 70, 68],
})
print(df)
```

실행 결과

```
   이름  국어  수학  영어
0  민수  80  75  85
1  지아  90  85  70
2  도윤  70  95  68
```

09.1 dtype · 자료형

dtype
data type, 열(또는 배열)에 들어 있는 값의 자료형

data + **type** = 데이터의 **종류**예요. 숫자인지(int64, float64), 글자인지(object/str), 참거짓인지(bool), 날짜인지(datetime64) 알려 줘요.

파이썬 코드

```
print(df.dtypes)
print(df["국어"].astype(float).dtype)     # 자료형 바꾸기: 정수 → 실수
```

실행 결과

```
이름      str
국어    int64
수학    int64
영어    int64
dtype: object
float64
```

TIP int**64** 의 64 는 숫자 하나를 저장하는 데 쓰는 칸(비트) 수예요. 숫자가 클수록 더 큰 수를 담을 수 있어요.

09.2 Series · 시리즈

Series
값과 인덱스로 이루어진 1차원 데이터 (엑셀의 한 열)

한 줄로 늘어선 값들에 **이름표(인덱스)** 가 붙은 1차원 데이터예요. 엑셀의 **한 열**과 같아요. 영어 series 는 "연속, 줄지어 있는 것"이라는 뜻이에요(드라마 시리즈처럼).

파이썬 코드
```
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s)
print(s["b"])              # 이름표로 값 꺼내기
```

실행 결과
```
a    10
b    20
c    30
dtype: int64
20
```

TIP 이름표(key)로 값(value)을 찾는다는 점에서 파이썬 딕셔너리, JSON 의 key-value 와 비슷해요.

09.3 DataFrame · 데이터프레임

DataFrame
행과 열로 이루어진 2차원 표 (Series 여러 개를 옆으로 붙인 것)

행과 열이 있는 표예요. 열 하나를 꺼내면 Series 가 돼요. data(데이터) + frame(틀) = "데이터를 담는 틀".

파이썬 코드
```
print(type(df))             # DataFrame
print(type(df["국어"]))      # 열 하나 = Series
print(df.shape)             # (행 수, 열 수)
```

실행 결과
```
<class 'pandas.DataFrame'>
<class 'pandas.Series'>
(3, 4)
```

09.4 axis · 축

axis
계산 방향 (0 = 행 방향 ↓ 위에서 아래로, 1 = 열 방향 → 옆으로)

axis 는 **"어느 방향으로 계산하나"** 예요. 가장 헷갈리는 옵션이니 그림으로 기억해요.

- **axis=0** (기본값): **↓ 위에서 아래로** 훑어요 → 결과가 **열마다** 하나 (과목별 합계)
- **axis=1**: **→ 옆으로** 훑어요 → 결과가 **행마다** 하나 (학생별 합계)



▲ axis=0 은 아래로, axis=1 은 옆으로

```
파이썬 코드
score = df.set_index("이름")
print(score.sum(axis=0))     # ↓ 과목별 합계
print(score.sum(axis=1))     # → 학생별 합계
```

```
실행 결과
국어    240
수학    255
영어    223
dtype: int64
이름
민수    240
지아    245
도윤    233
dtype: int64
```

TIP 기억법: **0 은 세로로 선 막대(|), 1 은 옆으로 누운 막대(—)** 를 떠올려 보세요. 또는 "axis=1 → 열(column) 이름을 따라 옆으로". `df.drop("국어", axis=1)` 이 열을 지우는 것도 같은 이치예요.

09.5 inplace · 제자리에서

inplace
True 면 원본을 직접 바꿈, False(기본)면 바뀐 복사본을 돌려줌

in + place = **"그 자리에서"** 라는 뜻이에요.

- `inplace=False` (기본값): 원본은 그대로, **바뀐 새 표를 돌려줘요.** → 변수에 다시 담아야 해요.
- `inplace=True`: **원본 자체를 바꾸고** 아무것도 돌려주지 않아요(None).

```
파이썬 코드
t = df.copy()
t2 = t.drop(columns="영어")              # 기본: 복사본을 돌려줌
print(t.columns.tolist(), t2.columns.tolist())
t.drop(columns="영어", inplace=True)      # 원본을 바꿈
print(t.columns.tolist())
```

실행 결과
```
['이름', '국어', '수학', '영어'] ['이름', '국어', '수학']
['이름', '국어', '수학']
```

주의 `df = df.drop(..., inplace=True)` 처럼 쓰면 df 가 **None** 이 돼 버려요! 요즘은 inplace 대신 `df = df.drop(...)` 처럼 **다시 담는 방식**을 더 권해요.

09.6 delimiter · 구분자

delimiter
파일에서 칸(값)과 칸을 나누는 기호

de + limit(경계) = **경계를 정해 주는 것**이에요. 글자 파일에서 칸을 나누는 기호예요. pandas 에서는 `sep` 이라는 이름도 같이 써요.

파일	구분자	예	
CSV	`,` 쉼표	민수,80,75	
TSV	`\t` 탭	민수(탭)80(탭)75	
기타	`;` `` ` ``	`` ` ``	민수;80;75

파이썬 코드
```
import io
text = "이름;점수\n민수;80\n지아;90"
print(pd.read_csv(io.StringIO(text), delimiter=";"))   # sep=";" 와 같음
```

실행 결과
```
   이름  점수
0  민수  80
1  지아  90
```

09.7 describe · 요약 통계

describe
개수·평균·표준편차·최솟값·사분위수·최댓값을 한 번에 보여 주는 함수

describe = **묘사하다, 설명하다.** 데이터를 한 줄로 "설명해 줘" 하면 요약 통계표가 나와요. 처음 데이터를 받으면 가장 먼저 해 보는 함수예요.

파이썬 코드
```
print(df.describe().round(1))            # 수치형 열 요약
print(df.describe(include="object"))     # 글자 열 요약: 개수, 고유값 수, 최빈값(top), 그 빈도(freq)
```

실행 결과

```
         국어    수학    영어
count   3.0   3.0   3.0
mean   80.0  85.0  74.3
std    10.0  10.0   9.3
min    70.0  75.0  68.0
25%    75.0  80.0  69.0
50%    80.0  85.0  70.0
75%    85.0  90.0  77.5
max    90.0  95.0  85.0
        이름
count    3
unique   3
top     민수
freq     1
```

TIP pandas 3 부터는 글자 열 자료형이 `str` 로 바뀌어서 `include="str"` 로 써도 돼요. 둘 다 동작하게 하려면 `include=["object", "string"]` 처럼 쓸 수도 있어요.

09.8 numeric · numeric_only

numeric_only
True 면 숫자 열만 골라 계산

numeric = **숫자의.** `numeric_only=True` 는 "**숫자 열만** 계산해 줘"라는 뜻이에요. 글자 열이 섞인 표에서 평균을 구하면 오류가 나는데, 이 옵션으로 글자 열을 건너뛰어요.

파이썬 코드

```
print(df.mean(numeric_only=True))        # '이름' 열은 건너뜀
```

실행 결과

```
국어    80.000000
수학    85.000000
영어    74.333333
dtype: float64
```

09.9 query · 질의

query
조건을 글자(문장)로 써서 행을 골라내는 함수

query = **질문, 질의.** 표에 "이런 조건인 행 보여 줘" 하고 묻는 거예요. 대괄호 조건보다 읽기 쉬워요.

파이썬 코드

```
print(df.query("수학 >= 80 and 국어 >= 80"))
print(df[(df["수학"] >= 80) & (df["국어"] >= 80)])   # 같은 결과 (대괄호 방식)
```

실행 결과

```
   이름  국어  수학  영어
1  지아  90  85  70
   이름  국어  수학  영어
1  지아  90  85  70
```

09.10 aggregation · agg

agg
여러 집계 함수(mean, sum, count, max, min)를 한 번에 적용

aggregate = **모으다, 합치다.** 여러 값을 모아 하나로 요약해요(3장 집계). `agg()` 에 함수 이름을 목록으로 주면 한 번에 여러 요약을 볼 수 있어요.

파이썬 코드
```
print(df[["국어", "수학"]].agg(["mean", "sum", "count", "max", "min"]))
```

실행 결과
```
          국어     수학
mean    80.0   85.0
sum    240.0  255.0
count    3.0    3.0
max     90.0   95.0
min     70.0   75.0
```

09.11 statistic · 통계치

statistic
데이터에서 계산한 하나의 요약 숫자 (평균, 검정 통계량 등)

statistic(단수) = 데이터로 계산한 **숫자 하나**(평균, 표준편차, t 값 …). statistic**s**(복수) = **통계학**이라는 학문. 검정 함수 결과에 `statistic` 과 `pvalue` 가 같이 나오는 경우가 많아요.

파이썬 코드
```
from scipy import stats
r = stats.ttest_ind([80, 90, 70], [75, 85, 95])   # 국어 vs 수학 평균 차이 검정
print(round(r.statistic, 3), round(r.pvalue, 3))
```

실행 결과
```
-0.612 0.573
```

09.12 matplotlib · 맷플롯립

matplotlib
파이썬의 대표 그래프(시각화) 라이브러리

Mat + **Plot** + **Lib** = MATLAB(공학 계산 프로그램) 스타일로 **그래프를 그리는 라이브러리**예요. 보통 `import matplotlib.pyplot as plt` 로 불러요.

파이썬 코드
```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.figure(figsize=(4, 3))
plt.bar(["KOR", "MATH", "ENG"], df[["국어", "수학", "영어"]].mean())
plt.title("average")
plt.show()
print("막대그래프 완성")
```

실행 결과
막대그래프 완성

TIP 앱에서 실행하면 그래프가 결과 창에 그림으로 보여요. 그래프 안 글자는 한글 글꼴이 없을 수 있어 영어로 썼어요.

09.13 scatter · 산점도

scatter
흩뿌리다 — 점을 흩뿌린 듯 찍는 산점도 함수

scatter = **흩뿌리다.** 씨앗을 뿌리듯 점을 흩뿌린 그래프라서 산점도(散點圖, 흩을 산·점 점)예요. `plt.scatter(x, y)` 로 그려요(4장 산점도).

파이썬 코드
```
plt.figure(figsize=(4, 3))
plt.scatter(df["국어"], df["수학"])
plt.xlabel("KOR"); plt.ylabel("MATH")
plt.show()
print("산점도 완성")
```

실행 결과
산점도 완성

09.14 prompt · 프롬프트

prompt
AI 에게 하는 요청(지시문)

prompt = **재촉하다, 일깨우다.** ChatGPT·Claude 같은 AI 에게 **입력하는 요청문**이에요. 데이터 분석에서도 AI 에 코드 작성·해석을 부탁할 때 프롬프트를 잘 쓰면 결과가 훨씬 좋아져요.

좋은 프롬프트의 예:

화면
```
pandas DataFrame df 에 '날짜', '매출' 열이 있어.
월별 매출 합계를 구하고 꺾은선 그래프로 그려 줘.
코드마다 한국어 주석을 달아 줘.
```

TIP 팁: **무엇을 가지고(데이터), 무엇을, 어떤 형태로** 원하는지 구체적으로 적어요. AI 가 만든 코드는 꼭 직접 실행해서 결과를 확인해요.

총정리 퀴즈

Q1. df.sum(axis=1) 의 결과는?

① 열(과목)마다 합계
② 행(학생)마다 합계
③ 전체 합계 하나
④ 오류

Q2. inplace=True 의 뜻은?

① 원본을 직접 바꾼다
② 복사본을 돌려준다
③ 결측치를 채운다
④ 숫자 열만 계산한다

Q3. TSV 파일의 구분자는?

① 쉼표 ,
② 탭 ₩t
③ 세미콜론 ;
④ 공백 두 칸

Q4. 엑셀의 한 열처럼 값과 인덱스로 된 1차원 데이터는?

① Series
② DataFrame
③ dtype
④ axis

Q5. 글자 열이 섞인 표에서 숫자 열만 평균 내는 옵션은?

① inplace=True
② axis=1
③ numeric_only=True
④ include='object'

Q6. df.query("수학 >= 80") 은 무엇을 하나요?

① 수학 열을 지운다
② 수학이 80 이상인 행만 고른다
③ 수학 평균을 구한다
④ 수학 열을 정렬한다

CHAPTER 10

통계 모델·분석 분야 용어

학습 목표

- Regression(회귀)이라는 이름의 유래와 뜻을 안다.
- 절편(Intercept)과 계수(Coefficient)를 회귀식에서 찾는다.
- statsmodels 의 OLS 결과와 Shapiro-Wilk 정규성 검정을 읽는다.
- KMeans 와 엘보 방법으로 군집 수를 정한다.
- 패널 데이터·금융 데이터·계량 분석이 무엇인지 안다.

10.1 Regression · 회귀

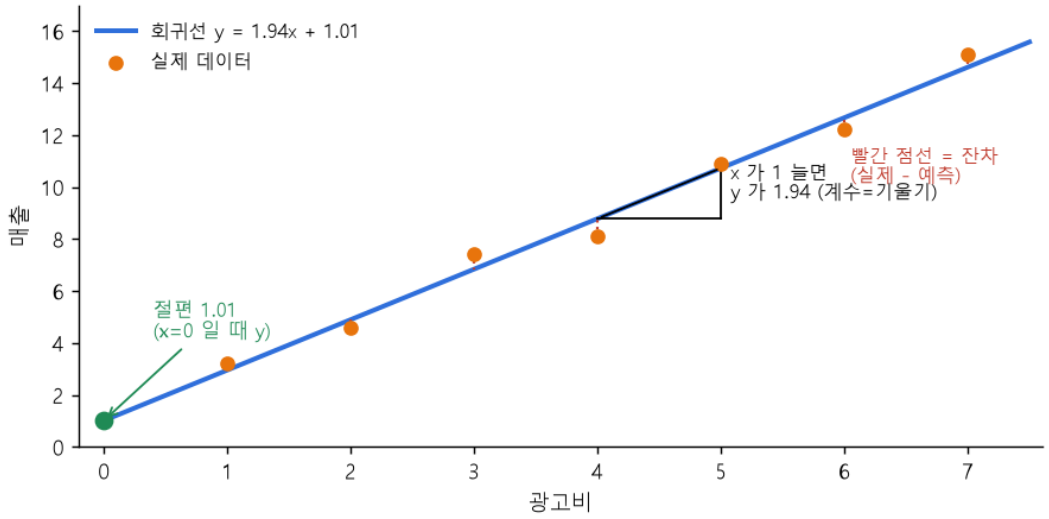
Regression

원래 뜻 '되돌아감' — 과거 데이터로 숫자 값을 짐작하는 방법

re(다시) + gress(가다) = **"되돌아가다"** 라는 뜻이에요. 그런데 왜 예측 방법에 이런 이름이 붙었을까요?

19세기 영국의 과학자 **골턴(Galton)** 은 부모와 자식의 키를 조사했어요. 그랬더니 아주 키 큰 부모의 자식은 부모보다 **조금 덜 크고**, 아주 작은 부모의 자식은 부모보다 **조금 더 커서**, 자식 키가 **평균 쪽으로 되돌아가는** 경향이 있었어요. 이것을 "평균으로의 회귀(regression toward the mean)"라고 불렀고, 이때 쓴 직선 맞추기 방법이 그대로 **회귀 분석**이라는 이름이 됐어요.

TIP 지금은 "회귀 = **과거 데이터의 관계를 이용해 숫자 값을 짐작**하는 방법" 으로 이해하면 충분해요. 예: 달걀(알) 무게로 병아리 무게를 짐작, 공부시간으로 점수를 짐작.



▲ 회귀선: 절편, 기울기(계수), 잔차

10.2 LinearRegression · 선형 회귀

LinearRegression

직선(y = ax + b)으로 관계를 나타내는 회귀 모델

Linear = **직선의.** 데이터에 가장 잘 맞는 **직선 y = ax + b** 를 찾아요. sklearn 의 `LinearRegression` 으로 만들어요.

파이썬 코드
```
from sklearn.linear_model import LinearRegression
import numpy as np

egg = np.array([[50], [55], [60], [65], [70]])   # 알 무게(g)
chick = np.array([33, 37, 40, 44, 47])           # 병아리 무게(g)
model = LinearRegression().fit(egg, chick)
print("기울기(계수) a =", model.coef_.round(3))
print("절편 b =", round(model.intercept_, 3))
print("알 62g → 병아리 약", model.predict([[62]]).round(1), "g")
```

실행 결과
```
기울기(계수) a = [0.7]
절편 b = -1.8
알 62g → 병아리 약 [41.6] g
```

10.3 Intercept · 절편

Intercept
x 가 0 일 때의 y 값, 직선이 y축과 만나는 점 (y = ax + b 의 b)

intercept = **가로채다, 길목에서 잡다.** 직선이 **y축을 가로지르며 만나는 지점**이에요. y = ax + **b** 의 **b** 예요. 그래프의 "시작점(출발점)" 으로 생각하면 쉬워요.

TIP 절편이 늘 현실적인 뜻을 갖는 건 아니에요. 위 예에서 절편은 "알 무게 0g 일 때 병아리 무게"라서 실제로는 의미가 없어요. 직선을 제자리에 놓기 위한 값이라고 보면 돼요.

10.4 Coefficient · 계수

Coefficient
변수 앞에 곱해지는 수 (기울기, 영향력)

co(함께) + efficient(작용하는) = **변수와 함께 작용하는 수**예요. y = **a**x + b 의 **a**, 즉 **기울기**예요. "x 가 1 늘면 y 가 a 만큼 늘어난다"로 읽어요.

변수가 여러 개인 **다중 회귀**($y = a_1x_1 + a_2x_2 + b$)에서는 계수가 각 변수의 **영향력**을 나타내요.

파이썬 코드
```
import pandas as pd
d = pd.DataFrame({"공부": [1, 2, 3, 4, 5, 6], "수면": [8, 7, 7, 6, 6, 5], "점수": [55, 61, 68, 72, 78, 83]})
m = LinearRegression().fit(d[["공부", "수면"]], d["점수"])
print(dict(zip(["공부", "수면"], m.coef_.round(2))), round(m.intercept_, 2))
```

실행 결과
```
{'공부': np.float64(6.25), '수면': np.float64(1.25)} 39.5
```

주의 단위가 다르면 계수 크기를 그대로 비교하면 안 돼요(시간 vs 원). 영향력을 비교하려면 **표준화한 뒤** 회귀해요.

10.5 statsmodels · 통계 모델 라이브러리

statsmodels
stats + models, 통계 모델과 검정을 자세한 표로 보여 주는 라이브러리

stats(통계) + **models**(모델). sklearn 이 "예측을 잘하는 것"에 집중한다면, statsmodels 는 **"결과를 통계적으로 설명하는 것"** 에 강해요. 계수마다 p-value, 신뢰구간, R^2 를 표로 보여 줘요. 경제·사회과학 연구에서 많이 써요.

10.6 OLS · 최소제곱법

OLS
Ordinary Least Squares, 잔차 제곱합이 가장 작은 직선을 찾는 방법

Ordinary(보통의) **L**east(가장 작은) **S**quares(제곱) = **"오차 제곱의 합이 가장 작은"** 직선을 찾는 방법이에요. **잔차(residual)** 는 실제 값과 직선의 차이(그림의 세로 점선)예요. 이걸 제곱해서 다 더한 값이 가장 작아지는 직선을 골라요. LinearRegression 도 내부에서 이 방법을 써요.

파이썬 코드
```
import statsmodels.api as sm
X = sm.add_constant(d[["공부", "수면"]])     # 절편(const) 열 추가
res = sm.OLS(d["점수"], X).fit()
print(res.params.round(2))                  # 절편과 계수
print("R² =", round(res.rsquared, 3))
print(res.pvalues.round(3))                 # 계수마다 p-value
```

실행 결과
```
const    39.50
공부        6.25
수면        1.25
dtype: float64
R² = 0.997
const    0.037
공부       0.003
수면       0.409
dtype: float64
```

TIP `res.summary()` 를 출력하면 큰 결과표가 나와요. 처음에는 **coef(계수), P>|t|(p-value), R-squared** 세 가지만 보면 돼요.

10.7 Shapiro-Wilk · 정규성 검정

Shapiro-Wilk
데이터가 정규분포를 따르는지 확인하는 검정

샤피로(Shapiro) 와 **윌크(Wilk)** 두 통계학자가 만든 검정이에요. "이 데이터는 정규분포다"를 가정하고 확인해요.

- p-value ≥ **0.05**: 정규분포라고 봐도 괜찮다
- p-value < **0.05**: 정규분포가 아닐 가능성이 크다

파이썬 코드

```
from scipy.stats import shapiro
rng = np.random.default_rng(3)
normal = rng.normal(170, 6, 50)        # 정규분포 키 데이터
skewed = rng.exponential(1, 50)        # 한쪽으로 치우친 데이터
print("정규:", round(shapiro(normal).pvalue, 3))
print("치우침:", f"{shapiro(skewed).pvalue:.1e}")   # 아주 작은 수 (e-05 = 0.00001 배)
```

실행 결과

```
정규: 0.199
치우침: 7.9e-06
```

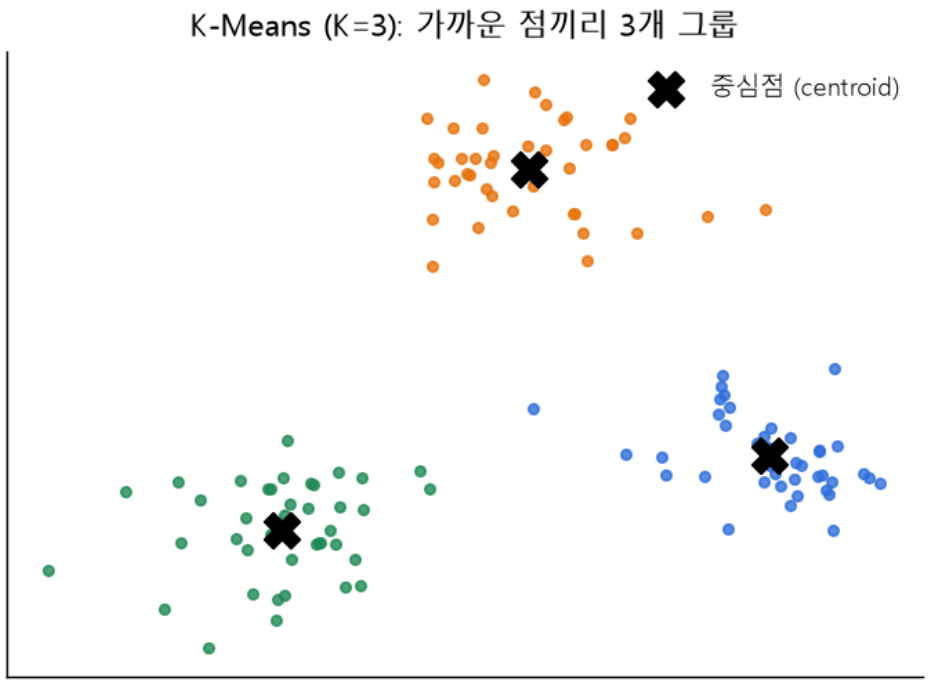
TIP t-검정, 선형 회귀 같은 많은 방법이 "정규분포"를 가정해서, 쓰기 전에 확인하는 거예요. 데이터가 아주 많으면(수천 개) 작은 차이도 잡아내니 히스토그램도 함께 봐요.

10.8 KMeans · K-평균 군집화

KMeans
K 개의 중심점(평균)을 기준으로 데이터를 K 개 그룹으로 나누는 방법

K = 나눌 **그룹 수**, **Means** = 각 그룹의 **중심점(평균)**. 중심점을 **센트로이드(centroid)** 라고 불러요.

1. 중심점 K 개를 아무 데나 놓는다
2. 각 점을 **가장 가까운 중심점** 그룹에 넣는다
3. 그룹마다 **평균 위치**로 중심점을 옮긴다
4. 더 이상 안 움직일 때까지 2~3 반복



▲ KMeans: 점들이 3개 그룹과 중심점(X)으로 나뉜 모습

파이썬 코드

```
from sklearn.cluster import KMeans
pts = np.array([[1, 2], [1, 4], [2, 3], [8, 8], [9, 9], [8, 10], [15, 2], [16, 1], [15, 3]])
km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(pts)
print(km.labels_)                       # 점마다 그룹 번호
print(km.cluster_centers_.round(1))     # 센트로이드
```

실행 결과

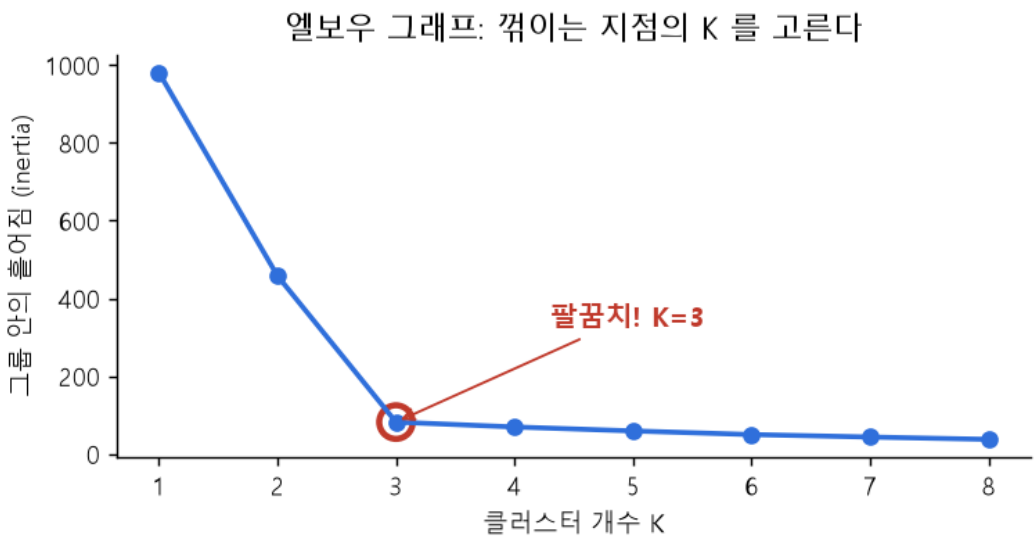
```
[2 2 2 0 0 0 1 1 1]
[[ 8.3  9. ]
 [15.3  2. ]
 [ 1.3  3. ]]
```

TIP 거리로 묶으니까 피처 단위가 다르면 꼭 **정규화·표준화**를 먼저 해요(3장).

10.9 Elbow · 엘보 방법

Elbow
팔꿈치처럼 꺾이는 지점으로 적당한 군집 수 K 를 고르는 방법

K 를 1, 2, 3 … 으로 늘리며 **그룹 안의 퍼짐(inertia, 점과 중심점 거리 제곱합)** 을 그래프로 그려요. K 가 커질수록 계속 줄어드는데, **팔꿈치(elbow)처럼 확 꺾이는 지점**부터는 줄어드는 폭이 작아요. 그 지점이 적당한 K 예요.



▲ 엘보 그래프: 꺾이는 지점이 적당한 K

파이썬 코드

```
for k in range(1, 6):
    print(k, round(KMeans(n_clusters=k, n_init=10, random_state=0).fit(pts).inertia_, 1))
```

실행 결과

```
1 388.0
2 135.5
3 8.0
4 5.8
5 4.2
```

- K 가 **너무 작으면**: 서로 다른 무리가 한 그룹에 섞여요.

- K 가 **너무 크면**: 같은 무리가 쓸데없이 쪼개져요(극단적으로 점 하나에 그룹 하나).

10.10 패널 데이터 · Panel Data

> **Panel Data**
> **여러 개체를 여러 시점에 걸쳐 반복 관찰한 데이터 (횡단면 + 시계열)**

여러 대상(사람·회사·나라)을 **여러 시점**에 걸쳐 **반복해서** 관찰한 데이터예요.

- **횡단면 데이터**: 한 시점에 여러 대상 (2026년 회사 100곳 매출)
- **시계열 데이터**: 한 대상의 여러 시점 (삼성 10년치 매출)
- **패널 데이터**: 둘을 합친 것 (회사 100곳 × 10년치 매출)

파이썬 코드
```
panel = pd.DataFrame({
    "회사": ["A", "A", "B", "B"],
    "연도": [2025, 2026, 2025, 2026],
    "매출": [100, 120, 80, 90],
}).set_index(["회사", "연도"])          # 대상 + 시점, 두 단계 인덱스
print(panel)
print(panel.groupby(level="회사").pct_change().round(2))   # 회사별 전년 대비 증가율
```

실행 결과
```
          매출
회사 연도
A  2025  100
   2026  120
B  2025   80
   2026   90
           매출
회사 연도
A  2025   NaN
   2026  0.20
B  2025   NaN
   2026  0.12
```

TIP 비유: 같은 학생들의 성적을 **매 학기 계속 기록한** 성적부예요.

10.11 금융 데이터 · Financial Data

> **Financial Data**
> **주가·환율·금리·재무제표 등 돈과 관련된 데이터**

주가, 환율, 금리, 재무제표(매출·이익·부채) 같은 돈과 관련된 데이터예요. 대부분 **시간 순서**가 중요한 시계열이라 날짜 처리와 **수익률**(전날 대비 변화율) 계산을 많이 해요.

파이썬 코드
```
price = pd.Series([10000, 10200, 9900, 10500], index=pd.date_range("2026-11-02", periods=4))
print(price.pct_change().round(4))      # 일간 수익률
print(price.rolling(2).mean())          # 2일 이동평균
```

실행 결과

```
2026-11-02       NaN
2026-11-03    0.0200
2026-11-04   -0.0294
2026-11-05    0.0606
Freq: D, dtype: float64
2026-11-02        NaN
2026-11-03    10100.0
2026-11-04    10050.0
2026-11-05    10200.0
Freq: D, dtype: float64
```

10.12 계량 분석 · Econometrics

> **Econometrics**
> **경제 이론을 데이터와 통계로 검증하는 분석**

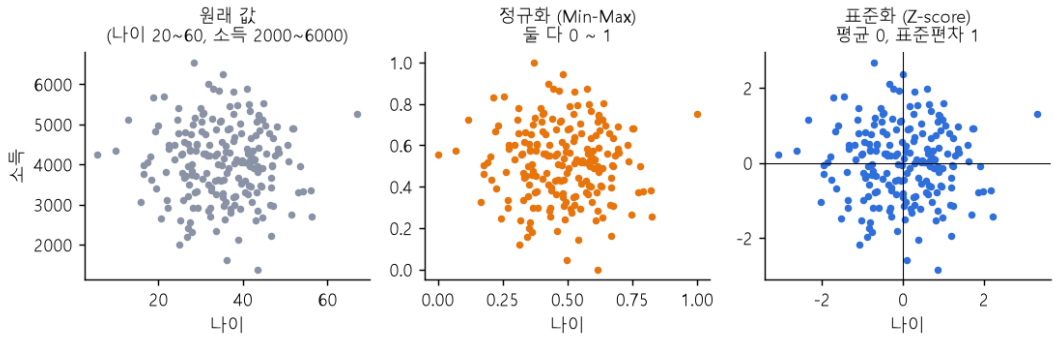
계량(計量, 셀 계·헤아릴 량) = 숫자로 **헤아려 잰다.** 경제학의 이론("금리가 오르면 소비가 줄어든다")이 실제 데이터에서도 맞는지 **통계로 검증**하는 분야예요. 계량경제학이라고도 해요. OLS 회귀, 패널 데이터 분석이 대표 도구예요.

10.13 정리: 표준화 vs 정규화

> **표준화 vs 정규화**
> **표준화 = 평균 기준 위치 맞추기, 정규화 = 0~1 범위 크기 맞추기**

헷갈리는 두 용어를 한 줄로 정리해요.

- **표준화**(Standardization): **평균을 기준으로 위치를 맞추기** → 평균 0, 표준편차 1
- **정규화**(Normalization): **범위를 0~1 로 크기 맞추기** → 최솟값 0, 최댓값 1



▲ 같은 데이터의 원래 값 · 정규화 · 표준화

파이썬 코드

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
x = np.array([[20], [30], [40], [60]])
print("표준화:", StandardScaler().fit_transform(x).ravel().round(2))
print("정규화:", MinMaxScaler().fit_transform(x).ravel().round(2))
```

실행 결과

```
표준화: [-1.18 -0.51  0.17  1.52]
정규화: [0.   0.25 0.5  1.  ]
```

주의 "정규화"는 분야에 따라 다른 뜻으로도 써요(데이터베이스 정규화, 모델 규제 regularization). 데이터 전처리에서는 보통 Min-Max 를 말해요.

총정리 퀴즈

Q1. Regression(회귀)이라는 이름이 생긴 계기는?

① 그래프가 원을 그리며 돌아서
② 자식의 키가 평균 쪽으로 되돌아가는 현상을 연구해서
③ 컴퓨터가 처음으로 돌아가서 다시 계산해서
④ 시간을 거꾸로 계산해서

Q2. y = 3x + 5 에서 절편은?

① 3
② 5
③ x
④ 8

Q3. y = 3x + 5 에서 x 가 1 늘면 y 는?

① 3 늘어난다
② 5 늘어난다
③ 8 늘어난다
④ 변하지 않는다

Q4. OLS 는 무엇을 가장 작게 만드나요?

① 절편
② 데이터 개수
③ 잔차 제곱의 합
④ 상관계수

Q5. Shapiro-Wilk 검정 p-value 가 0.001 이라면?

① 정규분포라고 봐도 된다
② 정규분포가 아닐 가능성이 크다
③ 데이터가 없다
④ 군집이 3개다

Q6. KMeans 의 K 는?

① 나눌 그룹(군집) 수
② 반복 횟수
③ 데이터 개수
④ 피처 개수

Q7. 회사 100곳의 매출을 10년간 기록한 데이터는?

① 횡단면 데이터
② 시계열 데이터
③ 패널 데이터
④ 범주형 데이터

CHAPTER 11

인공지능(딥러닝) 학습 용어

학습 목표

- 사전학습(Pre-training)과 미세조정(Fine-tuning)의 차이를 말할 수 있다.
- 토큰화·임베딩으로 AI 가 글을 숫자로 바꾸는 과정을 안다.
- 손실 함수·기울기·학습률·옵티마이저로 '학습'이 어떻게 일어나는지 그림으로 이해한다.
- 에포크·배치 크기·이터레이션을 계산할 수 있다.
- 분류기·감성 분석, Pydantic·SpooledTemporaryFile 같은 AI 서비스 용어를 안다.

11.0 들어가며: AI 는 어떻게 공부할까?

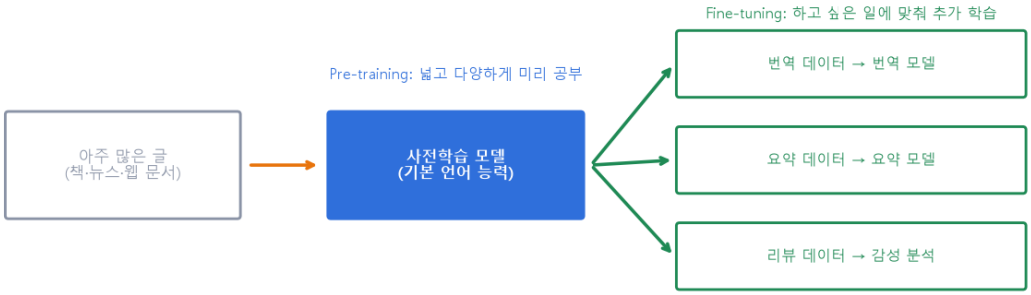
딥러닝(신경망) 모델의 공부는 **"예측해 보기 → 얼마나 틀렸는지 재기 → 덜 틀리게 고치기"** 의 반복이에요. 이 장의 용어들은 모두 이 반복의 어느 부분을 부르는 이름이에요.

단계	하는 일	관련 용어
글 준비	글을 조각내고 숫자로 바꾸기	토큰화, sentencepiece, 임베딩
예측	입력에서 출력까지 계산	순전파(Propagation)
채점	얼마나 틀렸나 재기	손실 함수, 엔트로피
고치기	덜 틀리게 가중치 조절	역전파, 기울기, 학습률, 최적화, 옵티마이저
반복	몇 번, 몇 개씩 공부할지	에포크, 배치 크기, 이터레이션

11.1 사전학습 · Pre-training

> **Pre-training**
> **AI 가 기본 언어 능력을 갖추도록 넓고 다양한 글로 미리 공부하는 것**

책·뉴스·웹 문서처럼 **아주 많고 다양한 글**로 먼저 공부시켜 **기본 언어 능력**(낱말 뜻, 문법, 상식)을 갖추게 하는 단계예요. 보통 "다음에 올 낱말 맞히기"를 수십억 번 반복해요. 이렇게 만든 모델을 **사전학습 모델(기반 모델, foundation model)** 이라 하고, ChatGPT·Claude 같은 대형 언어 모델(LLM)도 이 단계를 거쳐요.



▲ 사전학습으로 기본 능력을 만들고, 미세조정으로 일마다 맞춤 학습

TIP 비유: 초·중·고에서 **국어·수학·과학을 두루** 배우는 것. 어떤 직업을 갖든 바탕이 돼요.

11.2 미세조정 · Fine-tuning

Fine-tuning
사전학습 모델을 원하는 작업(번역·요약 등)에 맞춰 추가 학습하는 것

사전학습 모델에 **진짜 하고 싶은 일의 데이터**(번역 문장 쌍, 리뷰와 감정 등)를 조금 더 보여 줘서 **맞춤형으로 추가 학습**시키는 거예요. fine = 섬세한, tuning = 조율 → "악기 줄을 미세하게 맞추듯" 다듬는다는 뜻이에요.

TIP 비유: 고등학교까지 마친 사람이 **간호학과**에 들어가 전문 지식을 배우는 것.

사전학습 + 미세조정의 장점

- **더 빠르게**: 처음부터 가르치지 않으니 데이터와 시간이 훨씬 적게 들어요.
- **더 똑똑하게**: 이미 언어를 아는 상태라 적은 데이터로도 성능이 좋아요.
- 이렇게 이미 배운 지식을 다른 일에 옮겨 쓰는 것을 **전이 학습(Transfer Learning)** 이라 해요.

11.3 토큰 · 토큰화 · Token / Tokenization

Token
AI 모델이 글을 처리하는 가장 작은 단위(조각)

AI 는 글을 통째로 읽지 못하고 **토큰(조각)** 단위로 잘라서 처리해요. 이렇게 자르는 일을 **토큰화(tokenization)**, 자르는 도구를 **토크나이저(tokenizer)** 라고 해요. 토큰은 낱말일 수도, 낱말보다 작은 조각일 수도 있어요.

- 대략 영어는 **4글자 ≒ 1토큰**, 한글은 **1~2글자 ≒ 1토큰** 정도예요 (모델마다 달라요).
- AI 사용 요금, 한 번에 넣을 수 있는 글의 양(컨텍스트 윈도우, 12장)이 모두 **토큰 수**로 정해져요.

파이썬 코드
```
# 가장 단순한 토큰화: 띄어쓰기로 자르기 (실제 AI 는 더 작은 조각으로 자른다)
text = "데이터 분석은 재미있다"
tokens = text.split()
print(tokens, len(tokens), "개")
vocab = {w: i for i, w in enumerate(sorted(set(tokens)))}   # 조각마다 번호 붙이기
print([vocab[t] for t in tokens])                            # AI 에는 이 번호가 들어간다
```

실행 결과
```
['데이터', '분석은', '재미있다'] 3 개
[0, 1, 2]
```

11.4 sentencepiece · 센텐스피스

sentencepiece
문장을 낱말보다 작은 조각(subword)으로 쪼개는 토크나이저 도구

구글이 만든 **토크나이저 도구**예요. sentence(문장) + piece(조각) = 문장을 조각으로. 문장을 **낱말보다 더 작은 단위(subword)** 로 나눠서 AI 가 처리하기 쉽게 만들어 줘요. 띄어쓰기 자리는 _ 기호로 표시해요.



▲ 단어 단위 vs subword 조각

왜 더 작게 자를까?

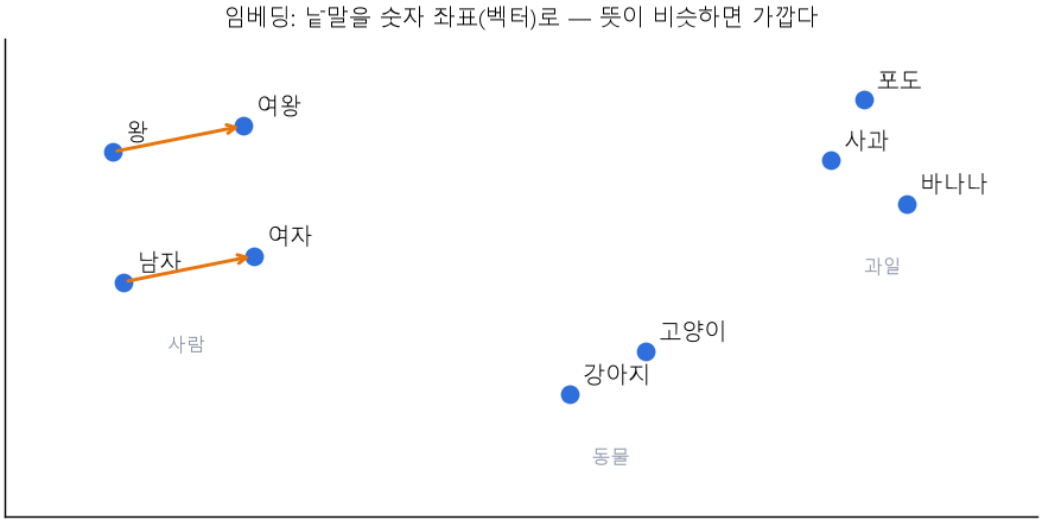
- "데이터분석은", "데이터분석을", "데이터분석이"를 모두 따로 외우면 낱말 사전이 끝없이 커져요.
- 데이터 + 분석 + 은 으로 나누면 **적은 조각으로 많은 낱말**을 표현하고, 처음 보는 낱말도 아는 조각으로 나눠 처리할 수 있어요.

```
파이썬 코드 (실행하지 않는 예시)
# 실제 사용 예 (pip install sentencepiece, 학습용 글 파일 필요 – 이 교안·앱에서는 실행하지 않아요)
import sentencepiece as spm
spm.SentencePieceTrainer.train(input="corpus.txt", model_prefix="ko", vocab_size=8000)   # 조각 사전 만들기
sp = spm.SentencePieceProcessor(model_file="ko.model")
print(sp.encode("데이터분석은 재미있다", out_type=str))   # 예: ['_데이터', '분석', '은', '_재미', '있다']
```

11.5 임베딩 · Embedding

> **Embedding**
> **글·이미지·소리 등을 의미가 담긴 숫자 벡터로 바꾸는 것**

토큰 번호만으로는 뜻을 알 수 없어요. 그래서 각 토큰을 **숫자 여러 개로 된 좌표(벡터)** 로 바꾸는데, **뜻이 비슷하면 가까운 곳**에 놓이도록 학습해요. 이것이 임베딩이에요. 문장·이미지·오디오도 임베딩할 수 있어요.



▲ 임베딩 공간: 비슷한 뜻은 가깝고, 관계(왕→여왕, 남자→여자)는 같은 방향

파이썬 코드

```
import numpy as np
# 아주 작은 가짜 임베딩 (실제는 수백~수천 개의 숫자)
emb = {"강아지": np.array([0.9, 0.8, 0.1]), "고양이": np.array([0.85, 0.75, 0.2]), "자동차": np.array([0.1, 0.2, 0.95])}
def cos(a, b):                                    # 코사인 유사도: 방향이 비슷할수록 1 에 가까움
    return round(float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b))), 3)
print("강아지-고양이", cos(emb["강아지"], emb["고양이"]))
print("강아지-자동차", cos(emb["강아지"], emb["자동차"]))
```

실행 결과

```
강아지-고양이 0.996
강아지-자동차 0.293
```

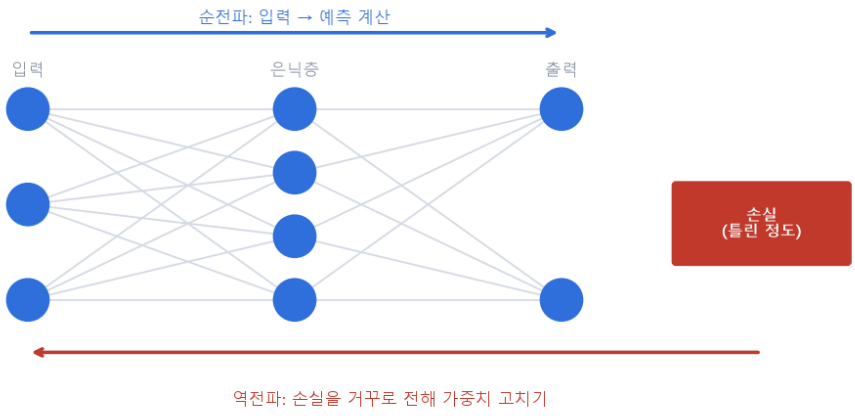
TIP 검색, 추천, RAG(12장)에서 "뜻이 비슷한 글 찾기"는 모두 임베딩 사이의 거리로 해요.

11.6 순전파 · 역전파 · Propagation

Propagation
전파 — 신경망에서 값(순전파)이나 오차(역전파)가 층을 따라 전해지는 것

propagation = **전파, 퍼져 나감.** 신경망에서는 두 방향이 있어요.

- **순전파(Forward Propagation)**: 입력 → 은닉층 → 출력으로 값을 전해 **예측을 계산**해요.
- **역전파(Backpropagation)**: 출력에서 생긴 **손실(틀린 정도)을 거꾸로** 전해, 각 가중치가 오차에 얼마나 책임이 있는지(기울기) 계산해요. 그 기울기로 가중치를 고쳐요.



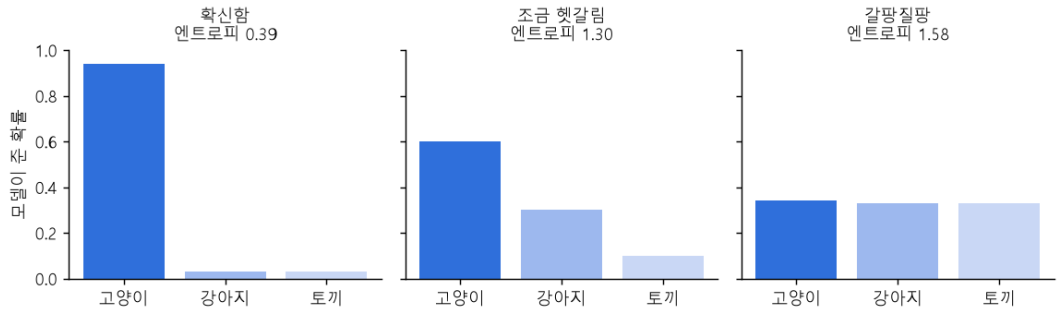
▲ 순전파로 예측하고, 역전파로 고친다

TIP 비유: 조별 과제 점수가 낮게 나오면(손실) **마지막 발표자부터 거꾸로** "누가 얼마나 잘못했지?"를 따져 각자 고치는 것.

11.7 엔트로피 · Entropy

Entropy
불확실성의 크기 — 모델이 얼마나 갈팡질팡하는지 재는 잣대

불확실한 정도를 숫자로 나타낸 거예요. 한 답에 확신하면 엔트로피가 **작고**, 여러 답 사이에서 갈팡질팡하면 **커요**.



▲ 확신할수록 엔트로피가 작다

파이썬 코드

```
import numpy as np
def entropy(p):                      # -Σ p·log2(p)
    p = np.array(p)
    return round(float(-(p * np.log2(p)).sum()), 3)
print("확신함   ", entropy([0.94, 0.03, 0.03]))
print("갈팡질팡 ", entropy([0.34, 0.33, 0.33]))
```

실행 결과

```
확신함    0.387
갈팡질팡  1.585
```

교차 엔트로피(Cross Entropy) 는 이 생각을 **손실 함수**로 쓴 거예요. 정답에 준 확률이 높을수록 손실이 작아져서, 분류 모델 학습에 가장 많이 써요.

파이썬 코드

```
# 정답이 '고양이'일 때, 모델이 고양이에 준 확률별 교차 엔트로피 손실 = -log(확률)
for p in [0.9, 0.5, 0.1]:
    print(f"고양이 확률 {p} → 손실 {-np.log(p):.2f}")
```

실행 결과

```
고양이 확률 0.9 → 손실 0.11
고양이 확률 0.5 → 손실 0.69
고양이 확률 0.1 → 손실 2.30
```

11.8 최적화 · Optimization

Optimization
손실 함수 값을 최소로 만들도록 가중치·편향을 조절해 가는 과정

최적화 = 가장 좋은(optimal) 상태로 만들기. 모델이 예측한 값과 실제 정답의 차이인 **손실 함수 값을 최소화**하도록, 모델 안의 **파라미터(가중치와 편향)** 를 조금씩 조절해 나가는 과정이에요.

TIP 비유 — 손실 함수는 **도달해야 할 골짜기 바닥**, 최적화는 **바닥을 향해 내려가는 행위 자체**, 옵티마이저는 **내려가는 등산법**(달리기, 미끄러지기, 조심조심 걷기 등)이에요.

최적화의 핵심 요소 3가지

요소	하는 일	질문으로 말하면
손실 함수 (Loss Function)	모델이 얼마나 틀렸는지 재는 기준 (예: MSE, Cross Entropy)	"지금 얼마나 틀렸나?"

요소	하는 일	질문으로 말하면
기울기 (Gradient)	손실이 줄어드는 방향을 알려 주는 지표	"어느 방향으로 가야 빨리 내려가나?"
학습률 (Learning Rate)	한 번에 얼마나 움직일지 정하는 보폭	"한 걸음을 얼마나 크게?"

11.8.1 손실 함수 · Loss Function

> **Loss Function**
> **모델의 예측이 정답과 얼마나 다른지(얼마나 틀렸는지) 재는 함수**

"현재 모델이 얼마나 틀렸나?"를 숫자 하나로 알려 주는 **채점 기준**이에요. 회귀에서는 **MSE**(6장, 오차 제곱 평균), 분류에서는 **교차 엔트로피**(11.7)를 주로 써요. 학습의 목표는 이 값을 **최대한 작게** 만드는 거예요.

파이썬 코드
```
import numpy as np
y_true = np.array([3.0, 5.0, 7.0])
for name, y_pred in [("엉터리 모델", np.array([6.0, 1.0, 9.0])), ("괜찮은 모델", np.array([3.2, 4.9, 6.8]))]:
    print(name, "MSE 손실 =", round(float(((y_true - y_pred) ** 2).mean()), 3))
```

실행 결과
```
엉터리 모델 MSE 손실 = 9.667
괜찮은 모델 MSE 손실 = 0.03
```

11.8.2 기울기 · Gradient

> **Gradient**
> **손실이 가장 가파르게 커지는 방향 — 반대쪽으로 가면 손실이 줄어든다**

손실 그래프에서 **지금 서 있는 곳의 경사**예요. 기울기가 양수면 오른쪽이 오르막이니 **왼쪽으로**, 음수면 **오른쪽으로** 가면 내려가요. 이렇게 기울기 반대 방향으로 조금씩 움직이는 방법이 **경사하강법(Gradient Descent)** 이에요.

새 가중치 = 지금 가중치 - 학습률 × 기울기

파이썬 코드
```
# 손실 = (w - 3)² + 1  →  바닥은 w = 3
w, lr = -1.0, 0.3
for step in range(6):
    grad = 2 * (w - 3)              # 손실을 w 로 미분한 값 = 기울기
    w = w - lr * grad               # 기울기 반대 방향으로 한 걸음
    print(f"{step + 1}걸음: w = {w:.3f}, 손실 = {(w - 3) ** 2 + 1:.3f}")
```

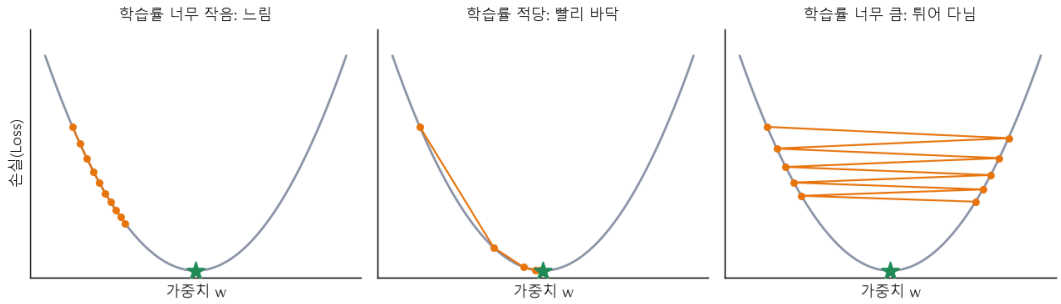
실행 결과
```
1걸음: w = 1.400, 손실 = 3.560
2걸음: w = 2.360, 손실 = 1.410
3걸음: w = 2.744, 손실 = 1.066
4걸음: w = 2.898, 손실 = 1.010
5걸음: w = 2.959, 손실 = 1.002
6걸음: w = 2.984, 손실 = 1.000
```

11.8.3 학습률 · Learning Rate

> **Learning Rate**
> **기울기 방향으로 한 번에 얼마나 움직일지 정하는 보폭**

한 걸음의 크기예요. 사람이 정하는 대표적인 **하이퍼파라미터**(5장)예요.

- **너무 작으면**: 바닥까지 한참 걸려요.
- **적당하면**: 빠르고 안정적으로 바닥에 가요.
- **너무 크면**: 바닥을 지나쳐 왔다 갔다 튀거나, 아예 멀리 날아가요(발산).



▲ 학습률 크기에 따른 경사하강법

파이썬 코드

```
for lr in [0.01, 0.3, 1.05]:
    w = -1.0
    for _ in range(20):
        w = w - lr * 2 * (w - 3)
    print(f"학습률 {lr}: 20걸음 뒤 w = {w:.2f}  (목표 3)")
```

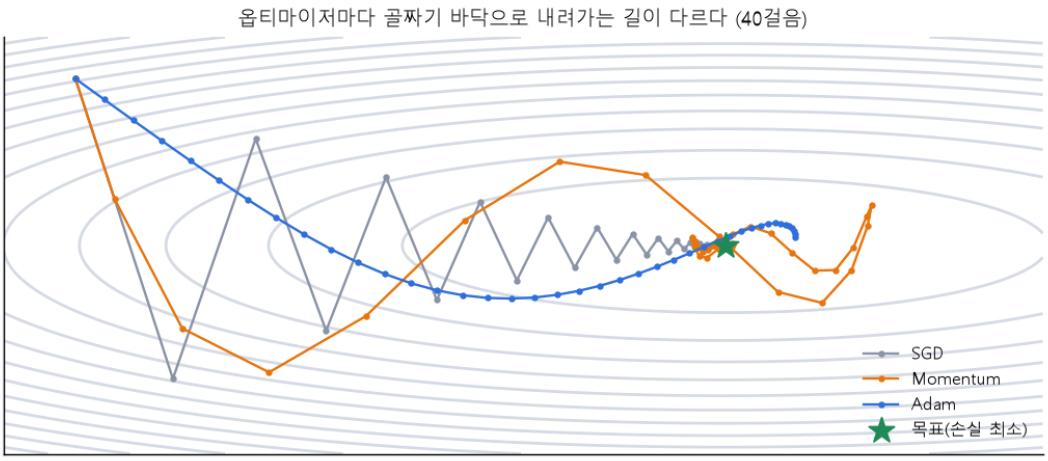
실행 결과

```
학습률 0.01: 20걸음 뒤 w = 0.33  (목표 3)
학습률 0.3: 20걸음 뒤 w = 3.00  (목표 3)
학습률 1.05: 20걸음 뒤 w = -23.91  (목표 3)
```

11.9 옵티마이저 · Optimizer

> **Optimizer**
> **최적화를 실제로 수행하는 알고리즘 — 방향과 보폭을 정해 손실 바닥으로 이끈다**

최적화를 **실제로 해 주는 주체(알고리즘)** 예요. 손실 함수가 '내려가야 할 산'이라면, 옵티마이저는 **산 아래로 내려가는 등산객의 전략**이에요. 기울기(방향)와 학습률(보폭)을 어떻게 쓸지 정해서 모델을 정답 쪽으로 이끌어요.



▲ 같은 골짜기에서도 옵티마이저마다 길이 다르다

종류	특징 (전략)	한 줄 비유
SGD	무작위로 고른 데이터로 빠르게 방향 결정	"일단 보이면 바로 뛰어라"
Momentum	가던 방향의 관성을 이용해 속도를 붙임	"내려오던 탄력을 받아라"
AdaGrad	많이 움직인 방향은 작게, 덜 움직인 방향은 크게	"모르는 길은 신중하게 가라"
RMSProp	AdaGrad 를 보완해 최근 기울기를 더 반영	"최근의 지형을 더 믿어라"
Adam	Momentum + RMSProp (가장 많이 사용)	"관성도 챙기고 보폭도 똑똑하게 조절해라"

아래 하위 항목에서 하나씩 살펴봐요.

11.9.1 SGD · 확률적 경사하강법

SGD
Stochastic Gradient Descent, 데이터 일부를 무작위로 골라 기울기를 구해 바로 움직이는 방법

Stochastic(확률적, 무작위의). 전체 데이터로 기울기를 구하면 너무 느리니까, **무작위로 고른 일부(배치)** 로 기울기를 구해 **바로 한 걸음** 움직여요. 빠르지만 걸음이 조금 **지그재그**예요.

TIP "일단 보이면 바로 뛰어라" — 전체 지도를 다 보지 않고 발밑 경사만 보고 바로 출발.

11.9.2 Momentum · 모멘텀

Momentum
이전에 가던 방향(관성)을 더해 속도를 붙이는 옵티마이저

모멘텀 = **관성 + 가속도.** 지금 기울기만 보지 않고 **지금까지 가던 방향**을 더해서 움직여요. 같은 방향이 계속되면 점점 빨라지고, 지그재그는 서로 상쇄돼 줄어들어요. 학습이라는 언덕을 내려갈 때 **가속도를 붙여** 더 효율적으로 목적지에 가게 해 줘요.

TIP 비유: 언덕을 굴러 내려가는 **공**. 작은 웅덩이(가짜 바닥)는 탄력으로 넘어가요.

파이썬 코드

```
# 1차원 예: 손실 = (w-3)², 같은 학습률로 SGD 와 Momentum 비교
def run(momentum, lr=0.05, steps=15):
    w, v = -1.0, 0.0
    for _ in range(steps):
        g = 2 * (w - 3)
        v = momentum * v - lr * g          # 속도 = 이전 속도의 일부 + 이번 기울기
        w = w + v
    return round(w, 3)
print("SGD     :", run(0.0))
print("Momentum:", run(0.8))
```

실행 결과

```
SGD     : 2.176
Momentum: 3.108
```

11.9.3 AdaGrad · 아다그라드

AdaGrad

Adaptive Gradient, 많이 움직인 변수는 보폭을 줄이고 덜 움직인 변수는 크게 하는 옵티마이저

Adaptive(적응하는) Gradient. 가중치마다 **지금까지 기울기 제곱을 누적**해서, 많이 움직인 가중치는 **보폭을 줄이고**, 거의 안 움직인 가중치는 **크게** 해 줘요. 단점은 누적만 하니 시간이 지나면 보폭이 너무 작아져 **멈춰 버릴 수** 있다는 거예요.

11.9.4 RMSProp · 알엠에스프롭

RMSProp

최근 기울기를 더 크게 반영해 AdaGrad 의 '멈춤' 문제를 고친 옵티마이저

AdaGrad 처럼 가중치마다 보폭을 조절하지만, 오래된 기울기는 **점점 잊고 최근 기울기를 더 믿어요**(지수 이동 평균). 그래서 보폭이 0 으로 줄어드는 문제가 없어요.

TIP "최근의 지형을 더 믿어라" — 오래전 지나온 길보다 지금 발밑이 중요해요.

11.9.5 Adam · 아담

Adam

Adaptive Moment Estimation, Momentum 과 RMSProp 을 합친 가장 많이 쓰는 옵티마이저

Momentum(관성) 과 **RMSProp(가중치마다 보폭 조절)** 의 장점을 합쳤어요. 대부분의 상황에서 잘 동작해서 딥러닝에서 **가장 먼저 써 보는 기본 옵티마이저**예요.

파이썬 코드

```
import numpy as np
# Adam 한 단계 공식 (m: 관성, s: 기울기 크기의 평균)
w, m, s, lr = -1.0, 0.0, 0.0, 0.1
for t in range(1, 61):
    g = 2 * (w - 3)
    m = 0.9 * m + 0.1 * g                # Momentum 부분
    s = 0.999 * s + 0.001 * g * g        # RMSProp 부분
    m_hat, s_hat = m / (1 - 0.9 ** t), s / (1 - 0.999 ** t)   # 초반 보정
    w = w - lr * m_hat / (np.sqrt(s_hat) + 1e-8)
    if t % 10 == 0:
        print(f"{t}걸음: w = {w:.3f}")      # 목표 3 에 점점 가까워짐
```

실행 결과

```
10걸음: w = -0.010
20걸음: w = 0.920
30걸음: w = 1.724
40걸음: w = 2.349
50걸음: w = 2.770
60걸음: w = 3.000
```

TIP PyTorch 에서는 `torch.optim.Adam(model.parameters(), lr=0.001)` 한 줄로 써요.

11.10 에포크 · Epoch

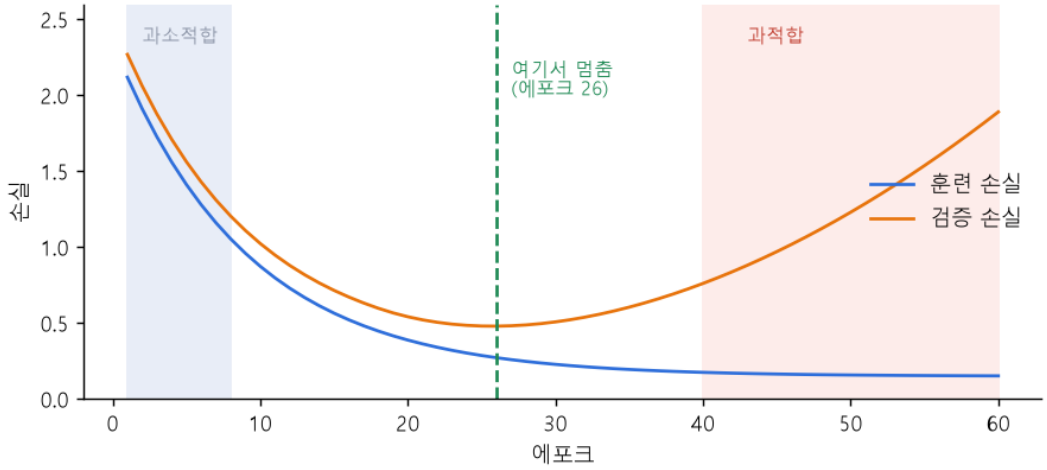
Epoch
학습 데이터 전체를 처음부터 끝까지 한 번 다 훑는 것 (전체 반복 횟수)

전체 데이터를 몇 번 반복해서 공부했나? 를 세는 단위예요. 데이터가 1,000개일 때 모델이 1,000개를 모두 한 번씩 공부하면 **1 에포크**예요.

- Epoch 1: 데이터를 처음 봄 (대략적인 특징 파악)
- Epoch 10: 반복 학습 (세부적인 특징과 오차 수정)
- Epoch 100: 숙련됨 (데이터를 거의 완벽하게 이해 — 너무 많으면 과적합!)

에포크 설정이 중요한 이유

에포크	결과	비유
너무 적으면	과소적합 (Underfitting): 패턴을 다 파악하지 못해 성능이 낮음	공부를 덜 한 학생
너무 많으면	과적합 (Overfitting): 훈련 데이터에만 집착, 새 문제는 못 풂	교과서를 통째로 외운 학생



▲ 에포크가 늘수록 훈련 손실은 계속 줄지만, 검증 손실은 다시 오른다

TIP 그래서 **검증 손실이 다시 오르기 시작하면 학습을 멈추는** 방법을 많이 써요. 이를 **조기 종료(Early Stopping)** 라고 해요.

11.11 배치 크기 · Batch Size

Batch Size
데이터를 몇 개씩 묶어서 한 번에 공부할지 정한 개수

데이터를 **몇 개씩 묶어서** 공부할지예요. 예: 1,000개를 100개씩 묶으면 배치 크기 100. 한 묶음을 보고 가중치를 한 번 고쳐요.

- 크게 하면: 한 번에 많이 봐서 안정적이지만 메모리(GPU)가 많이 필요해요.
- 작게 하면: 메모리는 적게 들지만 걸음이 들쭉날쭉해요.

11.12 이터레이션 · Iteration

Iteration
한 에포크를 끝내려고 가중치를 몇 번 업데이트했는지 (= 배치 개수)

배치 하나로 공부하고 가중치를 한 번 고치는 것이 **이터레이션 1번**이에요. 100개씩 10번 공부하면 1 에포크 완료 → **10 이터레이션**.

이터레이션 수 = 데이터 개수 ÷ 배치 크기 (나머지가 있으면 올림)



▲ 에포크 · 배치 · 이터레이션 한눈에 보기

파이썬 코드

```
import math
n_data, batch_size, epochs = 1000, 100, 5
iters = math.ceil(n_data / batch_size)
print("1 에포크 =", iters, "이터레이션")
print(epochs, "에포크 전체 =", iters * epochs, "번 가중치 업데이트")
print("배치 크기 64 이면:", math.ceil(n_data / 64), "이터레이션 (마지막 배치는", n_data % 64, "개)")
```

실행 결과

```
1 에포크 = 10 이터레이션
5 에포크 전체 = 50 번 가중치 업데이트
배치 크기 64 이면: 16 이터레이션 (마지막 배치는 40 개)
```

11.13 분류기 · Classifier

Classifier
데이터를 보고 어떤 그룹(카테고리)인지 판단해 주는 모델

classifier = **분류자, 분류기.** 데이터를 보고 **어느 그룹인지** 판단해 주는 모델이에요(5장 분류). 성능은 주로 **정확도(accuracy, 6장)** 로 재요.

- 스팸 메일 판별기 → 스팸인지 아닌지
- 감정 분석기 → 긍정인지 부정인지
- 이미지 분류기 → 고양이인지 개인지

TIP sklearn 의 분류 모델 이름은 대부분 `…Classifier` 로 끝나요. 예: `RandomForestClassifier`, `KNeighborsClassifier`

11.14 감성 분석 · Sentiment Analysis

Sentiment Analysis
글에서 긍정·부정·중립 같은 감정이나 의견을 자동으로 분석하는 기술

감성 분석(감정 분석) 은 상품 리뷰, 댓글, 설문 같은 글이 **긍정인지 부정인지(또는 중립인지)** 자동으로 분류하는 자연어처리(NLP) 기술이에요. 대표적인 분류기 사용 예예요.

- **neutral(뉴트럴, 중립)**: 긍정(positive)도 부정(negative)도 아닌 **중립적인 감정**. 예: "배송은 3일 걸렸어요."

파이썬 코드

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# 아주 작은 리뷰 학습 데이터 (실제로는 수천~수만 개)
reviews = ["정말 좋아요 최고", "배송 빠르고 좋아요", "최고 만족", "별로예요 실망", "최악 환불", "실망 별로",
           "보통이에요 그냥", "그냥 그래요 보통"]
labels = ["긍정", "긍정", "긍정", "부정", "부정", "부정", "중립", "중립"]
vec = CountVectorizer()                          # 글 → 낱말 개수 벡터
clf = MultinomialNB().fit(vec.fit_transform(reviews), labels)   # 분류기 학습
tests = ["좋아요 만족", "실망 최악", "그냥 보통"]
print(dict(zip(tests, clf.predict(vec.transform(tests)).tolist())))
```

실행 결과

```
{'좋아요 만족': '긍정', '실망 최악': '부정', '그냥 보통': '중립'}
```

11.15 Pydantic · 파이댄틱

Pydantic
파이썬 데이터의 형식을 검사(검증)하고 설정을 관리하는 라이브러리

AI 서비스는 사용자 입력이나 AI 답변(JSON)을 받아 처리해요. 이때 **"나이는 숫자여야 하고, 이메일은 꼭 있어야 한다"** 같은 규칙을 정해 두고 **자동으로 검사**해 주는 라이브러리예요. FastAPI(웹 API), LangChain, OpenAI·Anthropic SDK 에서 많이 써요.

파이썬 코드
```
from pydantic import BaseModel, ValidationError

class Review(BaseModel):          # 리뷰 데이터의 형식(설계도)
    text: str
    score: int                    # 정수여야 함
    sentiment: str = "중립"        # 기본값

print(Review(text="좋아요", score="5"))       # "5" → 5 로 자동 변환
try:
    Review(text="좋아요", score="다섯")         # 숫자가 아니면 오류
except ValidationError as e:
    print("검증 오류:", e.errors()[0]["loc"], e.errors()[0]["type"])
```

실행 결과
```
text='좋아요' score=5 sentiment='중립'
검증 오류: ('score',) int_parsing
```

TIP AI 에게 JSON 으로 답하게 한 뒤(12장 Format Constraint) Pydantic 으로 검사하면 형식이 틀린 답을 바로 걸러낼 수 있어요.

11.16 SpooledTemporaryFile · 스풀드 템퍼러리 파일

SpooledTemporaryFile
작을 땐 메모리에, 커지면 디스크 임시 파일에 저장하는 파일 객체

파이썬 표준 라이브러리 `tempfile` 에 있는 **임시 파일**이에요. 데이터가 **작으면 메모리(RAM)** 에 두어 빠르게 쓰고, **정해진 크기를 넘으면 자동으로 디스크**의 임시 파일로 옮겨요. 웹 서비스에서 사용자가 올린 파일(이미지·음성)을 잠깐 받아 둘 때 쓰여요(FastAPI 의 `UploadFile` 이 내부에서 사용).

spool = 실을 감아 두는 실패(실타래), spooled = 감아서 잠시 모아 둔, temporary = 임시의.

파이썬 코드
```
import tempfile
with tempfile.SpooledTemporaryFile(max_size=10) as f:   # 10바이트까지는 메모리
    f.write(b"hello")
    print("5바이트 → 디스크로 옮겨졌나?", f._rolled)
    f.write(b" world, AI!")
    print("16바이트 → 디스크로 옮겨졌나?", f._rolled)
    f.seek(0)
    print(f.read())
```

실행 결과
```
5바이트 → 디스크로 옮겨졌나? False
16바이트 → 디스크로 옮겨졌나? True
b'hello world, AI!'
```

총정리 퀴즈

Q1. 넓고 다양한 글로 기본 언어 능력을 먼저 갖추게 하는 단계는?

① Pre-training
② Fine-tuning
③ Iteration
④ Inference

Q2. sentencepiece 가 하는 일은?

① 문장의 감정을 분석한다
② 문장을 낱말보다 작은 조각(subword)으로 쪼갠다
③ 데이터 형식을 검사한다
④ 학습률을 정한다

Q3. 모델이 여러 답 사이에서 갈팡질팡할수록 커지는 값은?

① 정확도
② 학습률
③ 엔트로피
④ 배치 크기

Q4. 학습률이 너무 크면 생기는 일은?

① 학습이 아주 느려진다
② 바닥을 지나쳐 튀거나 발산한다
③ 데이터가 줄어든다
④ 에포크가 늘어난다

Q5. Momentum 과 RMSProp 을 합친, 가장 많이 쓰는 옵티마이저는?

① SGD
② AdaGrad
③ Adam
④ OLS

Q6. 데이터 2,000개, 배치 크기 100 이면 1 에포크는 몇 이터레이션?

① 100
② 20
③ 2,000
④ 1

Q7. 에포크를 너무 많이 하면 생길 수 있는 문제는?

① 과소적합
② 과적합
③ 결측치
④ 데이터 누수

Q8. "배송은 3일 걸렸어요" 처럼 긍정도 부정도 아닌 감정은?

① positive
② negative
③ neutral
④ outlier

CHAPTER 12

프롬프트 엔지니어링·생성형 AI 용어

학습 목표

- 프롬프트, 시스템 프롬프트, 사용자 프롬프트의 차이를 안다.
- Zero-shot·One-shot·Few-shot·Chain of Thought 를 상황에 맞게 고른다.
- 토큰·컨텍스트 윈도우·Temperature 가 답변에 주는 영향을 안다.
- Prompt Chaining·RAG 로 긴 문서와 회사 자료를 다루는 방법, 환각(Hallucination)을 줄이는 법을 안다.

12.0 들어가며: AI 에게 일 잘 시키기

ChatGPT·Claude 같은 **생성형 AI(대형 언어 모델, LLM)** 는 같은 모델이라도 **어떻게 묻느냐**에 따라 답의 품질이 크게 달라져요. 좋은 요청문을 설계하는 기술을 **프롬프트 엔지니어링(Prompt Engineering)** 이라고 해요. 데이터 분석에서도 코드 작성, 결과 해석, 리뷰 분류 등에 AI 를 많이 쓰니 꼭 알아 둘 용어들이에요.

TIP 프롬프트 예시는 AI 에게 보내는 글이라 실행 결과가 없어요. 직접 AI 에게 붙여 넣어 결과를 비교해 보세요.

12.1 프롬프트 · Prompt

Prompt
AI 모델에게 전달하는 입력 텍스트 (질문, 지시, 맥락, 예시 포함)

AI 에게 보내는 **모든 입력 글**이에요(9장 prompt). 좋은 프롬프트는 보통 네 가지가 들어 있어요.

- **지시**: 무엇을 해 달라는지 ("요약해 줘")
- **맥락**: 배경 정보 ("초등학생에게 설명할 거야")
- **입력 데이터**: 처리할 내용 (리뷰, 표, 코드)
- **출력 형식**: 어떤 모양으로 ("3줄로", "표로", "JSON 으로")

화면

```
[나쁜 예] 이 데이터 분석해 줘.
[좋은 예] 아래는 쇼핑몰 월별 매출 CSV 야. (입력 데이터)
          마케팅팀 회의에서 쓸 거야. (맥락)
          매출이 가장 많이 오른 달과 이유 후보 3가지를 찾아 줘. (지시)
          결과는 '달 ¦ 증가율 ¦ 이유 후보' 표로 줘. (출력 형식)
```

12.2 시스템 프롬프트 · System Prompt

System Prompt
대화 전에 AI 의 역할·규칙·출력 형식을 고정하는 최상위 지시문

대화가 시작되기 **전에** 정해 두는 **기본 설정 지시문**이에요. AI 의 역할, 지켜야 할 규칙, 말투, 출력 형식을 정해요. 사용자에게는 보통 보이지 않고, 대화 내내 적용돼요.

화면

```
[시스템] 너는 데이터 분석 입문자를 돕는 친절한 튜터야.
        - 전문 용어는 꼭 쉬운 말로 풀어 설명해.
        - 코드에는 한국어 주석을 달아.
        - 모르는 것은 모른다고 말해.
```

12.3 사용자 프롬프트 · User Prompt

User Prompt
사용자가 직접 입력하는 실제 요청 메시지

사용자가 **채팅창에 직접 입력하는** 질문·요청이에요. 시스템 프롬프트가 "무대 설정"이라면 사용자 프롬프트는 "이번 대사"예요.

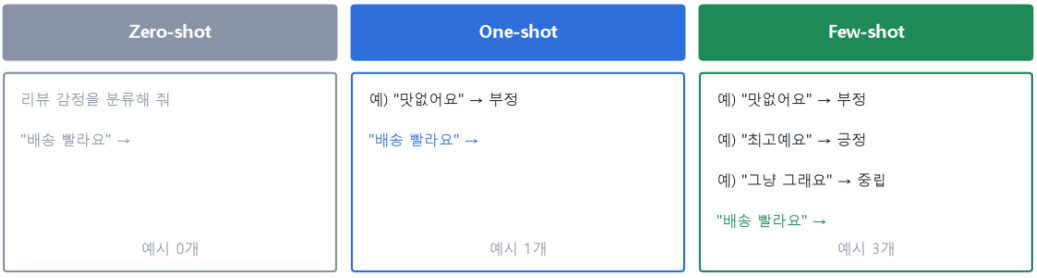
화면
[사용자] 표준편차가 뭔지 예를 들어 설명해 줘.

TIP API 로 AI 를 쓸 때는 `system`, `user`, `assistant`(AI 의 답) 역할을 나눠서 메시지를 보내요.

12.4 제로샷 · Zero-shot

Zero-shot
예시 없이 작업 설명만으로 AI 에게 요청하는 방식

예시를 하나도 주지 않고 설명만으로 시키는 방식이에요. 번역·요약처럼 AI 가 이미 잘 아는 간단한 일에 충분해요.



▲ 예시 개수에 따른 Zero-shot · One-shot · Few-shot

화면
다음 리뷰의 감정을 긍정/부정/중립 중 하나로 답해 줘.
리뷰: "배송이 빨라서 좋았어요"

12.5 원샷 · One-shot

One-shot
예시 1개를 주고 AI 가 패턴을 파악하게 하는 방식

예시를 딱 하나 보여 주는 방식이에요. 특히 **출력 형식**(표, JSON 모양)을 맞추고 싶을 때 효과가 좋아요.

화면
리뷰를 아래 형식으로 정리해 줘.
예시) 리뷰: "맛없어요" → {"감정": "부정", "키워드": "맛"}
리뷰: "배송이 빨라서 좋았어요" →

12.6 퓨샷 · Few-shot

Few-shot
예시 2~5개를 주어 정확도와 일관성을 높이는 방식

예시를 몇 개(보통 3~5개) 보여 주는 방식이에요. 분류·태깅·감성 분석처럼 **기준을 일관되게** 지켜야 하는 일에 좋아요. 예시는 여러 경우(긍정·부정·중립)를 골고루 넣어요.

```
화면
리뷰 감정을 분류해 줘.
"맛없어요" → 부정
"최고예요, 또 살게요" → 긍정
"그냥 그래요" → 중립
"배송이 빨라서 좋았어요" →
```

TIP 5장의 머신러닝은 수천 개 데이터로 **학습**하지만, Few-shot 은 모델을 고치지 않고 **프롬프트 안의 예시만으로** 따라 하게 해요. 이를 **문맥 내 학습(In-context Learning)** 이라고 해요.

12.7 생각의 사슬 · Chain of Thought

Chain of Thought
단계적으로 생각하라고 지시해 추론 품질을 높이는 기법

"**단계별로 생각해 보자**"처럼 **풀이 과정을 차례로 쓰게** 하는 기법이에요(CoT). 계산, 논리 추론, 의사결정 문제에서 실수가 줄어들어요. 사람도 암산보다 풀이를 쓰면 덜 틀리는 것과 같아요.

```
화면
가게 방문자 2,000명 중 50명이 샀고, 광고비는 30만 원, 객단가는 2만 원이야.
광고가 이익이었는지 판단해 줘. 전환율 → 매출 → 광고비와 비교 순서로 단계별로 계산해.
```

12.8 컨텍스트 윈도우 · Context Window

Context Window
모델이 한 번에 처리할 수 있는 최대 토큰 수

AI 가 **한 번에 읽고 기억할 수 있는 글의 양**이에요. **토큰 수**(11장)로 정해져요. 시스템 프롬프트 + 이전 대화 + 넣은 문서 + 질문 + 답변이 모두 이 칸 안에 들어가야 해요. 넘치면 앞부분을 잊거나 잘려요.



▲ 컨텍스트 윈도우 안에 들어가는 것들

TIP 그래서 아주 긴 문서는 나눠서 처리하거나(Prompt Chaining), 필요한 부분만 찾아서 넣어요(RAG).

12.9 토큰 수 계산 · Token

Token (LLM)
모델이 처리하는 최소 단위 — 영어 약 4글자, 한글 약 1~2글자가 1토큰

11.3 에서 본 토큰이 생성형 AI 에서는 **요금과 길이의 단위**가 돼요. 대략 **영어 약 4글자 ≒ 1토큰, 한글 약 1~2글자 ≒ 1토큰** 이에요(모델·토크나이저마다 다름). 같은 내용이면 보통 한글이 영어보다 토큰을 더 많이 써요.

파이썬 코드
```
# 아주 대략적인 토큰 수 어림 (정확한 값은 각 회사의 토큰 계산 도구로 확인)
def rough_tokens(text):
    ko = sum('가' <= ch <= '힣' for ch in text)        # 한글 글자 수
    other = len(text) - ko
    return round(ko / 1.5 + other / 4)
for t in ["Data analysis is fun.", "데이터 분석은 재미있다."]:
    print(f"{t!r}: 글자 {len(t)}개 → 약 {rough_tokens(t)} 토큰")
```

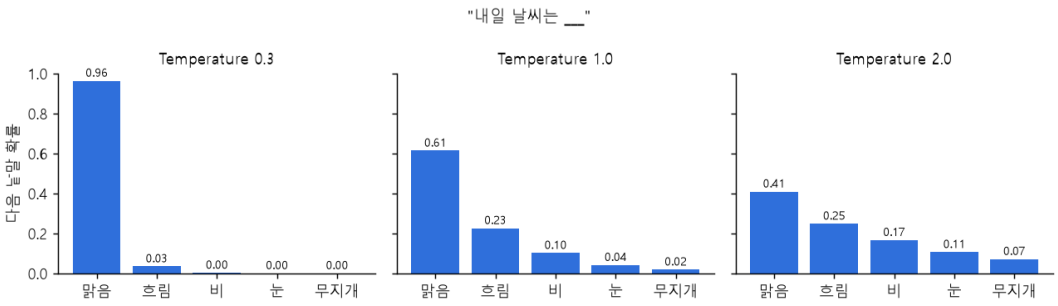
실행 결과
```
'Data analysis is fun.': 글자 21개 → 약 5 토큰
'데이터 분석은 재미있다.': 글자 13개 → 약 7 토큰
```

12.10 온도 · Temperature

Temperature
응답의 다양성을 조절하는 값 (0 = 결정적·일관, 1 이상 = 창의적·다양)

AI 는 다음 낱말을 **확률**로 골라요. Temperature 는 이 확률을 **얼마나 고르게 펼칠지** 정해요.

- **낮게(0~0.3)**: 가장 확률 높은 낱말만 골라 **늘 비슷한 답** → 분류, 데이터 추출, 코드
- **높게(0.8~1 이상)**: 덜 흔한 낱말도 골라 **다양하고 창의적인 답** → 아이디어, 글쓰기



▲ Temperature 에 따라 달라지는 다음 낱말 확률

파이썬 코드
```
import numpy as np
words = ["맑음", "흐림", "비", "눈", "무지개"]
scores = np.array([3.0, 2.0, 1.2, 0.3, -0.5])     # 모델이 낸 점수(logit)
for T in [0.3, 1.0, 2.0]:
    p = np.exp(scores / T); p = p / p.sum()       # 온도로 나눈 뒤 소프트맥스 → 확률
    print(f"T={T}:", dict(zip(words, p.round(2).tolist())))
```

실행 결과

```
T=0.3: {'맑음': 0.96, '흐림': 0.03, '비': 0.0, '눈': 0.0, '무지개': 0.0}
T=1.0: {'맑음': 0.61, '흐림': 0.23, '비': 0.1, '눈': 0.04, '무지개': 0.02}
T=2.0: {'맑음': 0.41, '흐림': 0.25, '비': 0.17, '눈': 0.11, '무지개': 0.07}
```

12.11 프롬프트 체이닝 · Prompt Chaining

Prompt Chaining
복잡한 작업을 여러 프롬프트로 나눠 순서대로 실행하는 기법

한 번에 다 시키지 않고 **작은 단계로 쪼개서**, 앞 단계의 결과를 다음 프롬프트에 넣어 **차례로(사슬처럼)** 실행해요. 긴 문서 처리, 보고서 작성처럼 복잡한 일에 좋고, 단계마다 결과를 확인할 수 있어요.

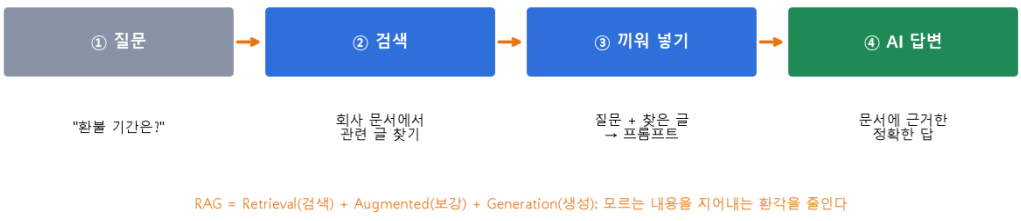
화면

```
1단계: 고객 리뷰 300개를 읽고 불만 주제를 5개로 묶어 줘.
2단계: (1단계 결과) 주제별로 대표 리뷰 2개와 개수를 표로 정리해 줘.
3단계: (2단계 표) 가장 시급한 주제 1개에 대한 개선안을 3줄로 써 줘.
```

12.12 RAG · 검색 증강 생성

RAG
Retrieval-Augmented Generation, 외부 문서를 검색해 프롬프트에 넣어 정확도를 높이는 방식

Retrieval(검색) + **A**ugmented(보강) + **G**eneration(생성). AI 가 모르는 내용(회사 규정, 최신 자료)을 묻기 전에 **관련 문서를 먼저 검색해서 프롬프트에 끼워 넣고** 답하게 해요. 근거 문서가 있으니 **지어내는 답(환각)**이 줄어요.



▲ RAG 의 4단계

파이썬 코드

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["환불은 구매 후 7일 이내에 신청할 수 있습니다.",
        "배송은 결제 후 2~3일 걸립니다.",
        "회원 등급은 구매 금액에 따라 매달 바뀝니다."]
question = "환불 신청 기간이 어떻게 되나요?"
vec = TfidfVectorizer(analyzer="char_wb", ngram_range=(2, 3)).fit(docs + [question])   # 글자 조각 기준 벡터
sims = cosine_similarity(vec.transform([question]), vec.transform(docs))[0]
best = docs[sims.argmax()]                                  # ② 가장 비슷한 문서 검색
print("유사도", sims.round(2))
print(f"③ 프롬프트: 다음 문서만 근거로 답해 줘.\n[문서] {best}\n[질문] {question}")
```

실행 결과
```
유사도 [0.11 0.   0.  ]
③ 프롬프트: 다음 문서만 근거로 답해 줘.
[문서] 환불은 구매 후 7일 이내에 신청할 수 있습니다.
[질문] 환불 신청 기간이 어떻게 되나요?
```

TIP 실제 RAG 는 11장의 **임베딩**으로 문서를 벡터로 바꿔 **벡터 데이터베이스**에 저장해 두고, 질문과 가장 가까운 문서를 찾아요.

12.13 환각 · Hallucination

Hallucination
AI 가 사실과 다른 내용을 그럴듯하고 자신 있게 만들어 내는 현상

hallucination = **환각, 헛것을 봄.** AI 가 없는 논문·통계·함수 이름을 **진짜처럼 자신 있게** 만들어 내는 현상이에요. AI 는 "사실"이 아니라 "그럴듯한 다음 낱말"을 고르기 때문에 생겨요.

줄이는 방법:

- 근거 자료를 함께 주고(RAG) **"자료에 없으면 모른다고 답해"** 라고 지시하기
- 출처를 함께 달라고 하고 **직접 확인하기**
- 숫자·코드는 **직접 실행·계산해 검증하기**
- Temperature 를 낮추기

주의 분석 보고서에 AI 가 준 숫자나 통계를 넣을 때는 반드시 원본 데이터로 다시 확인하세요.

12.14 역할 지정 · Role / Persona

Role / Persona
AI 에게 전문가 역할을 맡겨 관점과 말투를 일관되게 하는 기법

"너는 10년 차 데이터 분석가야"처럼 **역할(페르소나)** 을 정해 주면, 그 분야의 관점·용어·말투로 일관되게 답해요. 보통 시스템 프롬프트에 넣어요.

화면
```
너는 통계 교과서를 쓰는 교수야. 고등학생이 이해할 수 있게,
비유 1개와 계산 예시 1개를 넣어 p-value 를 설명해 줘.
```

12.15 출력 형식 제한 · Format Constraint

Format Constraint
답을 JSON·표 등 정해진 형식으로만 내도록 제한하는 기법

프로그램이 AI 의 답을 받아 자동으로 처리하려면(API 연동) 답의 **모양이 항상 같아야** 해요. "**JSON 으로만 답해**"처럼 형식을 정하고, 받은 뒤에는 11장의 **Pydantic** 같은 도구로 검사해요. 파싱(읽어 들이기) 실패가 줄어요.

파이썬 코드

```
import json
# AI 가 이런 형식으로 답하도록 프롬프트에서 요청했다고 하자
ai_answer = '{"감정": "긍정", "점수": 0.92, "키워드": ["배송", "빠름"]}'
data = json.loads(ai_answer)            # 형식이 맞으면 바로 파이썬 딕셔너리로
print(data["감정"], data["점수"], data["키워드"])
```

실행 결과

```
긍정 0.92 ['배송', '빠름']
```

12.16 프롬프트 라이브러리 · Prompt Library

Prompt Library
검증된 프롬프트를 목적별로 분류·저장해 두고 팀이 함께 쓰는 자원

잘 동작한 프롬프트를 **목적별로 모아 둔 모음집**이에요. "리뷰 감성 분류용", "SQL 작성용", "주간 보고서 요약용"처럼 정리해 두면 팀원 누구나 같은 품질의 결과를 얻을 수 있어요.

파이썬 코드

```
# 아주 간단한 프롬프트 라이브러리: 빈칸({})만 바꿔 다시 쓰기
LIBRARY = {
    "감성분류": "다음 리뷰의 감정을 긍정/부정/중립 중 하나로만 답해 줘.\n리뷰: {review}",
    "용어설명": "'{term}' 을(를) 데이터 분석 초보자에게 비유를 들어 3줄로 설명해 줘.",
}
print(LIBRARY["용어설명"].format(term="표준편차"))
```

실행 결과

```
'표준편차' 을(를) 데이터 분석 초보자에게 비유를 들어 3줄로 설명해 줘.
```

12.17 기법 선택 가이드

상황에 따라 어떤 기법을 쓰면 좋은지 정리했어요.

상황	추천 기법	이유
간단한 번역, 요약	Zero-shot	예시 없이도 충분
특정 형식 출력 (표, JSON)	One-shot + Format	형식 패턴 학습
분류, 태깅, 감성 분석	Few-shot (3~5개)	패턴 일관성 향상
복잡한 추론, 수학, 의사결정	Chain of Thought	단계적 사고 유도
긴 문서 처리, 분석	Prompt Chaining + RAG	컨텍스트 분산
전문가 관점 필요	Role / Persona	전문성·일관성 확보
API 연동 자동화	Format Constraint (JSON)	파싱 안정성 확보

총정리 퀴즈

Q1. 대화 전에 AI 의 역할·규칙·출력 형식을 고정하는 지시문은?

① System Prompt
② User Prompt
③ Token
④ Temperature

Q2. 예시 3~5개를 주고 패턴을 따라 하게 하는 방식은?

① Zero-shot
② One-shot
③ Few-shot
④ RAG

Q3. "단계별로 생각해 보자"라고 풀이 과정을 쓰게 하는 기법은?

① Prompt Library
② Chain of Thought
③ Temperature
④ Embedding

Q4. 늘 같은 형식의 일관된 답이 필요할 때 Temperature 는?

① 낮게 (0 가까이)
② 높게 (1 이상)
③ 상관없다
④ 음수로

Q5. 회사 문서를 검색해 프롬프트에 넣어 답하게 하는 방식은?

① Fine-tuning
② RAG
③ Zero-shot
④ Hallucination

Q6. AI 가 없는 통계를 사실처럼 자신 있게 만들어 내는 현상은?

① Overfitting
② Context Window
③ Hallucination
④ Iteration

Q7. 모델이 한 번에 처리할 수 있는 최대 토큰 수는?

① Context Window
② Batch Size
③ Epoch
④ Learning Rate

정답과 해설

01장 총정리 퀴즈

번호	정답	해설
Q1	②	맞혀야 하는 정답 열은 타깃(종속변수, y)이에요.
Q2	④	피처는 열(변수)이에요. 행은 관측치·레코드·샘플이라고 불러요.
Q3	③	숫자로 적혀 있어도 크기 비교가 의미 없는 우편번호는 범주형이에요.
Q4	①	결측치(Missing Value)는 pandas 에서 NaN 으로 보여요.
Q5	②	다른 값과 크게 동떨어진 값은 이상치예요.

02장 총정리 퀴즈

번호	정답	해설
Q1	②	중앙값은 가운데 값이라 이상치에 잘 흔들리지 않아요.
Q2	②	정렬한 가운데(3번째) 값 4, 가장 많이 나온 값 4 예요.
Q3	②	표준편차는 분산에 루트를 씌운 값이라 원래 단위로 읽을 수 있어요.
Q4	③	음수이면서 -1 에 가까우면 강한 음의 상관이에요. 인과관계는 따로 확인해야 해요.
Q5	①	오른쪽 꼬리가 길면 양의 왜도예요.
Q6	②	가운데 50% 가 퍼진 폭(Q3 - Q1)이에요. 이상치 판별에도 써요.

03장 총정리 퀴즈

번호	정답	해설
Q1	①	(x - 최솟값) / (최댓값 - 최솟값) 으로 0~1 로 맞춰요.
Q2	②	표준화(Z-score)는 평균 0, 표준편차 1 로 바꿔요.
Q3	②	순서가 없는 범주는 원-핫 인코딩이 숫자 크기 오해를 막아요.
Q4	②	groupby 로 그룹화하고 mean 으로 집계해요.
Q5	②	기준은 훈련 데이터로만 정해야 테스트 정보가 섞이지 않아요.

04장 총정리 퀴즈

번호	정답	해설
Q1	①	EDA(탐색적 데이터 분석)예요.
Q2	①	히스토그램은 구간별 개수로 분포를 보여 줘요.
Q3	②	산점도(scatter)예요.
Q4	②	상자 안의 선은 중앙값(Q2)이에요.
Q5	②	pd.crosstab 으로 만드는 교차표예요.

05장 총정리 퀴즈

번호	정답	해설
Q1	①	문제와 정답을 함께 주는 것이 지도학습이에요.
Q2	②	정답이 연속된 숫자이면 회귀예요.

번호	정답	해설
Q3	③	정답 없이 비슷한 것끼리 묶는 비지도학습, 군집화예요.
Q4	①	훈련 데이터만 외운 과적합이에요.
Q5	②	학습 전에 사람이 정하는 설정값이에요. 기울기는 모델이 배우는 파라미터예요.
Q6	③	테스트 데이터는 수능처럼 마지막에 한 번만 써요.
Q7	②	나무(tree)가 모인 숲(forest), 여러 트리의 다수결이에요.
Q8	③	DBSCAN 은 정답 없이 빽빽한 점끼리 묶는 군집 모델이에요.
Q9	①	확률을 계산해 범주를 고르는 분류 모델이에요.

06장 총정리 퀴즈

번호	정답	해설
Q1	②	양성(스팸)이라고 했는데(P) 틀렸으니(F) FP 예요.
Q2	②	실제 양성을 얼마나 찾았는지 보는 재현율이에요.
Q3	③	정확도는 쏠린 데이터에서 속기 쉬워요.
Q4	②	AUC 0.5 는 무작위로 찍는 것과 같아요.
Q5	②	절댓값 평균: (10 + 10 + 20) / 3 ≒ 13.3
Q6	②	R^2 0 은 평균값으로만 예측한 것과 같은 수준이에요.

07장 총정리 퀴즈

번호	정답	해설
Q1	②	50 / 2000 × 100 = 2.5%
Q2	①	이탈률 = 100% - 유지율
Q3	③	주문 한 건당 평균 금액, AOV 예요.
Q4	①	고객 한 명이 가져다주는 가치가 데려오는 비용보다 커야 해요.
Q5	②	A/B 테스트예요.
Q6	①	같은 시기 가입 그룹(코호트)을 비교하는 코호트 분석이에요.

08장 총정리 퀴즈

번호	정답	해설
Q1	①	모으고, 다듬고, 살펴보고, 피처를 만들고, 배우고, 평가해요.
Q2	②	데이터를 다듬는 전처리 단계예요.
Q3	②	기존 열로 새 열을 만드는 피처 생성(Creation)이에요.
Q4	③	성능을 재는 평가 단계예요.

09장 총정리 퀴즈

번호	정답	해설
Q1	②	axis=1 은 옆으로(→) 더해서 행마다 결과가 하나씩 나와요.
Q2	①	in place = 그 자리에서. 원본을 바꾸고 None 을 돌려줘요.
Q3	②	Tab-Separated Values, 탭으로 칸을 나눠요.

번호	정답	해설
Q4	①	Series 예요. 여러 개를 옆으로 붙이면 DataFrame 이 돼요.
Q5	③	numeric_only=True 는 숫자 열만 계산해요.
Q6	②	query 는 조건에 맞는 행을 골라내요.

10장 총정리 퀴즈

번호	정답	해설
Q1	②	골턴의 '평균으로의 회귀' 연구에서 이름이 왔어요.
Q2	②	y = ax + b 에서 b 가 절편이에요.
Q3	①	계수(기울기) 3 만큼 늘어나요.
Q4	③	Ordinary Least Squares, 잔차 제곱합을 최소로 해요.
Q5	②	p < 0.05 이면 정규분포 가정을 받아들이기 어려워요.
Q6	①	K 는 그룹 수, Means 는 그룹의 중심점(평균)이에요.
Q7	③	여러 대상 × 여러 시점, 패널 데이터예요.

11장 총정리 퀴즈

번호	정답	해설
Q1	①	사전학습(Pre-training)이에요. 그다음 원하는 일에 맞춰 미세조정(Fine-tuning)해요.
Q2	②	토크나이저 도구예요.
Q3	③	엔트로피는 불확실성의 크기예요.
Q4	②	보폭이 너무 커서 최솟값을 계속 지나쳐요.
Q5	③	Adam 이에요.
Q6	②	2000 ÷ 100 = 20 이터레이션이에요.
Q7	②	훈련 데이터를 통째로 외워 새 데이터에 약해지는 과적합이에요.
Q8	③	중립(neutral)이에요.

12장 총정리 퀴즈

번호	정답	해설
Q1	①	시스템 프롬프트예요. 사용자가 채팅창에 쓰는 것은 User Prompt 예요.
Q2	③	Few-shot 이에요.
Q3	②	Chain of Thought(생각의 사슬)예요.
Q4	①	낮을수록 결정적이고 일관된 답이 나와요.
Q5	②	Retrieval-Augmented Generation 이에요.
Q6	③	환각(Hallucination)이에요. 원본으로 꼭 확인하세요.
Q7	①	컨텍스트 윈도우예요.

용어 찾아보기

이 교안의 용어 161개를 가나다순으로 모았어요. 오른쪽 **절 번호**로 찾아가세요. (앱의 '용어 찾기' 탭에서는 검색도 돼요)

용어	영어	한 줄 뜻	절
감성 분석	Sentiment Analysis	글에서 긍정·부정·중립 같은 감정이나 의견을 자동으로 분석하는 기술	11.14
검색 증강 생성	RAG	Retrieval-Augmented Generation, 외부 문서를 검색해 프롬프트에 넣어 정확도를 높이는 방식	12.12
결정 트리	Decision Tree	예/아니오 질문을 이어 가며 답을 찾는 나무 모양 모델	05.1.4
결정계수	R^2	모델이 데이터의 변화를 얼마나 설명하는지 (보통 0 ~ 1)	06.10
결측치	Missing Value	값이 비어 있는(누락된) 데이터	01.8
계량 분석	Econometrics	경제 이론을 데이터와 통계로 검증하는 분석	10.12
계수	Coefficient	변수 앞에 곱해지는 수 (기울기, 영향력)	10.4
계층적 군집	Hierarchical Clustering	가장 가까운 것부터 차례로 합쳐 나무 모양(덴드로그램)을 만드는 군집	05.2.2
고객 생애 가치	LTV / CLV	Life Time Value, 한 고객이 거래하는 동안 가져다주는 총 가치	07.6
고객 획득 비용	CAC	Customer Acquisition Cost, 새 고객 한 명을 데려오는 데 드는 비용	07.7
과소적합	Underfitting	모델이 너무 단순해 데이터의 패턴을 제대로 못 배운 상태	05.9
과적합	Overfitting	훈련 데이터에만 너무 맞춰져 새 데이터에서 성능이 떨어지는 현상	05.8
관측치	Observation	데이터 한 행에 해당하는 관측 대상	01.4
교차표	Crosstab	두 범주형 변수의 조합별 개수를 센 표	04.7
구분자	delimiter	파일에서 칸(값)과 칸을 나누는 기호	09.6
군집화	Clustering	비슷한 데이터끼리 그룹으로 묶는 것	05.5
그레이디언트 부스팅	Gradient Boosting	앞 나무가 틀린 부분을 다음 나무가 고쳐 가며 쌓는 앙상블 모델	05.1.6
그룹화	Grouping	특정 기준으로 데이터를 묶는 일	03.8
금융 데이터	Financial Data	주가·환율·금리·재무제표 등 돈과 관련된 데이터	10.11
기울기	Gradient	손실이 가장 가파르게 커지는 방향 — 반대쪽으로 가면 손실이 줄어든다	11.8.2
나이브 베이즈	Naive Bayes	확률(베이즈 정리)로 가장 그럴듯한 범주를 고르는 분류 모델	05.1.8
데이터 누수	Data Leakage	학습에 쓰면 안 되는 정보가 모델에 섞여 들어가는 문제	03.11
데이터 병합	Merge / Join	여러 데이터를 공통 키로 이어 붙이는 일	03.7
데이터셋	Dataset	분석할 데이터 전체	01.1

용어	영어	한 줄 뜻	절
데이터프레임	DataFrame	행과 열로 이루어진 2차원 표 (Series 여러 개를 옆으로 붙인 것)	09.3
들어가며: 머신러닝이란?	Machine Learning	데이터에서 규칙을 스스로 찾아내는 방법	05.0
들어가며: 탐색적 데이터 분석	EDA	Exploratory Data Analysis, 데이터를 여러 각도로 살펴보는 첫 분석	04.0
랜덤 포레스트	Random Forest	결정 트리 여러 그루의 투표로 예측하는 앙상블 모델	05.1.5
레이블 인코딩	Label Encoding	범주를 0, 1, 2 같은 정수로 바꾸는 방법	03.6
로지스틱 회귀	Logistic Regression	확률(0~1)을 계산해 범주를 고르는 분류 모델	05.1.2
맷플롯립	matplotlib	파이썬의 대표 그래프(시각화) 라이브러리	09.12
모델 학습 단계	Model Training	데이터로 모델이 규칙을 배우게 하는 단계	08.5
모멘텀	Momentum	이전에 가던 방향(관성)을 더해 속도를 붙이는 옵티마이저	11.9.2
미세조정	Fine-tuning	사전학습 모델을 원하는 작업(번역·요약 등)에 맞춰 추가 학습하는 것	11.2
박스플롯	Box Plot	사분위수와 이상치를 상자 모양으로 나타낸 그래프	04.4
배치 크기	Batch Size	데이터를 몇 개씩 묶어서 한 번에 공부할지 정한 개수	11.11
백분위수	Percentile	데이터를 100 등분했을 때의 위치값	02.8
범주형 데이터	Categorical Data	성별, 지역, 상품 종류처럼 종류를 나타내는 데이터	01.6
변수	Variable	데이터의 속성 또는 특성	01.5
분류	Classification	정해진 범주 중 하나를 예측하는 문제	05.3
분류기	Classifier	데이터를 보고 어떤 그룹(카테고리)인지 판단해 주는 모델	11.13
분산	Variance	데이터가 평균에서 얼마나 퍼져 있는지 나타내는 값	02.4
분포	Distribution	데이터 값들이 어떻게 퍼져 있는지 나타낸 모양	04.1
비지도학습	Unsupervised Learning	정답 없이 데이터 속 구조를 찾는 방법	05.2
사분위수	Quartile	데이터를 4등분하는 경계값 (Q1·Q2·Q3)	02.7
사용자 프롬프트	User Prompt	사용자가 직접 입력하는 실제 요청 메시지	12.3
사용자당 평균 매출	ARPU	Average Revenue Per User, 사용자 한 명당 평균 매출	07.5
사전학습	Pre-training	AI 가 기본 언어 능력을 갖추도록 넓고 다양한 글로 미리 공부하는 것	11.1
산점도	Scatter Plot	두 수치형 변수의 관계를 점으로 찍은 그래프	04.3
산점도	scatter	흩뿌리다 — 점을 흩뿌린 듯 찍는 산점도 함수	09.13
상관계수	Correlation Coefficient	두 변수가 함께 움직이는 정도 (-1 ~ 1)	02.9
상관분석	Correlation Analysis	변수들 사이의 관계 정도를 분석하는 것	04.6
생각의 사슬	Chain of Thought	단계적으로 생각하라고 지시해 추론 품질을 높이는 기법	12.7
서포트 벡터 머신	SVM	Support Vector Machine, 두 그룹 사이의 간격(여백)이 가장 넓은 경계를 찾는 모델	05.1.7

용어	영어	한 줄 뜻	절
선형 회귀	Linear Regression	직선(y = ax + b)으로 숫자를 예측하는 가장 기본 회귀 모델	05.1.1
선형 회귀	LinearRegression	직선(y = ax + b)으로 관계를 나타내는 회귀 모델	10.2
센텐스피스	sentencepiece	문장을 낱말보다 작은 조각(subword)으로 쪼개는 토크나이저 도구	11.4
손실 함수	Loss Function	모델의 예측이 정답과 얼마나 다른지(얼마나 틀렸는지) 재는 함수	11.8.1
수집	Data Collection	분석에 필요한 데이터를 모으는 단계	08.1
수치형 데이터	Numerical Data	나이, 매출, 가격처럼 크기를 비교·계산할 수 있는 숫자 데이터	01.7
순전파 · 역전파	Propagation	전파 — 신경망에서 값(순전파)이나 오차(역전파)가 층을 따라 전해지는 것	11.6
스풀드 템퍼러리 파일	SpooledTemporaryFile	작을 땐 메모리에, 커지면 디스크 임시 파일에 저장하는 파일 객체	11.16
시리즈	Series	값과 인덱스로 이루어진 1차원 데이터 (엑셀의 한 열)	09.2
시스템 프롬프트	System Prompt	대화 전에 AI 의 역할·규칙·출력 형식을 고정하는 최상위 지시문	12.2
신경망	Neural Network	뇌의 신경세포를 본뜬 층(layer)을 쌓아 복잡한 패턴을 배우는 모델	05.1.9
아다그라드	AdaGrad	Adaptive Gradient, 많이 움직인 변수는 보폭을 줄이고 덜 움직인 변수는 크게 하는 옵티마이저	11.9.3
아담	Adam	Adaptive Moment Estimation, Momentum 과 RMSProp 을 합친 가장 많이 쓰는 옵티마이저	11.9.5
알엠에스프롭	RMSProp	최근 기울기를 더 크게 반영해 AdaGrad 의 '멈춤' 문제를 고친 옵티마이저	11.9.4
에포크	Epoch	학습 데이터 전체를 처음부터 끝까지 한 번 다 훑는 것 (전체 반복 횟수)	11.10
엔트로피	Entropy	불확실성의 크기 — 모델이 얼마나 갈팡질팡하는지 재는 잣대	11.7
엘보 방법	Elbow	팔꿈치처럼 꺾이는 지점으로 적당한 군집 수 K 를 고르는 방법	10.9
역할 지정	Role / Persona	AI 에게 전문가 역할을 맡겨 관점과 말투를 일관되게 하는 기법	12.14
연관 규칙	Association Rules	"A를 사면 B도 산다" 같은 함께 일어나는 규칙을 찾는 방법	05.2.6
온도	Temperature	응답의 다양성을 조절하는 값 (0 = 결정적·일관, 1 이상 = 창의적·다양)	12.10
옵티마이저	Optimizer	최적화를 실제로 수행하는 알고리즘 — 방향과 보폭을 정해 손실 바닥으로 이끈다	11.9
왜도	Skewness	분포가 좌우로 얼마나 비대칭인지	02.10
요약 통계	describe	개수·평균·표준편차·최솟값·사분위수·최댓값을 한 번에 보여 주는 함수	09.7
원-핫 인코딩	One-Hot Encoding	범주마다 0과 1로 된 열을 만드는 방법	03.5

용어	영어	한 줄 뜻	절
원샷	One-shot	예시 1개를 주고 AI 가 패턴을 파악하게 하는 방식	12.5
유지율	Retention Rate	일정 기간 뒤에도 계속 이용하는 고객의 비율	07.3
이상 탐지	Anomaly Detection	대부분과 다른 드문 데이터(이상치)를 찾아내는 방법	05.2.7
이상치	Outlier	다른 값들과 크게 차이 나는 데이터	01.9
이탈률	Churn Rate	일정 기간 동안 서비스를 떠난 고객의 비율	07.4
이터레이션	Iteration	한 에포크를 끝내려고 가중치를 몇 번 업데이트했는지 (= 배치 개수)	11.12
인코딩	Encoding	범주형 데이터를 숫자로 바꾸는 일	03.4
임베딩	Embedding	글·이미지·소리 등을 의미가 담긴 숫자 벡터로 바꾸는 것	11.5
자료형	dtype	data type, 열(또는 배열)에 들어 있는 값의 자료형	09.1
재현율	Recall	실제 양성 중 모델이 찾아낸 비율	06.4
전처리	Preprocessing	데이터를 분석하기 알맞게 다듬는 일	03.1
전처리 단계	Preprocessing Step	결측치·스케일·인코딩 등으로 데이터를 다듬는 단계	08.2
전환율	Conversion Rate	방문자 중 목표 행동(구매·가입 등)을 한 사람의 비율	07.2
절편	Intercept	x 가 0 일 때의 y 값, 직선이 y축과 만나는 점 (y = ax + b 의 b)	10.3
정규분포	Normal Distribution	평균을 중심으로 좌우 대칭인 종 모양 분포	02.12
정규성 검정	Shapiro-Wilk	데이터가 정규분포를 따르는지 확인하는 검정	10.7
정규화	Normalization	값을 일정한 범위(보통 0~1)로 바꾸는 일	03.2
정리: 표준화 vs 정규화	표준화 vs 정규화	표준화 = 평균 기준 위치 맞추기, 정규화 = 0~1 범위 크기 맞추기	10.13
정밀도	Precision	양성이라고 예측한 것 중 진짜 양성의 비율	06.3
정확도	Accuracy	전체 중 맞힌 비율	06.2
제로샷	Zero-shot	예시 없이 작업 설명만으로 AI 에게 요청하는 방식	12.4
제자리에서	inplace	True 면 원본을 직접 바꿈, False(기본)면 바뀐 복사본을 돌려줌	09.5
주성분 분석	PCA	Principal Component Analysis, 정보를 가장 많이 담은 새 축으로 차원을 줄이는 방법	05.2.4
중복 데이터	Duplicate Data	같은 정보가 반복해서 들어간 데이터	01.10
중앙값	Median	크기순으로 정렬했을 때 가운데 값	02.2
지도학습	Supervised Learning	정답(레이블)이 있는 데이터로 학습하는 방법	05.1
질의	query	조건을 글자(문장)로 써서 행을 골라내는 함수	09.9
집계	Aggregation	여러 값을 모아 합계·평균·개수 등 하나로 요약	03.9
차원 축소	Dimensionality Reduction	변수(차원) 개수를 줄이면서 정보는 최대한 남기는 것	05.6
첨도	Kurtosis	분포가 얼마나 뾰족하고 꼬리가 두꺼운지	02.11
최빈값	Mode	가장 자주 나타나는 값	02.3

용어	영어	한 줄 뜻	절
최소제곱법	OLS	Ordinary Least Squares, 잔차 제곱합이 가장 작은 직선을 찾는 방법	10.6
최솟값 · 최댓값	Min / Max	가장 작은 값 / 가장 큰 값	02.6
최적화	Optimization	손실 함수 값을 최소로 만들도록 가중치·편향을 조절해 가는 과정	11.8
축	axis	계산 방향 (0 = 행 방향 ↓ 위에서 아래로, 1 = 열 방향 → 옆으로)	09.4
출력 형식 제한	Format Constraint	답을 JSON·표 등 정해진 형식으로만 내도록 제한하는 기법	12.15
컨텍스트 윈도우	Context Window	모델이 한 번에 처리할 수 있는 최대 토큰 수	12.8
코호트 분석	Cohort Analysis	같은 시기·특징을 가진 고객 그룹을 묶어 시간에 따라 비교하는 분석	07.9
타깃	Target	예측하려는 정답 변수	01.3
토큰 · 토큰화	Token	AI 모델이 글을 처리하는 가장 작은 단위(조각)	11.3
토큰 수 계산	Token (LLM)	모델이 처리하는 최소 단위 — 영어 약 4글자, 한글 약 1~2글자가 1토큰	12.9
통계 모델 라이브러리	statsmodels	stats + models, 통계 모델과 검정을 자세한 표로 보여 주는 라이브러리	10.5
통계치	statistic	데이터에서 계산한 하나의 요약 숫자 (평균, 검정 통계량 등)	09.11
파이댄틱	Pydantic	파이썬 데이터의 형식을 검사(검증)하고 설정을 관리하는 라이브러리	11.15
패널 데이터	Panel Data	여러 개체를 여러 시점에 걸쳐 반복 관찰한 데이터 (횡단면 + 시계열)	10.10
평가·해석 단계	Evaluation & Interpretation	성능을 재고 결과의 의미를 설명하는 단계	08.6
평균	Mean	데이터 합계를 개수로 나눈 값	02.1
평균 절대 오차	MAE	Mean Absolute Error, 오차 절댓값의 평균	06.7
평균 제곱 오차	MSE	Mean Squared Error, 오차 제곱의 평균	06.8
평균 제곱근 오차	RMSE	Root Mean Squared Error, MSE 의 제곱근	06.9
평균 주문 금액	AOV	Average Order Value, 주문 한 건당 평균 금액	07.8
표준편차	Standard Deviation	분산의 제곱근 (평균에서 보통 얼마나 떨어져 있나)	02.5
표준화	Standardization	평균 0, 표준편차 1 이 되도록 바꾸는 일	03.3
퓨샷	Few-shot	예시 2~5개를 주어 정확도와 일관성을 높이는 방식	12.6
프롬프트	prompt	AI 에게 하는 요청(지시문)	09.14
프롬프트	Prompt	AI 모델에게 전달하는 입력 텍스트 (질문, 지시, 맥락, 예시 포함)	12.1
프롬프트 라이브러리	Prompt Library	검증된 프롬프트를 목적별로 분류·저장해 두고 팀이 함께 쓰는 자원	12.16
프롬프트 체이닝	Prompt Chaining	복잡한 작업을 여러 프롬프트로 나눠 순서대로 실행하는 기법	12.11

용어	영어	한 줄 뜻	절
피처	Feature	분석·예측에 사용하는 입력 변수	01.2
피처 엔지니어링	Feature Engineering	예측에 도움이 되는 새 변수를 만드는 일	03.10
피처 엔지니어링 단계	Feature Selection / Creation	쓸모 있는 피처를 고르고(선택) 새로 만드는(생성) 단계	08.4
하이퍼파라미터	Hyperparameter	학습 전에 사람이 정해 주는 모델 설정값	05.10
학습률	Learning Rate	기울기 방향으로 한 번에 얼마나 움직일지 정하는 보폭	11.8.3
핵심성과지표	KPI	Key Performance Indicator, 목표 달성 정도를 재는 핵심 지표	07.1
혼동행렬	Confusion Matrix	실제 정답과 예측 결과를 2×2 표로 정리한 것	06.1
확률적 경사하강법	SGD	Stochastic Gradient Descent, 데이터 일부를 무작위로 골라 기울기를 구해 바로 움직이는 방법	11.9.1
환각	Hallucination	AI 가 사실과 다른 내용을 그럴듯하고 자신 있게 만들어 내는 현상	12.13
회귀	Regression	연속된 숫자 값을 예측하는 문제	05.4
회귀	Regression	원래 뜻 '되돌아감' — 과거 데이터로 숫자 값을 짐작하는 방법	10.1
훈련 / 검증 / 테스트 데이터	Train / Validation / Test	학습용 / 조정용 / 최종 평가용으로 나눈 데이터	05.7
히스토그램	Histogram	구간별 데이터 개수를 막대로 나타낸 그래프	04.2
히트맵	Heatmap	숫자 크기를 색의 진하기로 나타낸 표 그래프	04.5
A/B 테스트	A/B Test	두 가지 버전을 무작위로 나눠 보여 주고 어느 쪽이 나은지 비교하는 실험	07.10
aggregation	agg	여러 집계 함수(mean, sum, count, max, min)를 한 번에 적용	09.10
DBSCAN	DBSCAN	점이 빽빽하게 모인 곳을 따라 묶고, 외딴 점은 이상치로 남기는 군집	05.2.3
EDA 단계	EDA Step	분포와 상관관계를 보며 데이터를 이해하는 단계	08.3
F1-score	F1-score	정밀도와 재현율의 조화평균	06.5
K-최근접 이웃	KNN	K-Nearest Neighbors, 가장 가까운 이웃 K개의 다수결로 예측	05.1.3
K-평균 군집	K-Means	중심점 K개를 기준으로 가까운 데이터끼리 묶는 군집 모델	05.2.1
K-평균 군집화	KMeans	K 개의 중심점(평균)을 기준으로 데이터를 K 개 그룹으로 나누는 방법	10.8
numeric	numeric_only	True 면 숫자 열만 골라 계산	09.8
ROC-AUC	ROC-AUC	분류 모델이 양성과 음성을 얼마나 잘 구별하는지 (0.5 ~ 1)	06.6
t-SNE	t-SNE	고차원 데이터를 2~3차원 그림으로 보여 주는 시각화용 차원 축소	05.2.5